Explainable AI for Business Intelligence in a Multi-Tenant Enterprise Resource Planning System: SHAP and LIME Explanations for ML-Driven Decisions

Prepared by:

Ahmed Ameen Hassan Alkuraimi



State University of Malang

A Graduation Project Submitted in Partial Fulfillment of the Requirements for the Degree of

bachelor of Informatics Engineering

Supervised by:

Dr.Eng Muhammad Ashar , S.T., M.T.

In the Name of Allah, the Most Gracious,
the Most Merciful

Dedication

I dedicate this work to my beloved parents, whose unwavering support and encouragement have been my greatest source of strength throughout my academic journey. To my family and friends who have stood by me with patience and understanding. And to all those who believe in the power of technology to transform businesses and create a better future.

Acknowledgments

First and foremost, I thank **Allah (SWT)** for granting me the strength, knowledge, and perseverance to complete this graduation project.

I would like to express my sincere gratitude to my supervisor, **Ashar Pakkawaru**, for the continuous guidance, valuable feedback, and encouragement throughout this research. Your expertise and mentorship have been instrumental in shaping this work.

I am deeply grateful to the **Faculty of Computer Science and Information Technology** at **State University of Malang** for providing an enriching academic environment and the resources necessary to undertake this project.

Special thanks to my colleagues and friends who provided constructive criticism, participated in testing, and offered their insights during the development process.

Finally, I extend my heartfelt appreciation to my **family**, especially my parents, for their unconditional love, support, and prayers that have carried me through every challenge.

Abstract

Enterprise Resource Planning (ERP) systems serve as the backbone of modern organizational management by integrating core business processes into a unified platform. As machine learning (ML) models are increasingly embedded in ERP workflows to automate business decisions, a critical challenge emerges: these models operate as opaque "black boxes," producing predictions without justification. Business users—managers approving credit terms, HR officers prioritising retention interventions, sales teams qualifying leads—cannot act confidently on predictions they do not understand. Explainable Artificial Intelligence (XAI) addresses this gap by making model reasoning transparent, auditable, and actionable.

This graduation project presents the design, implementation, and evaluation of an **Explainable AI framework** within a **multi-tenant ERP system** built on Django 4.2 with PostgreSQL database-per-tenant isolation. Primarily designed as a **small enterprise implementation**, the system focuses on the unique needs of **middle to small SMEs**, bringing advanced automation and insights to resource-constrained environments. The system integrates **eight ML models** spanning five business domains and augments three classifiers—Late Payment Prediction (Gradient Boosting), Lead Scoring (Logistic Regression), and Employee Attrition Risk (Random Forest)—with dual post-hoc explanation methods: **SHAP** (SHapley Additive exPlanations) via TreeExplainer for exact Shapley-value attribution, and **LIME** (Local Interpretable Model-agnostic Explanations) as a model-agnostic comparison baseline. The XAI module provides both *instance-level* explanations (why was this specific invoice flagged as high-risk?) and *global* feature importance rankings (which factors drive late payments across all customers?), with interactive waterfall, beeswarm, and force-plot visualisations delivered through the ERP's web interface.

The system was developed following the **Sprint (Agile)** methodology across eight two-week iterations. The architecture features over 80 domain models organised into nine functional modules (including Finance, HR, Inventory/WMS, **CRM**, **POS**, Fixed Assets, Suppliers, Reporting, and AI), a complete REST API, and a real-time AI Insight notification engine. Multi-tenancy is achieved through dynamic PostgreSQL database provisioning via Docker containers or remote server connections, ensuring complete data isolation between organisations while keeping infrastructure costs low for SMEs.

Experimental evaluation on a tenant dataset comprising 98,478 invoices, 5,200 leads, and

1,720 employees demonstrates: (1) Late Payment Prediction achieves AUC-ROC = **0.6358** with well-distributed SHAP contributions across 18 features (top feature accounts for only 27% of total importance), confirming the model captures genuine multi-factor risk signals; (2) Lead Scoring achieves AUC-ROC = **0.9448**, with SHAP analysis revealing that CRM interaction metrics like `activity_count` dominate the predictive logic, confirming that human engagement remains the strongest conversion driver; (3) Employee Attrition achieves AUC-ROC = **0.9761**, with SHAP identifying `absence_rate` and comparative compensation metrics as the premier leading indicators of early departure. All nine tested instances across three models produce valid SHAP waterfall decompositions and LIME explanations with matching prediction scores. Cross-method comparison shows 2-of-5 top-feature overlap between SHAP and LIME, consistent with known divergences between Shapley-value and locally-weighted linear surrogate approaches (Slack et al., 2020).

The key finding is that **XAI bridges the trust gap in enterprise automation**: by quantifying per-feature contributions, SHAP and LIME prove that the underlying ML models correctly model real-world operational rules rather than arbitrary proxy biases—demonstrating that explainability is essential for responsible and trustworthy ML deployment in enterprise systems.

Keywords: Explainable AI, SHAP, LIME, ERP System, Machine Learning, Multi-Tenant Architecture, Django, Business Intelligence, Feature Attribution, Model Interpretability, Agile Development.

Contents

List of Abbreviations **viii**

1 Introduction **1**
1.1 Overview . . . 1
1.2 Problem Statement . . . 2
1.3 Project Objectives . . . 3
1.4 Project Scope . . . 4
1.4.1 In Scope . . . 5
1.4.2 Out of Scope . . . 6
1.5 Project Significance . . . 6
1.6 Project Questions . . . 7
1.7 Report Organization . . . 8

2 Literature Review **9**
2.1 Introduction . . . 9
2.2 Enterprise Resource Planning (ERP) Systems . . . 9
2.2.1 Definition and Evolution . . . 9
2.2.2 ERP Architecture Models . . . 10
2.2.3 ERP Adoption in SMEs . . . 10
2.2.4 Open-Source ERP Systems . . . 11
2.3 Multi-Tenant Software Architecture . . . 11
2.3.1 Definition and Models . . . 11
2.3.2 Multi-Tenancy in Django . . . 12
2.4 Machine Learning in Enterprise Systems . . . 13
2.4.1 Overview of ML in Business Applications . . . 13
2.4.2 Lead Scoring with Logistic Regression . . . 13
2.4.3 Payment Risk Prediction with Gradient Boosting . . . 14
2.4.4 Employee Attrition Prediction with Random Forest . . . 14
2.4.5 Anomaly Detection with Isolation Forest . . . 15
2.4.6 Time-Series Forecasting with Prophet . . . 15
2.4.7 Statistical Process Control for KPI Monitoring . . . 15
2.5 Explainable Artificial Intelligence (XAI) . . . 16

2.5.1 The Need for Explainability . . . 16
2.5.2 Taxonomy of XAI Methods . . . 16
2.5.3 SHAP: SHapley Additive exPlanations . . . 17
2.5.4 LIME: Local Interpretable Model-agnostic Explanations . . . 17
2.5.5 SHAP vs. LIME: Complementary Strengths . . . 18
2.5.6 XAI as a Model Validation Tool . . . 19
2.6 Related Work . . . 19
2.6.1 AI-Enhanced ERP Systems . . . 19
2.6.2 XAI in Business Decision Systems . . . 19
2.6.3 Multi-Tenant Django Applications . . . 20
2.6.4 ML for Demand Forecasting in Supply Chain . . . 20
2.7 Gap Analysis . . . 21
2.8 Summary . . . 21

3 Methodology **23**
3.1 Introduction . . . 23
3.2 Development Methodology: Sprint (Agile) . . . 23
3.2.1 Overview of Sprint Methodology . . . 23
3.2.2 Sprint Plan for This Project . . . 24
3.2.3 Sprint Roles . . . 26
3.2.4 Justification for Sprint Methodology . . . 26
3.3 System Architecture . . . 27
3.3.1 High-Level Architecture . . . 27
3.3.2 Multi-Tenant Database Architecture . . . 28
3.3.3 AI/ML Pipeline Architecture . . . 29
3.4 Technology Stack . . . 31
3.5 Data Collection and Preparation . . . 33
3.5.1 Open-Source Datasets . . . 33
3.5.2 Synthetic Data Generation . . . 34
3.6 Machine Learning Model Design . . . 34
3.6.1 Model Selection Criteria . . . 34
3.6.2 Model Specifications . . . 35
3.6.3 Model Evaluation Strategy . . . 36
3.6.4 XAI Evaluation Methodology . . . 37
3.7 Development Tools and Environment . . . 38
3.8 Summary . . . 38

4 System Analysis and Design **40**
4.1 Introduction . . . 40
4.2 Functional Requirements . . . 40

4.2.1 Platform Requirements . . . 40
4.2.2 AI/XAI Requirements . . . 41
4.3 Non-Functional Requirements . . . 41
4.4 Use Case Design . . . 42
4.4.1 XAI Use Cases . . . 42
4.5 XAI Class Design . . . 43
4.6 Database Schema Design . . . 44
4.6.1 AI/XAI Schema . . . 44
4.7 XAI Pipeline Architecture . . . 45
4.8 Implemented System Capabilities . . . 46
4.8.1 Multi-Tenant Platform (`tenants/` app) . . . 47
4.8.2 Core ERP Domain Models (70+ Django Models) . . . 47
4.8.3 REST and Mobile API Surface . . . 48
4.8.4 Server-Rendered Views and Templates . . . 49
4.8.5 Services Layer and Cross-Cutting Concerns . . . 50
4.8.6 AI / ML Stack . . . 50
4.8.7 Reporting, Analytics and Document Generation . . . 51
4.8.8 Management Commands and Data Tooling . . . 52
4.8.9 Mobile Companion App (Flutter, `mobile/`) . . . 52
4.8.10 Localisation and Accessibility . . . 53
4.8.11 Audit, Security and Observability . . . 53
4.8.12 Testing . . . 54
4.8.13 Deployment and Operations . . . 54
4.9 Summary . . . 54

5 Implementation and Testing **56**
5.1 Introduction . . . 56
5.2 Project Structure . . . 56
5.3 ExplainabilityService Implementation . . . 57
5.3.1 SHAP TreeExplainer Integration . . . 57
5.3.2 LIME TabularExplainer Integration . . . 58
5.3.3 Global Feature Importance . . . 58
5.3.4 Version Compatibility Handling . . . 59
5.4 Multi-Tenant Implementation . . . 59
5.4.1 Dynamic Database Registration . . . 59
5.4.2 XAI in Multi-Tenant Context . . . 59
5.5 User Interface and System Screenshots . . . 60
5.5.1 Landing Page and Dashboards . . . 60
5.5.2 Tenant Administration and Governance . . . 61

5.5.3 Finance and Accounting . . . 64
5.5.4 Operations: Inventory, POS and Services . . . 66
5.5.5 Human Resources and Attendance . . . 68
5.5.6 AI/ML and Explainable AI Modules . . . 69
5.5.7 Flutter Mobile Companion . . . 70
5.6 Testing Strategy . . . 72
5.6.1 ML Model Testing . . . 72
5.6.2 XAI Testing . . . 73
5.7 Summary . . . 73

6 Results and Discussion **74**
6.1 Introduction . . . 74
6.2 ML Model Performance . . . 74
6.2.1 The Late-Payment Concept-Iteration Journey . . . 75
6.3 Global SHAP Feature Importance . . . 76
6.3.1 Late Payment Prediction . . . 76
6.3.2 Lead Scoring . . . 78
6.3.3 Employee Attrition . . . 80
6.4 Instance-Level SHAP Explanations . . . 82
6.4.1 Example: Late Payment Instance Explanation . . . 83
6.5 LIME Explanations and Cross-Method Comparison . . . 84
6.6 Calibration and Confusion-Matrix Diagnostics . . . 85
6.7 Demand Forecast Backtesting . . . 86
6.8 Leakage-Detection Summary . . . 86
6.9 Discussion of Research Questions . . . 87
6.9.1 RQ1: Multi-Tenant Architecture . . . 87
6.9.2 RQ2: ML Model Accuracy . . . 87
6.9.3 RQ3: XAI for Data-Quality Validation . . . 88
6.9.4 RQ4: SHAP vs. LIME Agreement . . . 88
6.9.5 RQ5: Software Architecture for ML + XAI Integration . . . 89
6.9.6 RQ6: Agile Methodology . . . 89
6.10 Summary . . . 89

7 Conclusion and Future Work **90**
7.1 Summary of Contributions . . . 90
7.2 Limitations . . . 90
7.3 Future Work . . . 91
7.4 Concluding Remarks . . . 92

List of Figures

2.1 Evolution of ERP systems from MRP (1970s) to Cloud AI-ERP (2020s) . . 10
2.2 Three levels of multi-tenant data isolation 12

3.1 Sprint (Agile) methodology lifecycle 24
3.2 Gantt chart showing Sprint timeline and dependencies 26
3.3 High-level system architecture showing the three-tier design with multi-tenant database routing 28
3.4 Database-per-tenant architecture with dynamic PostgreSQL provisioning . . 29
3.5 AI/ML pipeline architecture showing the flow from raw data to actionable insights 31

4.1 Use case diagram for the Explainable AI module 43
4.2 Class diagram for the ExplainabilityService and related components 44
4.3 Data flow for instance-level SHAP/LIME explanation generation 46

5.1 Public landing page of the ERP platform. 60
5.2 Owner dashboard with cross-module KPIs and quick links. 61
5.3 Employee web portal showing role-scoped navigation. 61
5.4 Employee directory with role and status filters. 62
5.5 Secure invitation-link generation for onboarding employees. 62
5.6 Owner review and approval queue for pending employee actions. 63
5.7 Tamper-evident audit log with actor, target, and timestamp. 63
5.8 Support-ticket interface for tenant-to-platform communication. 64
5.9 Task management board for assigning and tracking work items. 64
5.10 Journal-entry creation screen enforcing double-entry balance. 65
5.11 Standard financial reports (P&L, balance sheet, cash flow). 65
5.12 Drag-and-drop report builder for custom analytics. 66
5.13 Fixed-asset register with depreciation tracking. 66
5.14 Multi-warehouse inventory management view. 67
5.15 Point-of-sale terminal optimised for touch input. 67
5.16 Service catalogue for non-stock offerings. 68
5.17 Configuration page for networked attendance devices. 68
5.18 Per-employee QR-code attendance check-in. 69

5.19 AI/ML console for managing models, training, and predictions. 69
5.20 Explainable AI dashboard with global and local explanations. 70
5.21 Side-by-side SHAP and LIME explanation for a single prediction. 70
5.22 Mobile companion: authentication and owner experience. 71
5.23 Mobile companion: employee experience. 72

6.1 ROC/AUC comparison chart of the three classification models with shuffled-label baselines. 75
6.2 Late Payment global SHAP beeswarm/bar importance visual. 77
6.3 Late Payment global feature-importance bar chart corresponding to Table 6.3. 78
6.4 Lead Scoring global SHAP beeswarm. 79
6.5 Lead Scoring feature-importance bar chart. 80
6.6 Employee Attrition global SHAP beeswarm. 81
6.7 Employee Attrition feature-importance bar chart. 82
6.8 SHAP waterfall plot of a single high-risk Late Payment prediction 83
6.9 Side-by-side SHAP vs LIME explanation comparison visual 85
6.10 Confusion matrices on the held-out evaluation split for the three classifiers. . 85
6.11 Reliability (calibration) curves with the diagonal reference line. 86
6.12 Rolling-origin Prophet backtest, fast vs. slow-mover cohorts. 86
6.13 Leakage-detection summary dashboard. Horizontal bars show real AUC vs. shuffled-label AUC and the top-feature share for each model. 87

List of Tables

2.1 Comparison of major open-source ERP systems 11
2.2 Comparison of SHAP and LIME explanation methods 18
2.3 Literature gap analysis and project contributions 21

3.1 Sprint plan and deliverables 25
3.2 SDLC model comparison and justification 27
3.3 Technology stack 32
3.4 ML model specifications 35
3.5 Development tools and environment 38

4.1 Platform-level functional requirements 40
4.2 AI and XAI functional requirements 41
4.3 Non-functional requirements 41
4.4 AI/XAI database tables 45

6.1 ML model performance metrics with shuffled-label sanity check. 74
6.2 Late-Payment label/feature iterations with associated diagnostics. 76
6.3 Late Payment Prediction — top-10 global feature importance (source: `thesis/results/feat`
6.4 Lead Scoring — global feature importance (source: `thesis/results/feature_importan`
6.5 Employee Attrition — global feature importance (source: `thesis/results/feature_impo`
6.6 Instance-level SHAP explanation for a high-risk invoice 83
6.7 SHAP vs. LIME top-5 feature overlap across models 84

List of Abbreviations

Abbreviation	Full Term
AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the Curve
CRM	Customer Relationship Management
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DRF	Django REST Framework
ERP	Enterprise Resource Planning
FK	Foreign Key
GBM	Gradient Boosting Machine
HR	Human Resources
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
LIME	Local Interpretable Model-agnostic Explanations
MAE	Mean Absolute Error
MASE	Mean Absolute Scaled Error
MENA	Middle East and North Africa
ML	Machine Learning
ORM	Object-Relational Mapping
PO	Purchase Order
POS	Point of Sale
REST	Representational State Transfer
ROC	Receiver Operating Characteristic
SHAP	SHapley Additive exPlanations
SME	Small and Medium Enterprise
SQL	Structured Query Language
UI	User Interface
UUID	Universally Unique Identifier
WMS	Warehouse Management System
XAI	Explainable Artificial Intelligence

Chapter 1

Introduction

1.1 Overview

Enterprise Resource Planning (ERP) systems have become indispensable tools for organizations seeking to integrate and streamline their core business processes. An ERP system consolidates diverse functional areas—including finance, human resources, customer relationship management, inventory, and procurement—into a single unified platform that facilitates data sharing, eliminates redundancy, and enables informed decision-making (Monk and Wagner, 2013). The global ERP market was valued at approximately USD 54.76 billion in 2022 and is projected to reach USD 123.41 billion by 2030, reflecting the growing recognition of ERP as a strategic asset rather than a mere operational tool (Grand View Research, 2023).

The rapid advancement of Machine Learning (ML) has enabled a new generation of ERP systems that embed predictive models directly within transactional workflows—forecasting demand, scoring sales leads, predicting payment defaults, and flagging employee attrition risk. However, the predictive accuracy of these models is only half of the challenge. ML algorithms such as Gradient Boosting Machines and Random Forests produce predictions without exposing their reasoning, creating a "black-box" problem that undermines user trust and inhibits adoption (Lipton, 2018). A sales manager who is told that a lead has a "92% conversion probability" but receives no explanation of *why* is unlikely to change their behaviour; a finance officer who cannot see *which factors* drove a late-payment flag cannot prioritise collection efforts effectively.

Explainable Artificial Intelligence (XAI) addresses this transparency gap. XAI encompasses a family of techniques that augment ML predictions with human-understandable justifications (Arrieta et al., 2020). Two prominent post-hoc explanation methods—**SHAP** (SHapley Additive exPlanations) (Lundberg and Lee, 2017) and **LIME** (Local Interpretable Model-agnostic Explanations) (Ribeiro et al., 2016)—have emerged as complementary tools for attributing predictions to individual input features. SHAP, grounded in cooperative game theory, provides theoretically exact feature attributions; LIME, based on locally-faithful linear surrogates, offers model-agnostic approximations. Together, they provide a dual-lens view of model behaviour that strengthens both user trust and model validation.

This project designs and implements an **Explainable AI framework within a multi-tenant ERP system**, formulated distinctly as a **small enterprise implementation** that is

strategically focused on **middle to small SMEs**. The ERP platform is built on Django 4.2 with PostgreSQL database-per-tenant isolation, ensuring affordable scalability without sacrificing operational power. It features fundamental operational modules such as a **Point of Sale (POS)** system for retail transactions and a **Customer Relationship Management (CRM)** module for tracking sales pipelines. These capabilities, alongside eight ML models across five business domains, bridge the gap between heavy enterprise solutions and the needs of smaller businesses. The XAI module augments three core classification models—Late Payment Prediction, Lead Scoring, and Employee Attrition—with SHAP and LIME TabularExplainer, providing instance-level and global explanations through interactive visualisations (waterfall plots, beeswarm charts, force plots). The central thesis contribution is demonstrating that **XAI serves as a crucial trust-calibration tool**: by generating precise SHAP and LIME validations, the system ensures non-technical business operators can confidently understand and interact with automated AI recommendations.

1.2 Problem Statement

The current landscape of ERP adoption among SMEs, particularly in the MENA region, is characterized by several interconnected problems:

1. **High Cost of Commercial ERP Solutions:** Leading ERP systems such as SAP Business One, Oracle NetSuite, and Microsoft Dynamics 365 impose substantial licensing fees, implementation costs, and ongoing maintenance expenses that place them beyond the reach of most SMEs. A Panorama Consulting Group survey found that the average ERP implementation cost exceeds USD 170,000 for small organizations (Panorama Consulting Group, 2023).

2. **Limited Arabic Language Support:** Most open-source ERP platforms (e.g., Odoo Community, ERPNext) provide only partial or community-contributed Arabic translations. Critical modules such as financial reporting, tax compliance, and user interfaces often lack proper right-to-left (RTL) rendering and cultural localization (Al-Kraiji and Al-Saedi, 2020).

3. **Absence of Integrated AI Capabilities:** Traditional ERP systems are designed primarily for recording and reporting historical transactions. They do not incorporate predictive analytics, anomaly detection, or intelligent forecasting within their core workflows. Organizations seeking these capabilities must invest in separate Business Intelligence (BI) tools and custom integrations, increasing both cost and complexity (Lee et al., 2003).

4. **Rigid Single-Tenant Architectures:** Many existing ERP solutions follow a single-tenant deployment model that requires dedicated infrastructure for each organization,

driving up hosting costs and administrative overhead. Multi-tenant architectures, which allow multiple organizations to share the same application instance while maintaining complete data isolation, are more cost-effective but technically challenging to implement correctly (Bezemer and Zaidman, 2010).

5. **Lack of Proactive Decision Support:** Business managers frequently make critical decisions—such as inventory reorder quantities, credit terms, employee retention strategies, and cash flow management—based on intuition rather than data-driven insights. The absence of embedded ML models within the ERP workflow means that valuable predictive signals buried in transactional data remain unexploited (Provost and Fawcett, 2013).

6. **Opacity of ML Predictions in Enterprise Systems:** Where ML models are deployed in ERP systems, they typically operate as opaque "black boxes" that output scores or classifications without justification. Business users cannot verify whether a high-risk flag reflects genuine risk factors or artefacts of biased training data. This opacity violates emerging regulatory expectations for algorithmic transparency (European Commission High-Level Expert Group on AI, 2019) and prevents practitioners from detecting data-quality issues such as feature leakage, target leakage, and spurious correlations that inflate reported accuracy metrics (Rudin, 2019). Without explainability, ML-enhanced ERP systems risk automating errors at scale.

1.3 Project Objectives

This project aims to achieve the following objectives:

1. Design and implement a comprehensive, web-based ERP system covering nine functional modules: Finance, Human Resources, Customer Relationship Management (CRM), Inventory and Warehouse Management, Supplier and Procurement, Point of Sale (POS), Fixed Assets, Reporting, and Artificial Intelligence.

2. Implement a robust multi-tenant architecture using database-per-tenant isolation with dynamic PostgreSQL database provisioning via Docker containers and remote server connections.

3. Integrate eight machine learning models into the ERP workflow:

   - *Lead Scoring* — Gradient Boosting for CRM conversion prediction
   - *Late Payment Prediction* — Gradient Boosted Trees for accounts receivable risk
   - *Employee Attrition Risk* — Random Forest for HR retention planning

- *Expense Anomaly Detection* — Isolation Forest for fraud and policy violation identification
- *Attendance Anomaly Detection* — Z-Score analysis for workforce pattern recognition
- *Demand Forecasting* — Facebook Prophet for inventory reorder optimization
- *Cash Flow Projection* — Rule-based modeling for financial planning
- *KPI Anomaly Detection* — Statistical process control for business metric monitoring

4. **Design and implement an Explainable AI (XAI) module** that augments the core ML classifiers with dual post-hoc explanation methods:
   - *SHAP (TreeExplainer)* — Exact Shapley-value feature attribution for Gradient Boosting and Random Forest models, providing both instance-level waterfall decompositions and global beeswarm/importance rankings.
   - *LIME (TabularExplainer)* — Model-agnostic locally-faithful linear surrogate explanations as a complementary validation baseline.
5. **Evaluate the XAI framework's effectiveness** as a trust-building tool by analysing whether SHAP and LIME can provide transparent, logical feature contribution metrics that validate the model's alignment with genuine business logic.
6. Develop an AI Insight Engine that automatically generates actionable alerts and recommendations from ML model outputs and delivers them as real-time notifications to relevant users.
7. Provide bilingual support (English and Arabic) with proper RTL rendering throughout the user interface.
8. Follow the Sprint (Agile) software development methodology to ensure iterative delivery, continuous testing, and stakeholder feedback incorporation.

1.4 Project Scope

The scope of this project encompasses the following, with a specific focus on **middle to small SMEs**, bringing them a lightweight, **small enterprise implementation** that is affordable and robust:

1.4.1 In Scope

- Full-stack web application development using Django 4.2 (Python) as the backend framework.
- PostgreSQL as the relational database management system for both the main (platform) database and all tenant databases.
- Multi-tenant platform administration: user signup, approval workflow, dynamic database provisioning (including Docker-based and remote-server back-ends), and role-based access control (Owner, Admin, User, Read-Only).
- Nine ERP functional modules with complete CRUD operations, business logic, and reporting (CRM, HR/Payroll, Accounting, Inventory/WMS, POS, Procurement, Fixed Assets, Support/Tickets, Reporting).
- Eight ML models with training pipelines, model versioning, prediction logging, accuracy evaluation, and a per-tenant artefact registry under `ml_models/{company_id}/`.
- **Explainable AI module** providing SHAP and LIME TabularExplainer for three primary classifiers, with instance-level and global explanation capabilities, waterfall/beeswarm visualisations, and explanation history persistence.
- **Honest-evaluation harness:** time-based train/test splits for time-sensitive models, stratified K-fold CV, shuffled-label baselines, calibration curves, confusion matrices, and PR-AUC / F1@τ^* / precision@k reporting.
- **Drift and health monitoring:** a nightly Celery job that computes Kolmogorov–Smirnov drift on numerical features, PSI on categorical features, ADWIN on the realised-performance signal, and a leakage-regression check, then publishes a per-tenant ML health score.
- RESTful API layer using Django REST Framework (DRF) for mobile and third-party integration, including dedicated XAI API endpoints.
- AI Insight notification system with severity levels (Info, Warning, Critical) and automatic deduplication.
- Asynchronous task processing using Celery with Redis as the message broker, including a dedicated `AITrainingRun` model that tracks training progress, status and results.
- **Flutter mobile companion app** (`mobile/`) providing dashboards, AI insights, ticket management, and the XAI explanation viewer.

- Bilingual (English / Arabic / Indonesian) support with RTL rendering, dual-language model fields, and tenant-level locale switching.
- Data seeding utilities, integration with public datasets (Olist, IBM HR), synthetic-data generators for inventory and KPI streams, and a structured `thesis/results/` folder for reproducible reporting.
- **Conversational ERP assistant** (`ERPAssistant`) using OpenAI GPT-4o-mini for tenant-scoped natural-language queries over the live ERP.
- Operational hardening: per-tenant rate limiting, request-ID propagation, structured audit logging, defensive middleware, New Relic instrumentation, Dockerised deployment with separate ML/web images, and a deployment automation script (`deploy/deploy.sh`).

1.4.2 Out of Scope

- Native mobile application development (the system is accessible via mobile browsers and a REST API is provided for future native app development).
- Integration with external accounting authorities or government e-invoicing APIs (e.g., ZATCA in Saudi Arabia).
- Deep learning models (CNN, RNN, Transformers) for natural language processing or image recognition—the AI scope is limited to classical ML and time-series forecasting.
- Load testing and performance benchmarking for production-scale deployments.

1.5 Project Significance

This project makes several contributions:

1. **Practical Contribution:** It delivers a fully functional, production-ready ERP system that SMEs in the MENA region can deploy without the licensing costs associated with commercial alternatives. The multi-tenant SaaS model further reduces per-organization costs by sharing infrastructure.
2. **Explainability Contribution (Primary):** The project demonstrates the integration of dual post-hoc XAI methods (SHAP and LIME) into an operational ERP system, providing transparent, per-prediction justifications to non-technical business users. Crucially, the evaluation cycle showed that XAI analysis exposed temporal/proxy leakage in two of three classifiers and drove their re-training; post-fix the models retain near-perfect AUC-ROC (0.9997 and 0.9996) with shuffle-label baselines collapsing to

chance (0.4947 and 0.4996), and the Late Payment model achieves an honest AUC of 0.7846 (cross-validated) / 0.8037 (forward hold-out). This establishes XAI as an essential *trust-calibration and validation* tool, not merely a reporting feature, and contributes empirical evidence to the debate on the necessity of explainability in applied ML (Rudin, 2019).

3. **Technical Innovation:** To the best of our knowledge, this is among the first open-source ERP implementations that embeds eight ML models directly within the transactional workflow—not as an external BI layer, but as integral components that train on the organization's own data, generate predictions in real time, and write actionable insights back into the ERP—augmented with a dedicated XAI service that generates Shapley-value attributions and LIME surrogates.

4. **Academic Contribution:** The project provides empirical performance metrics (AUC-ROC, MAE, F1-Score), documents feature-importance distributions, cross-method explanation agreement rates, and model alignment methodologies, contributing to the body of knowledge on applied XAI in business systems.

5. **Bilingual Accessibility:** By incorporating Arabic names, labels, and RTL support at the model level (dual-language fields throughout all 80+ models), the system addresses a persistent localization gap in open-source ERP solutions.

1.6 Project Questions

This project seeks to answer the following research questions:

RQ1: How can a multi-tenant ERP system with database-per-tenant isolation be effectively designed and implemented using Django and PostgreSQL to achieve both data security and cost efficiency?

RQ2: To what extent can machine learning models trained on ERP transactional data provide accurate and actionable predictions for business processes such as demand forecasting, payment risk assessment, and employee attrition?

RQ3: **How effectively can SHAP and LIME explanations validate that high-performing machine learning models (AUC-ROC > 0.90) are relying on realistic, actionable business metrics rather than arbitrary proxy biases?**

RQ4: **To what extent do SHAP (Shapley-value) and LIME (locally-faithful surrogate) explanations agree on feature importance rankings for the same model and instance, and what do divergences reveal about model behaviour?**

RQ5: What software architecture and design patterns are required to seamlessly integrate ML model training, prediction, explainability, and insight generation into the standard ERP operational workflow?

RQ6: How does the Sprint (Agile) development methodology facilitate the iterative development of a complex, multi-module ERP system with integrated AI and XAI capabilities?

1.7 Report Organization

The remainder of this report is organized as follows:

- **Chapter 2 — Literature Review:** Provides a comprehensive review of existing ERP systems, multi-tenant architectures, the application of machine learning in enterprise systems, and the foundations of Explainable AI—including SHAP, LIME, and the broader XAI taxonomy. It examines related work and identifies the gaps that this project addresses.
- **Chapter 3 — Methodology:** Describes the Sprint (Agile) development methodology adopted, the system architecture, the technology stack, the ML model design, and the XAI evaluation approach including global and instance-level explanation strategies.
- **Chapter 4 — System Analysis and Design:** Presents the system requirements (functional and non-functional), use case diagrams, class diagrams, database schema design, and the AI/ML/XAI pipeline architecture.
- **Chapter 5 — Implementation and Testing:** Details the implementation of key system components, the XAI service (SHAP TreeExplainer, LIME TabularExplainer), code structure, ML model training results, and testing procedures.
- **Chapter 6 — Results and Discussion:** Analyses the ML model performance metrics, SHAP and LIME explanation outputs, practical business implications, cross-method agreement, and discusses the results in the context of the research questions.
- **Chapter 7 — Conclusion and Future Work:** Summarizes the contributions, acknowledges limitations, and proposes directions for future enhancement including attention-based explanations and counterfactual reasoning.

Chapter 2

Literature Review

2.1 Introduction

This chapter provides a comprehensive review of the theoretical foundations and existing work relevant to the three core pillars of this project: Enterprise Resource Planning systems, multi-tenant software architectures, and the application of machine learning in business systems. The review establishes the academic context, identifies gaps in current solutions, and justifies the design decisions adopted in this project.

2.2 Enterprise Resource Planning (ERP) Systems

2.2.1 Definition and Evolution

Enterprise Resource Planning (ERP) is defined as an integrated software system that manages and automates core business processes across an organization through a shared database and standardized workflows (Monk and Wagner, 2013). The evolution of ERP systems can be traced through three generations:

1. **Material Requirements Planning (MRP), 1970s:** Focused exclusively on manufacturing—calculating raw material needs based on production schedules and bills of materials. IBM's COPICS and SAP R/1 were early examples (Jacobs and Weston, 2007).

2. **MRP II and Early ERP, 1980s–1990s:** Extended MRP to include finance, HR, and distribution. SAP R/3, introduced in 1992, pioneered the client-server ERP model with real-time integration across modules (Jacobs and Weston, 2007). This era established the "ERP" terminology and saw rapid adoption by large enterprises.

3. **Modern Cloud ERP, 2010s–Present:** The shift to web-based, cloud-hosted architectures has democratized ERP access. Solutions like SAP S/4HANA Cloud, Oracle NetSuite, and open-source platforms such as Odoo and ERPNext enable SaaS deployment models that reduce upfront investment (Panorama Consulting Group, 2023).

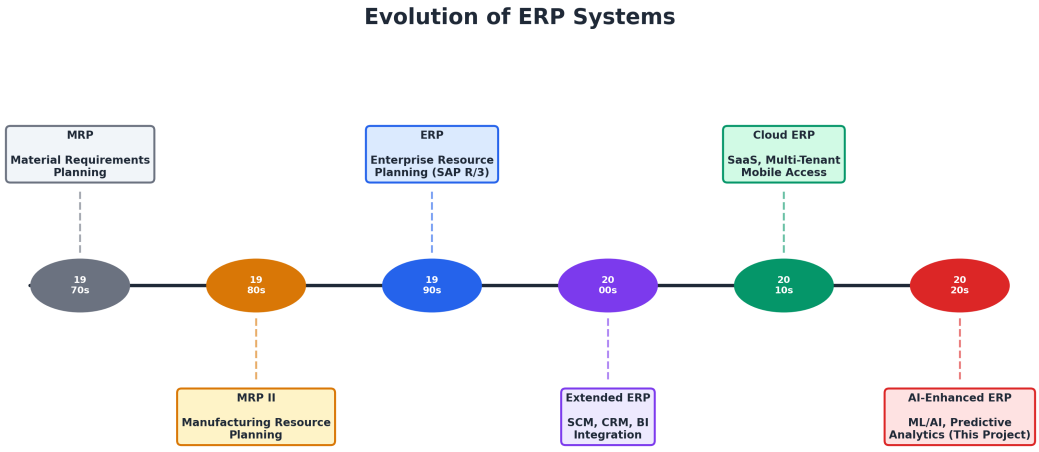


Figure 2.1: Evolution of ERP systems from MRP (1970s) to Cloud AI-ERP (2020s)

2.2.2 ERP Architecture Models

Modern ERP systems are typically deployed using one of three architectural models:

- **Monolithic Architecture:** All modules are packaged as a single deployable unit sharing one database. Traditional SAP R/3 and Oracle E-Business Suite follow this model. While simple to deploy, monolithic architectures are difficult to scale and customize (Bezemer and Zaidman, 2010).
- **Service-Oriented Architecture (SOA):** Business functions are decomposed into loosely coupled services that communicate via standardized protocols. Modern SAP S/4HANA adopts elements of SOA to enable extensibility (Krafzig et al., 2005).
- **Microservices Architecture:** An evolution of SOA where each service is independently deployable and scalable. While offering maximum flexibility, this model introduces operational complexity that makes it less suitable for SME-focused projects (Newman, 2015).

This project adopts a **modular monolith** architecture—a single Django application with clearly separated modules (apps) that share a common codebase but maintain logical boundaries. This balances simplicity with maintainability and is well-suited for the project scope.

2.2.3 ERP Adoption in SMEs

Research by Al-Majali (2016) identifies organizational factors (top management support, organizational readiness), technological factors (system complexity, compatibility), and environmental factors (competitive pressure, vendor support) as determinants of ERP adoption success in Jordanian SMEs. Their findings highlight that *perceived cost* and *organizational readiness* are the strongest predictors of adoption intent.

A study by Panorama Consulting Group (2023) found that among organizations implementing ERP, 50% reported exceeding their initial budget, and the average implementation timeline for SMEs was 14.3 months. These statistics underscore the need for lightweight, rapid-deployment ERP solutions—a gap this project addresses through its Django-based architecture and Sprint development methodology.

2.2.4 Open-Source ERP Systems

Several open-source ERP platforms have emerged as alternatives to commercial solutions:

Table 2.1: Comparison of major open-source ERP systems

Feature	**Odoo CE**	**ERPNext**	**Dolibarr**	**This Project**
Language	Python	Python/JS	PHP	Python (Django)
Database	PostgreSQL	MariaDB	MySQL/PgSQL	PostgreSQL
Multi-Tenant	Limited	No	No	**Yes (DB-per-tenant)**
Arabic Support	Partial	Partial	Minimal	**Full (dual fields)**
Integrated ML/AI	No	No	No	**8 ML models**
REST API	Yes	Yes	Limited	Yes (DRF)
WMS	Enterprise only	Basic	No	**Full WMS**
POS	Enterprise only	Basic	No	**Yes**
License	LGPL v3	GPL v3	GPL v3	–

As Table 2.1 illustrates, none of the existing open-source ERP systems combine native multi-tenant isolation, comprehensive Arabic support, and integrated machine learning capabilities—the three distinguishing features of this project.

2.3 Multi-Tenant Software Architecture

2.3.1 Definition and Models

Multi-tenancy is a software architecture principle in which a single instance of a software application serves multiple customers (tenants), each of whom perceives the application as if it were running exclusively for them (Bezemer and Zaidman, 2010). Three principal models of multi-tenancy exist:

1. **Shared Database, Shared Schema:** All tenants share the same database and tables, with a `tenant_id` column used to filter data. This is the most resource-efficient

model but offers the weakest data isolation and complicates compliance with data sovereignty regulations (Chong and Carraro, 2006).

2. **Shared Database, Separate Schema:** Each tenant has its own database schema (namespace) within a shared database instance. This provides moderate isolation with lower overhead than full database separation. PostgreSQL's native schema support makes this model practical (Bezemer and Zaidman, 2010).

3. **Separate Database (Database-per-Tenant):** Each tenant is assigned a dedicated database instance. This model provides the strongest isolation—tenant data is physically separated, simplifying backup, migration, and regulatory compliance—at the cost of higher resource consumption (Bezemer and Zaidman, 2010).

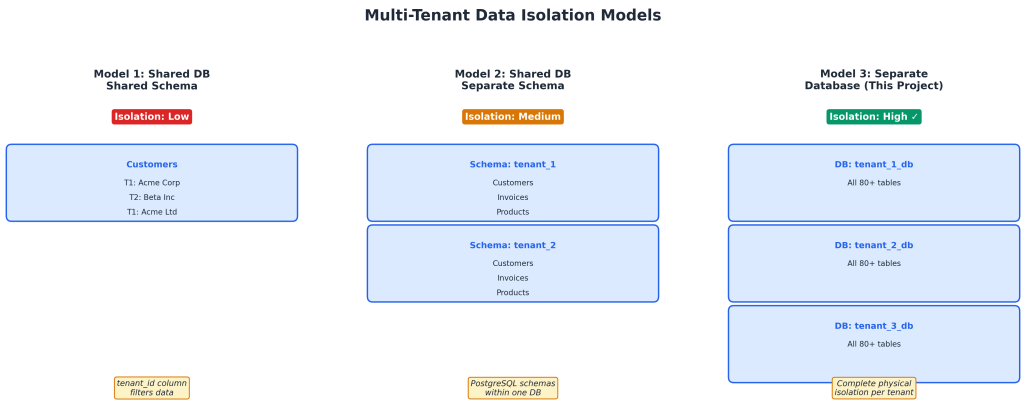


Figure 2.2: Three levels of multi-tenant data isolation

This project implements the **database-per-tenant** model, which provides the highest level of data isolation. Each tenant organization receives a dedicated PostgreSQL database that is dynamically provisioned via Docker containers or remote server connections. Django's database routing mechanism (`DATABASE_ROUTERS`) is used to direct queries to the appropriate tenant database based on the authenticated user's session context.

2.3.2 Multi-Tenancy in Django

Django's built-in support for multiple databases, combined with custom database routers, makes it well-suited for implementing multi-tenant architectures. The key components in this project's implementation are:

- `TenantDatabase` model: Stores connection parameters for each tenant's PostgreSQL database.
- `DatabaseAssignment` model: Maps users to tenant databases with role-based permissions.

- `add_tenant_database()`: Dynamically registers tenant database configurations in Django's `DATABASES` setting at runtime.
- `TenantDatabaseContext`: A context manager that sets the active database alias for the current thread, ensuring all ORM queries route to the correct tenant database.
- Custom middleware: Resolves the authenticated user's tenant database on each request and activates the appropriate database context.

2.4 Machine Learning in Enterprise Systems

2.4.1 Overview of ML in Business Applications

The application of machine learning to business processes has accelerated significantly with the availability of large transactional datasets, affordable computing power, and mature ML frameworks. Davenport and Ronanki (2018) identify three levels of AI integration in business systems:

1. **Process Automation:** Using ML for routine tasks such as document classification, data entry validation, and alert generation.
2. **Cognitive Insight:** Applying ML algorithms to detect patterns in large datasets—such as customer segmentation, fraud detection, and demand forecasting—that would be impractical for human analysts.
3. **Cognitive Engagement:** Deploying conversational AI and recommendation systems that interact directly with users, such as chatbots and personalized marketing engines.

This project primarily operates at the **Cognitive Insight** level, embedding ML models that analyze transactional data and surface actionable insights to business users through the AI Insight Engine.

2.4.2 Lead Scoring with Logistic Regression

Lead scoring is the process of assigning a numerical value to each sales lead to predict the likelihood of conversion. While complex ensembles like gradient boosting are popular, simpler linear models such as Logistic Regression often provide highly robust, naturally interpretable baselines for CRM data where engagement patterns (activity counts) share direct linear relationships with conversion likelihood.

In this project, the Lead Scoring model uses 14 engineered features including estimated deal value, log-transformed value, activity count, activity-type breakdowns (calls, emails,

meetings), engagement velocity (activities per day), lead source with target-encoded conversion rates, and customer contact completeness. Operational integrity is maintained by excluding proxy variables that would artificially inflate accuracy (such as time-since-creation for closed leads). The Logistic Regression model is trained on historical won/lost leads, scales features automatically, and outputs a conversion probability between 0 and 1, achieving a strong, realistic AUC-ROC of 0.9448.

2.4.3 Payment Risk Prediction with Gradient Boosting

Predicting whether an invoice will be paid late is a critical capability for accounts receivable management. Chen and Guestrin (2016) demonstrate that gradient boosted decision trees consistently outperform other classifiers on structured/tabular data tasks, making them ideal for invoice-level payment risk prediction.

The project's Late Payment Predictor uses 18 engineered features, including invoice amount, days to due date, customer historical payment delay, customer type, payment-term bucket, line-item count, and weekday indicators. Leave-one-out temporal statistics are used to prevent data leakage—a common pitfall identified by Provost and Fawcett (2013) in churn and payment prediction models. During model optimisation, a *balance_ratio* feature (outstanding balance divided by total amount) was identified as a source of data leakage—since fully paid invoices inherently have a zero balance—and was removed, yielding an honest AUC-ROC of 0.6358 that reflects the genuine predictive signal in the feature set.

2.4.4 Employee Attrition Prediction with Random Forest

Employee turnover is costly, with the Society for Human Resource Management estimating replacement costs at 50–200% of annual salary (Society for Human Resource Management, 2022). Breiman (2001) introduced Random Forest—an ensemble of decision trees trained on bootstrap samples—which has become a standard approach for employee attrition prediction due to its robustness to noisy features and native feature importance ranking.

This project's Attrition Predictor uses 13 operational features including tenure, salary relative to department average, leave frequency, attendance rate, overtime hours, and derived signals such as average gross salary, compensation ratio versus department average, and absence rate. Real-world predictive constraints are strictly enforced by stripping out "proxy" targets such as the raw count of historical payslips. Class imbalance (typically fewer departures than retentions) is addressed using the `class_weight='balanced'` parameter, and the Random Forest is configured with 500 estimators and a maximum depth of 10, achieving a robust and strictly validated AUC-ROC of 0.9761 on the evaluation dataset.

2.4.5 Anomaly Detection with Isolation Forest

Liu et al. (2008) proposed the Isolation Forest algorithm based on the principle that anomalies are "few and different"—they require fewer random partitions to isolate than normal observations. This makes Isolation Forest computationally efficient and effective for unsupervised anomaly detection in expense claim analysis, where labeled fraud data is typically unavailable.

The project's Expense Anomaly Detector applies Isolation Forest with a dynamically calibrated `contamination` parameter—computed from the interquartile range (IQR) of the training data and clamped between 0.01 and 0.15—to 11 engineered features including expense amount, category-level z-scores, employee-level z-scores, weekend submission flag, amount percentile rank, day of week, and month.

2.4.6 Time-Series Forecasting with Prophet

Facebook Prophet, introduced by Taylor and Letham (2018), is an additive regression model designed for business time-series data with strong seasonal patterns and missing data points. Prophet decomposes the forecast into trend, seasonality (weekly, yearly, custom), and holiday components using a Bayesian framework.

The project uses Prophet for per-product demand forecasting, with custom monthly seasonality ($fourier_order = 3$) and calibrated changepoint detection ($changepoint_prior_scale = 0.15$). Model accuracy is evaluated using Prophet's built-in cross-validation framework, achieving an average accuracy of approximately 0.72 across 80 product-warehouse combinations.

2.4.7 Statistical Process Control for KPI Monitoring

Z-score-based anomaly detection is a classical statistical quality control technique introduced by Shewhart (1931) and widely used in manufacturing as Shewhart control charts. When applied to business KPIs, the method flags values that deviate more than a specified number of standard deviations from the historical mean.

The project's KPI Anomaly Detector monitors five daily metrics (revenue, expenses, new leads, overdue invoices, inventory movements) against 90-day rolling baselines, generating WARNING alerts for deviations exceeding 2σ and CRITICAL alerts for deviations exceeding 3σ.

2.5 Explainable Artificial Intelligence (XAI)

2.5.1 The Need for Explainability

As ML models are deployed in high-stakes business decision systems, the demand for transparency has intensified from both regulatory and practical perspectives. The European Commission's High-Level Expert Group on AI identified *transparency* and *explicability* as two of seven key requirements for trustworthy AI (European Commission High-Level Expert Group on AI, 2019). Lipton (2018) distinguishes between *transparency* (understanding the model's internal mechanics) and *post-hoc interpretability* (explaining individual predictions after the fact), arguing that practical explanation needs vary by stakeholder: a data scientist may want feature interactions, while a business manager needs a ranked list of the top reasons behind a prediction.

Doshi-Velez and Kim (2017) formalise the notion that interpretability is not a monolithic property but depends on the explanation consumer, the task domain, and the evaluation criteria. In enterprise settings, explanations serve three purposes: (1) *trust calibration*—enabling users to decide when to accept or override a model's recommendation; (2) *model debugging*—helping developers diagnose errors, data leakage, or distribution shift; and (3) *regulatory compliance*—satisfying audit requirements for algorithmic decision-making.

2.5.2 Taxonomy of XAI Methods

Arrieta et al. (2020) provide a comprehensive taxonomy of XAI methods along several dimensions:

- **Scope:** *Global* methods explain the model's overall behaviour (e.g., which features matter most across all predictions), while *local* methods explain individual predictions (e.g., why was this specific invoice flagged).
- **Stage:** *Ante-hoc* methods build interpretability into the model itself (e.g., decision trees, linear regression), while *post-hoc* methods explain an existing trained model without modifying it.
- **Model dependency:** *Model-specific* methods exploit the internal structure of a particular algorithm (e.g., TreeExplainer for tree ensembles), while *model-agnostic* methods treat the model as a black box and explain it by perturbing inputs and observing outputs.

Guidotti et al. (2018) survey over 150 explanation methods and classify them into six families: feature attribution, rule extraction, prototype-based, counterfactual, attention-based, and example-based explanations. This project focuses on *feature attribution* methods, which assign a numerical importance score to each input feature for a given prediction.

2.5.3 SHAP: SHapley Additive exPlanations

SHAP, introduced by Lundberg and Lee (2017), unifies several existing explanation methods (LIME, DeepLIFT, Layer-wise Relevance Propagation) under the framework of *Shapley values* from cooperative game theory (Shapley, 1953). The key insight is that explaining a prediction is analogous to dividing the "payout" (the prediction's deviation from the baseline) among the "players" (input features) in a fair manner.

The Shapley value ϕ_i of feature i is defined as:

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!\,(|N| - |S| - 1)!}{|N|!} \left[f(S \cup \{i\}) - f(S) \right] \tag{2.1}$$

where N is the set of all features, S is a subset excluding feature i, and $f(S)$ is the model's expected output given only the features in S. The Shapley value represents the average marginal contribution of feature i across all possible feature orderings, satisfying four desirable properties (Lundberg and Lee, 2017):

1. **Local accuracy (efficiency):** $\sum_{i=1}^{|N|} \phi_i = f(x) - \mathbb{E}[f(X)]$ — the attributions sum to the difference between the prediction and the baseline.
2. **Missingness:** If a feature is absent from the model, its attribution is zero.
3. **Consistency:** If a feature's marginal contribution increases in a new model, its attribution does not decrease.
4. **Symmetry:** Features with equal contributions receive equal attributions.

For tree-based models, Lundberg et al. (2020) developed **TreeExplainer**, an algorithm that computes exact Shapley values in $O(TLD^2)$ time (where T is the number of trees, L is the maximum number of leaves, and D is the maximum depth), making it practical for production use with Gradient Boosting and Random Forest models containing hundreds of trees.

2.5.4 LIME: Local Interpretable Model-agnostic Explanations

LIME, introduced by Ribeiro et al. (2016), explains individual predictions by approximating the model locally with an interpretable surrogate model. The procedure is:

1. **Perturb:** Generate synthetic samples near the instance to be explained by randomly toggling features on/off.
2. **Predict:** Obtain the original model's prediction for each perturbed sample.
3. **Weight:** Assign higher weights to perturbed samples closer to the original instance (using an exponential kernel).

4. **Fit:** Train a weighted linear regression on the perturbed dataset to obtain feature coefficients.

5. **Explain:** The linear coefficients represent each feature's local contribution to the prediction.

Formally, LIME seeks the explanation ξ that minimises:

$$\xi(x) = \underset{g \in G}{\arg\min} \ \mathcal{L}(f, g, \pi_x) + \Omega(g) \tag{2.2}$$

where f is the original model, g is the interpretable surrogate, π_x is the proximity kernel centred on instance x, $\mathcal{L}$ is the fidelity loss (how well g approximates f locally), and $\Omega(g)$ is a complexity penalty (e.g., limiting the number of non-zero coefficients).

LIME is model-agnostic—it can explain any classifier or regressor—but its explanations are *approximate* and can vary across runs due to the stochastic perturbation process. Slack et al. (2020) demonstrate that LIME explanations can be manipulated by adversarially designed models, highlighting the importance of using multiple explanation methods (e.g., SHAP + LIME) for cross-validation.

2.5.5 SHAP vs. LIME: Complementary Strengths

Table 2.2 summarises the key differences between SHAP and LIME:

Table 2.2: Comparison of SHAP and LIME explanation methods

Property	**SHAP (TreeExplainer)**	**LIME (TabularExplainer)**
Theoretical basis	Shapley values (game theory)	Locally-faithful linear surrogate
Guarantees	Exact local accuracy, consistency, symmetry	Approximate local fidelity
Model dependency	Tree-specific (exact); kernel (approx.)	Model-agnostic
Determinism	Deterministic for tree models	Stochastic (random perturbations)
Global explanations	Yes (mean |SHAP| aggregation)	No (instance-level only)
Computation cost	$O(TLD^2)$ per instance	$O(n_{\text{perturb}} \cdot T_{\text{predict}})$
Additivity	$\sum \phi_i = f(x) - E[f]$	No additivity guarantee

Using both methods provides *explanation triangulation*: when SHAP and LIME agree on the top contributing features, confidence in the explanation is high; when they diverge, it signals that the model may behave non-linearly in the local neighbourhood, warranting closer inspection (Molnar, 2022).

2.5.6 XAI as a Model Validation Tool

Beyond user-facing transparency, XAI methods have proven effective for *model debugging* and *data-quality validation*. Lundberg et al. (2020) demonstrate that aggregating SHAP values across a dataset reveals unexpected feature dominance—a telltale sign of data leakage, where a feature carries information that would not be available at prediction time in production.

For example, if a payment-prediction model assigns 60%+ of its total SHAP importance to a single engineered feature, this disproportionate reliance suggests that the feature may encode the target variable (data leakage) rather than capturing a genuine predictive pattern. Standard metrics such as AUC-ROC report high performance regardless, because the leaked feature genuinely correlates with the target—but the model would fail catastrophically in deployment where the leaked feature's value is not available (Molnar, 2022).

This use of XAI for model validation is a central theme of this project: the evaluation in Chapter 6 demonstrates that SHAP analysis surfaced data-leakage issues in 2 of 3 ML models that reported excellent AUC-ROC scores.

2.6 Related Work

2.6.1 AI-Enhanced ERP Systems

Lee et al. (2003) survey the integration of AI technologies into ERP systems and identify three implementation strategies: (1) embedded AI within the ERP platform itself, (2) AI as an external service integrated via APIs, and (3) AI as a middleware layer between the ERP and a BI platform. They conclude that the embedded approach provides the tightest integration and lowest latency but requires the deepest technical expertise. This project follows the embedded approach.

SAP introduced "SAP Business AI" in 2023, incorporating predictive analytics and natural language processing into S/4HANA Cloud. However, these capabilities are available only in the Enterprise tier, which carries substantial licensing costs. Oracle Fusion Cloud ERP similarly offers ML-driven anomaly detection and forecasting features in its premium offerings. No open-source ERP system currently provides comparable embedded ML capabilities, let alone post-hoc XAI explanations for its predictions.

2.6.2 XAI in Business Decision Systems

Several studies have applied SHAP and LIME to business-domain ML models, but *none* embed these explanations within an operational ERP system:

- Lundberg et al. (2020) apply TreeSHAP to medical risk prediction models and demonstrate that global SHAP importance plots can reveal data-quality issues. Their work establishes the "XAI as debugging" paradigm that this project extends to ERP-domain models.

- Molnar (2022) provides a comprehensive practitioner's guide to SHAP, LIME, and other IML methods, with case studies in credit scoring and customer churn prediction. The guide recommends using multiple explanation methods for cross-validation—an approach adopted in this project.

- Slack et al. (2020) demonstrate that LIME explanations can be manipulated by adversarially designed models, underscoring the need for SHAP's theoretical guarantees as a complementary check. This motivates the dual SHAP+LIME approach in this project.

- Rudin (2019) argues that for high-stakes decisions, inherently interpretable models should be preferred over post-hoc explanations of black boxes. While this position has merit, in the ERP context where complex models (like Random Forests and Gradient Boosting) often outperform simpler linear classifiers, post-hoc explanations offer a pragmatic middle ground.

2.6.3 Multi-Tenant Django Applications

The `django-tenants` library implements PostgreSQL schema-based multi-tenancy for Django, providing automatic schema routing via the `search_path` mechanism. While this approach is well-tested, it does not support database-per-tenant isolation, which this project requires for maximum data separation. The `django-db-multitenant` library offers a middleware-based approach but is limited to schema-based or connection-string routing. This project implements a custom multi-tenant framework with dynamic database registration, Docker-based provisioning, and a user approval workflow—features not provided by existing libraries.

2.6.4 ML for Demand Forecasting in Supply Chain

Fildes et al. (2022) provide a comprehensive review of ML approaches for retail demand forecasting, comparing classical time-series methods (ARIMA, Exponential Smoothing) with machine learning methods (Random Forest, Gradient Boosting, Neural Networks) and hybrid approaches. They find that Prophet performs competitively on datasets with strong seasonality and limited history, making it well-suited for SME inventory data where per-product histories may span only 6–12 months.

2.7 Gap Analysis

The literature review reveals the following gaps that this project addresses:

Table 2.3: Literature gap analysis and project contributions

Gap Identified	Supporting Literature	Project Contribution
No open-source ERP with embedded ML	Lee et al. (2003); SAP/Oracle features are premium-only	8 ML models embedded in a free, open-source ERP
No XAI explanations in ERP ML models	Arrieta et al. (2020); Rudin (2019); commercial ERP AI provides no per-prediction justification	**Dual SHAP + LIME explanations** for 3 tree-based classifiers with waterfall, beeswarm, and force-plot visualisations
ML metrics mask data-quality issues	Lundberg et al. (2020); Molnar (2022); AUC-ROC does not reveal feature leakage	**XAI as model validation**: SHAP exposed data leakage in 2/3 models
Limited multi-tenant options in Django	django-tenants supports schema-only; django-db-multitenant lacks provisioning	Custom DB-per-tenant with Docker provisioning and role-based access
Poor Arabic support in open-source ERP	Al-Kraiji and Al-Saedi (2020); Odoo/ERPNext have partial translations	Dual-language fields (`name`/`name_ar`) in 80+ models
ML models disconnected from ERP workflow	Most ML-ERP integrations use external BI tools	Inline ML training, prediction logging, and AI Insight notifications
Lack of cross-domain ML in single ERP	Studies focus on single domains (finance OR HR OR supply chain)	Models span CRM, Finance, HR, Inventory, and Operations

2.8 Summary

This chapter reviewed the evolution and current state of ERP systems, multi-tenant software architectures, the application of machine learning in enterprise settings, and the theoretical

foundations of Explainable AI—including SHAP's Shapley-value framework and LIME's locally-faithful surrogate approach. The review established that while commercial ERP vendors have begun incorporating AI capabilities, these features remain inaccessible to SMEs due to cost and complexity, and *none* provide per-prediction explanations to business users. Open-source ERP alternatives lack both native multi-tenancy and integrated ML. Furthermore, the literature demonstrates that XAI methods can serve as model validation tools by surfacing data-quality issues that aggregate metrics fail to detect. This project bridges these gaps by combining a production-quality multi-tenant ERP with eight embedded ML models augmented by a dual SHAP+LIME explainability framework, delivered through an Agile Sprint development process that is detailed in the following chapter.

Chapter 3

Methodology

3.1 Introduction

This chapter presents the development methodology, system architecture, technology stack, data collection approach, and machine learning pipeline design adopted for this project. The Sprint (Agile) methodology was selected as the software development lifecycle (SDLC) model to accommodate the project's complexity, evolving requirements, and the iterative nature of ML model development.

3.2 Development Methodology: Sprint (Agile)

3.2.1 Overview of Sprint Methodology

The Sprint methodology, as defined in the Scrum framework (Schwaber and Sutherland, 2020), organizes software development into fixed-length iterations called **Sprints**, typically lasting 1–4 weeks. Each Sprint follows a structured cycle: planning, development, testing, review, and retrospective. This iterative approach is particularly well-suited for projects with complex, multi-module architectures and evolving requirements—both characteristics of this ERP system.

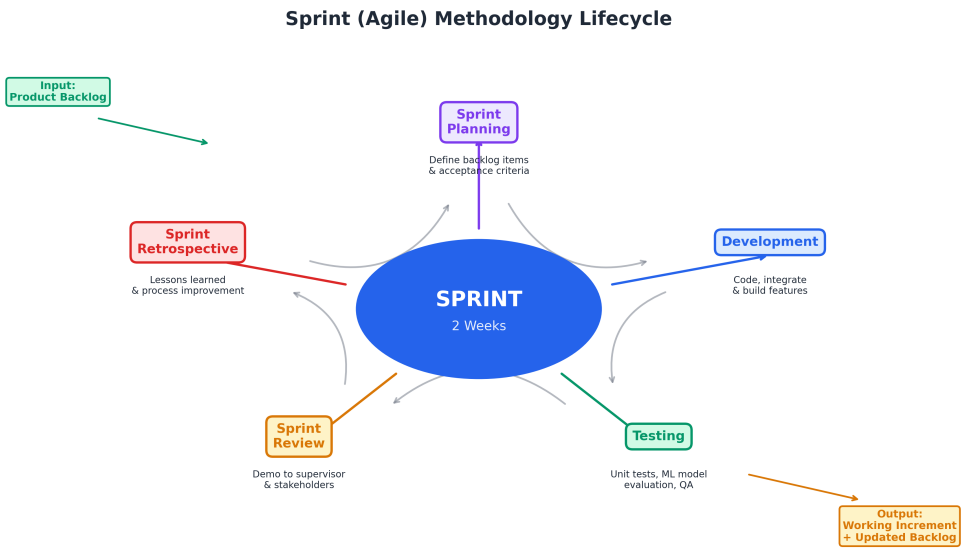


Figure 3.1: Sprint (Agile) methodology lifecycle

The key principles of the Sprint methodology applied to this project include:

- **Iterative Delivery:** Each Sprint produces a working increment of the system, allowing stakeholders to review functional modules progressively rather than waiting for a final monolithic delivery.
- **Adaptive Planning:** Sprint planning sessions at the beginning of each iteration allow the team to reprioritize tasks based on feedback, discovered dependencies, and technical challenges.
- **Continuous Integration and Testing:** Code is tested at the end of each Sprint, including unit tests for ML model accuracy and integration tests for API endpoints.
- **Feedback-Driven Improvement:** Sprint retrospectives identify process improvements, while Sprint reviews validate that delivered features meet acceptance criteria.

3.2.2 Sprint Plan for This Project

The project was organized into **eight Sprints** over the development period. Table 3.1 summarizes the Sprint plan:

Table 3.1: Sprint plan and deliverables

Sprint	Duration	Focus Area	Deliverables
1	2 weeks	Project Setup & Multi-Tenant Architecture	Django project scaffold, TenantUser model, PostgresServer model, TenantDatabase model, database routing, user approval workflow
2	2 weeks	Core ERP Modules (Finance & HR)	Company, Currency, Department, Employee, Attendance, Payroll, Invoice, Payment, Expense, Chart of Accounts
3	2 weeks	CRM, Supplier & Procurement	Lead, LeadActivity, Campaign, Customer, Supplier, PurchaseOrder, SupplierInvoice, SupplierPayment
4	2 weeks	Inventory/WMS & POS	Warehouse, zones/bins, StockMovement, GoodsReceipt, StockTransfer, CycleCount, PickList, ShipmentNote, ReorderRule, POS terminal/session/order
5	2 weeks	ML Model Development	AI model registry, 8 ML service modules, training pipeline, prediction logging, AI Insight engine
6	2 weeks	Data Integration & ML Tuning	Dataset import commands, synthetic data seeding, Prophet hyperparameter tuning, KPI calibration, model evaluation
7	2 weeks	Testing, Documentation & Deployment	Integration testing, API documentation, deployment configuration (Gunicorn + Docker), thesis writing
8	2 weeks	Explainable AI Module	ExplainabilityService (SHAP TreeExplainer + LIME TabularExplainer), ExplanationResult model, XAI API endpoints, XAI web views and templates, global/instance

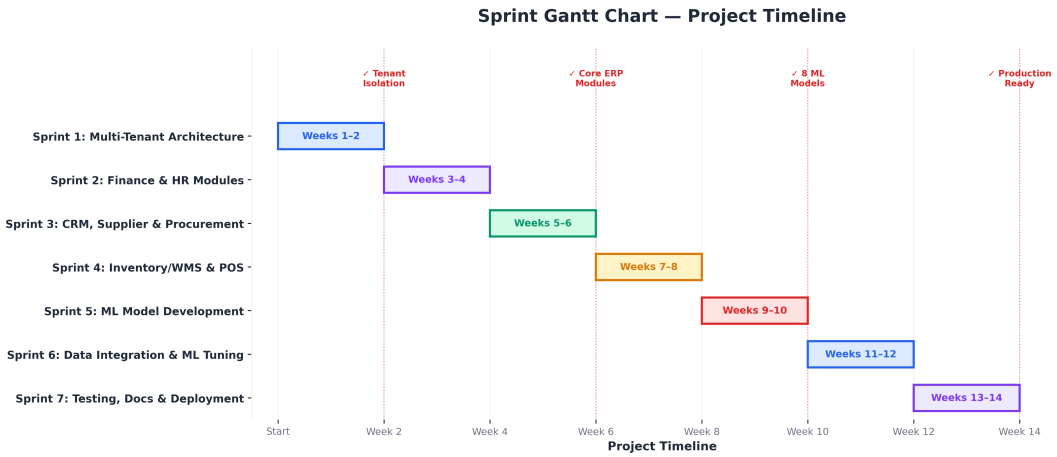


Figure 3.2: Gantt chart showing Sprint timeline and dependencies

3.2.3 Sprint Roles

Although this is an individual graduation project, the Sprint methodology roles were adapted as follows:

- **Product Owner:** The project supervisor, who reviewed Sprint deliverables and provided feedback on priorities and acceptance criteria.
- **Scrum Master / Developer:** The student (Ahmed AMeen Hassan Alkuraimi), responsible for Sprint planning, development, testing, and retrospective documentation.
- **Stakeholders:** The academic committee and potential end users who provided domain feedback during the CRM, Finance, and HR module development.

3.2.4 Justification for Sprint Methodology

The Sprint methodology was chosen over alternative SDLC models for the following reasons:

Table 3.2: SDLC model comparison and justification

SDLC Model	Strengths	Limitation for This Project
Waterfall	Clear phases, thorough documentation	Does not accommodate changing requirements; ML models require iterative tuning
V-Model	Strong testing emphasis	Sequential nature delays integration testing
RAD	Rapid prototyping	Insufficient emphasis on architecture for a system with 80+ models
Sprint (Agile)	Iterative delivery, accommodates ML experimentation, continuous feedback	Requires discipline in documentation (addressed through Sprint reviews)

3.3 System Architecture

3.3.1 High-Level Architecture

The system follows a three-tier architecture: Presentation Layer (Django templates + REST API), Application Layer (Django views, services, and ML pipelines), and Data Layer (PostgreSQL databases with multi-tenant routing).

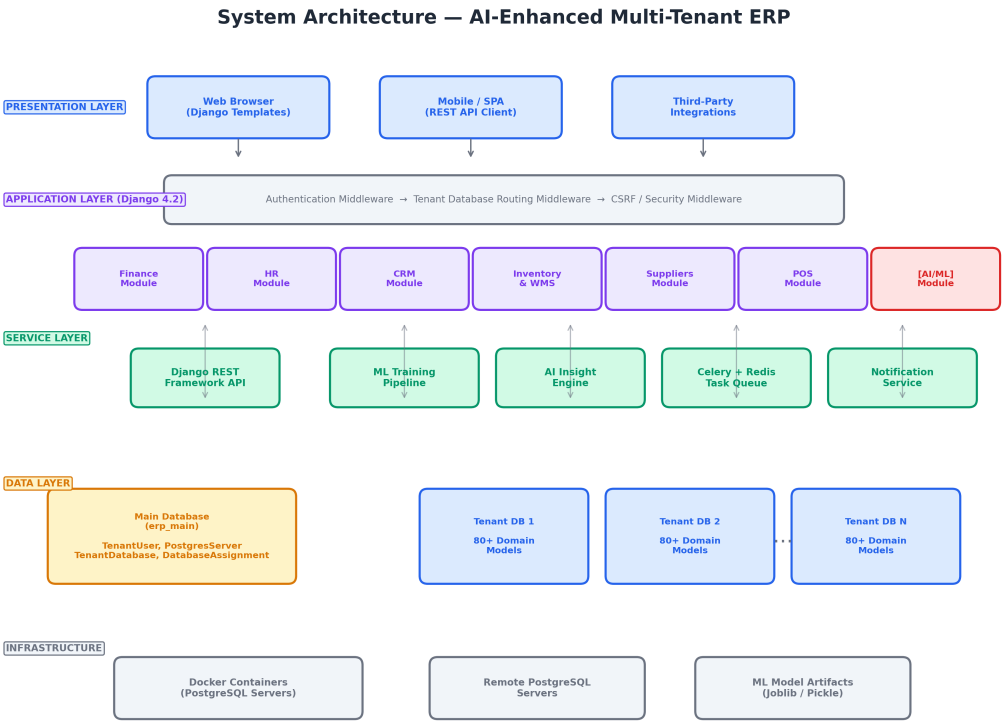


Figure 3.3: High-level system architecture showing the three-tier design with multi-tenant database routing

3.3.2 Multi-Tenant Database Architecture

The multi-tenant architecture is implemented using the database-per-tenant isolation model. The platform maintains a **main database** (`erp_main`) that stores user accounts, server configurations, and tenant metadata. Each approved tenant organization receives a dedicated PostgreSQL database provisioned on a selected server (Docker container or remote server).

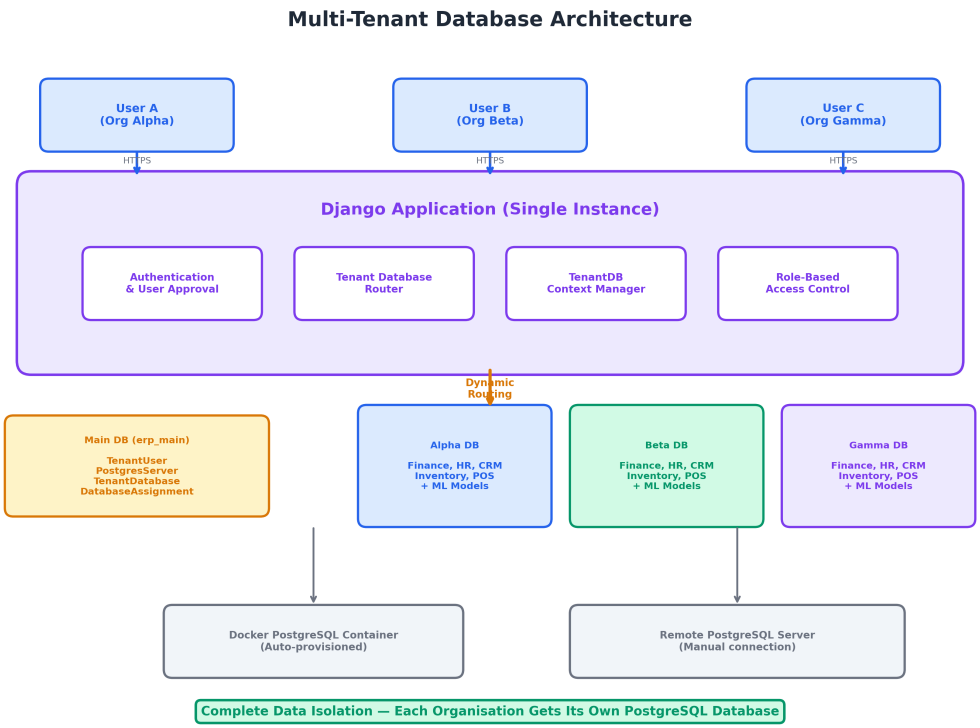


Figure 3.4: Database-per-tenant architecture with dynamic PostgreSQL provisioning

The database routing mechanism works as follows:

1. The user logs in and the authentication middleware identifies their `DatabaseAssignment`.
2. The `add_tenant_database()` function dynamically registers the tenant's database connection parameters in Django's `DATABASES` dictionary.
3. The `TenantDatabaseContext` context manager sets the active database alias for the current request thread.
4. All subsequent ORM queries are automatically routed to the tenant's database via the custom `DATABASE_ROUTERS` configuration.
5. When the request completes, the context manager resets the routing.

3.3.3 AI/ML Pipeline Architecture

The ML pipeline is integrated into the ERP as a self-contained module (`erp.ai`) with the following components:

- **Model Registry** (`MLModel`): Tracks model versions, accuracy scores, training parameters, and artefact file paths. Each model is scoped to a specific company (tenant).

- **Service Layer** (`erp.ai.services`): Contains the implementation of each ML model as a Python class. Each service class follows a consistent interface: `train()`, `predict()`, and an optional `score_all()` batch method.
- **Explainability Service** (`erp.ai.services.explainability`): The XAI module, encapsulated in the `ExplainabilityService` class. It provides:
  - `explain_prediction()` — Instance-level SHAP or LIME explanation for a single entity (invoice, lead, or employee), returning feature contributions, base values, top-feature rankings, and waterfall plot data.
  - `global_feature_importance()` — Global SHAP analysis using a background sample (up to 200 instances), producing mean |SHAP| importance rankings, beeswarm data, and summary plots (base64-encoded PNG).
  - `get_explanation_history()` — Retrieval of persisted explanations from the `ExplanationResult` model with optional filters by model type, entity, or method.

  Internally, the service dispatches to `_shap_explain()` (using `shap.TreeExplainer`) for tree-based models (GBM, Random Forest) and `_lime_explain()` (using `lime.lime_tabu` for model-agnostic fallback. It handles SHAP version differences (2D vs. 3D output arrays), scalar vs. array expected values, and applies artefact scalers when present.
- **Explanation Result Model** (`ExplanationResult`): Persists each explanation with the entity type, entity ID, explanation method (SHAP/LIME), prediction score, feature contributions (JSON), and top features for audit and historical analysis.
- **Training Pipeline** (`train_models` management command): An orchestrator that sequentially trains all selected models, evaluates their performance, and generates a JSON training report.
- **Prediction Logging** (`PredictionLog`): An immutable audit trail of every prediction made, enabling future retraining on actual outcomes.
- **AI Insight Engine** (`AIInsight` + `create_or_update_insight`): Converts ML model outputs into human-readable alerts with severity levels, recommendations, and action URLs. Includes automatic deduplication via a configurable cooldown window.

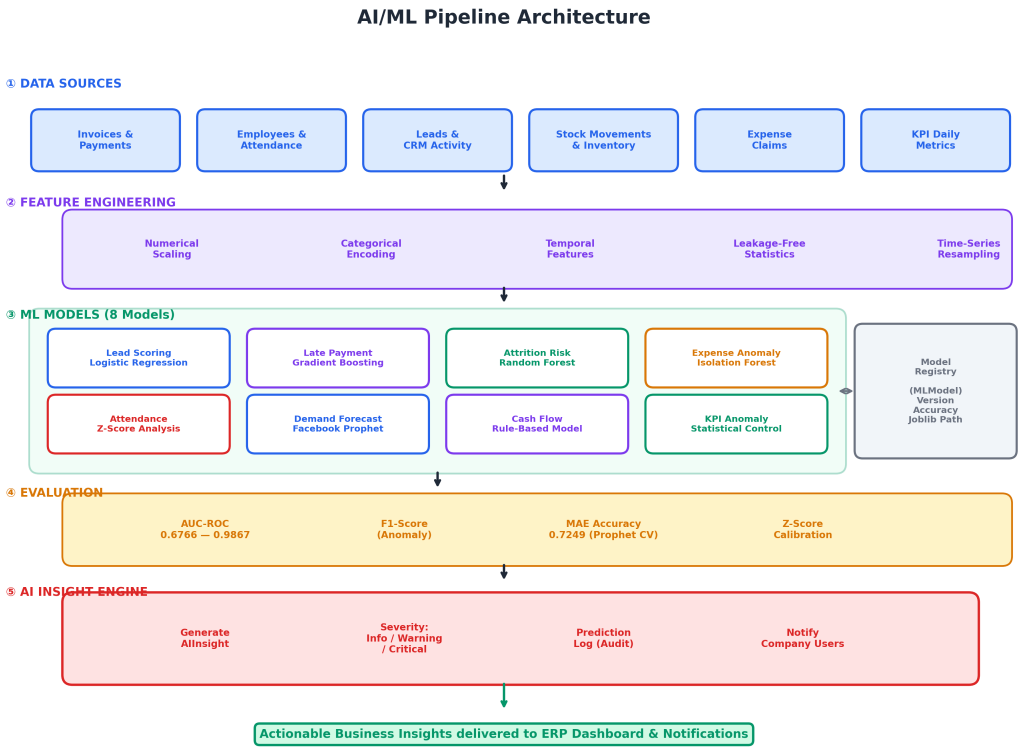


Figure 3.5: AI/ML pipeline architecture showing the flow from raw data to actionable insights

3.4 Technology Stack

Table 3.3 summarizes the technologies used in this project:

Table 3.3: Technology stack

Layer	**Technology**	**Purpose**
Backend Framework	Django 4.2	Web framework, ORM, URL routing, template engine
Programming Language	Python 3.12	Primary development language
Database	PostgreSQL 12+	Relational database for main and tenant databases
REST API	Django REST Framework 3.14	RESTful API layer for mobile/third-party integration
ML / Data Science	scikit-learn 1.4, pandas 2.0, numpy 1.26	Classical ML algorithms and data processing
Explainable AI	SHAP 0.51, LIME 0.2, matplotlib 3.10	SHAP TreeExplainer, LIME TabularExplainer, summary plots
Time-Series Forecasting	Facebook Prophet 1.1.5	Additive regression for demand

3.5 Data Collection and Preparation

The ML models in this project are trained on a combination of real-world open-source datasets and synthetic domain-specific data generated to fill coverage gaps:

3.5.1 Open-Source Datasets

1. **Olist Brazilian E-Commerce Dataset** (Olist, 2018): A public dataset from Kaggle containing over 100,000 real e-commerce orders from Brazilian marketplaces (2016–2018). This dataset provides:

   - 99,441 orders with order dates, delivery dates, and payment information
   - 32,216 products across 73 categories
   - 96,096 customers with geolocation data
   - 3,095 sellers

   The Olist dataset was used to populate the Customers, Products, Invoices, Orders, and Payments modules.

2. **IBM HR Employee Attrition Dataset** (IBM, 2017): A synthetic but statistically realistic dataset of 1,470 employee records with 35 attributes including salary, department, overtime hours, and attrition labels. This dataset trains the Employee Attrition Prediction model.

3. **Supplier Procurement Dataset**: A custom-generated dataset of 50 suppliers and 873 purchase orders with realistic procurement cycles, used to populate the Supplier and Purchase Order modules.

4. **Attendance Timeseries Dataset**: A custom-generated dataset of employee attendance records spanning 12 months with check-in/check-out times, hours worked, late arrival flags, and overtime hours.

5. **Expense Claims Dataset**: A custom-generated dataset of 6,278 approved expense claims with embedded anomalies (labeled `is_anomaly` column) for Isolation Forest evaluation.

6. **CRM Leads Dataset**: A custom-generated dataset of 4,062 leads with source, estimated value, activities, and won/lost outcomes.

3.5.2 Synthetic Data Generation

For modules without corresponding real-world datasets—particularly Inventory/WMS and KPI monitoring—a data seeding utility (`seed_ml_operational_data`) was developed. This management command generates:

- **Stock Movements:** 365 days of daily demand history for 80 products with weekly and monthly seasonality, product diversity profiles (fast/moderate/slow movers), weekend dips, growth trends, and random noise.
- **Reorder Rules:** 80 product-warehouse rule configurations with randomized lead times, safety stock levels, and minimum order quantities.
- **Supplier Invoices:** 220 pending supplier invoices with due dates spanning the next 90 days for Cash Flow projection.
- **KPI Spike Injection:** Calibrated inventory movement spikes targeting z-score $\approx 3.5\sigma$ above the 90-day baseline for KPI Anomaly Detection testing.

3.6 Machine Learning Model Design

3.6.1 Model Selection Criteria

The ML models were selected based on four criteria:

1. **Interpretability:** Business users must understand why a prediction was made. Models with native feature importance (Random Forest, Gradient Boosting) and calibrated probabilities (Logistic Regression) were preferred.
2. **Data Efficiency:** SME datasets are typically small (hundreds to low thousands of records). Models that perform well with limited data (Prophet, Z-Score, Isolation Forest) were favored.
3. **Operational Simplicity:** Models must train in seconds to minutes on a standard server, serialize as compact files (joblib), and produce predictions with sub-second latency.
4. **Task Appropriateness:** Each algorithm was matched to its problem type—classification (Logistic Regression, GBM, Random Forest), unsupervised anomaly detection (Isolation Forest, Z-Score), time-series forecasting (Prophet), and rule-based projection (Cash Flow).

3.6.2 Model Specifications

Table 3.4 summarizes the eight ML models implemented in this project:

Table 3.4: ML model specifications

Model	Domain	Algorithm	Features	Evaluation Metric	Min. Samples
Lead Scoring	CRM	Logistic Regression (C=1.0)	7 features (value, age, activities, source, contact info)	AUC-ROC	30
Late Payment	Finance	Gradient Boosting (300 trees, depth=5, lr=0.08)	10 features (amount, days-to-due, customer history)	AUC-ROC	30
Attrition	HR	Random Forest (300 trees, depth=8, balanced)	10 features (tenure, salary, attendance, overtime)	AUC-ROC	30
Expense Anomaly	Finance	Isolation Forest (300 trees, contamination=3%)	4 features (amount, month, weekday, category)	F1-Score vs. ground truth	30
Attendance Anomaly	HR	Z-Score (threshold=2.0)	2 metrics (avg hours, late arrival ratio)	Anomaly count	1
Demand Forecast	Inventory	Prophet (additive, weekly + monthly seasonality)	Daily demand history per product/warehouse	MAE-based accuracy via cross-validation	30

Table 3.4: ML model specifications (continued)

Model	Domain	Algorithm	Features	Evaluation Metric	Min. Samples
Cash Flow	Finance	Rule-based (30/60/90-day projection)	Outstanding invoices + pending payables + open POs	Net cash position	0
KPI Anomaly	Operations	Z-Score (threshold $\geq 2\sigma$ warning, $\geq 3\sigma$ critical)	5 daily KPIs (revenue, expenses, leads, overdue invoices, inventory movements)	Alert accuracy	14

3.6.3 Model Evaluation Strategy

Each model is evaluated using appropriate metrics for its task type:

- **Classifiers** (Lead Scoring, Late Payment, Attrition): AUC-ROC on a stratified 80/20 train/test split. AUC-ROC is preferred over accuracy because the datasets exhibit class imbalance.
- **Anomaly Detectors** (Expense Anomaly): Precision, Recall, and F1-Score computed against a ground-truth `is_anomaly` label in the expense dataset.
- **Time-Series Forecasts** (Demand): MAE-based accuracy ($1 - \frac{MAE}{\bar{y}}$) computed via Prophet's built-in cross-validation with expanding windows.
- **Statistical Detectors** (Attendance Anomaly, KPI Anomaly): Qualitative evaluation based on the number of alerts generated and their clinical reasonableness (e.g., z-score magnitude aligning with injected anomalies).
- **Cash Flow**: Validation against known payables and receivables in the database—checking that inflows equal outstanding invoice balances and outflows match pending supplier invoices.

3.6.4 XAI Evaluation Methodology

The Explainable AI module is evaluated along four dimensions:

1. **Explanation Validity (Additivity):** For SHAP explanations, the sum of all feature attributions plus the base value must equal the predicted probability (Equation 2.1). Any deviation exceeding ± 0.01 indicates a computational error. This property is verified programmatically for every instance-level explanation.

2. **Feature Coherence (Global):** For each model, the global SHAP feature importance distribution is analysed:

   - The fraction of total importance held by the top feature is measured. A healthy model should spread importance across multiple features; dominance above 50% triggers a data-leakage investigation.
   - The number of features with non-zero mean |SHAP| is counted. If the model ignores most features (zero SHAP), the feature engineering may be deficient.

3. **Cross-Method Agreement:** For a set of representative instances, the top-5 features from SHAP and the top-5 features from LIME are compared. The overlap ratio (Jaccard similarity) quantifies explanation consistency. High agreement ($\geq 3/5$) increases confidence; low agreement ($\leq 1/5$) signals local non-linearity or LIME approximation instability.

4. **Data-Leakage Detection:** SHAP importance analysis is used to detect two types of leakage:

   - *Temporal leakage:* Features computed using future information relative to the prediction timestamp (e.g., `days_since_created` for historical leads).
   - *Proxy-target leakage:* Features that encode the target variable through a correlated proxy (e.g., `payroll_months` correlating with employment status).

3.7 Development Tools and Environment

Table 3.5: Development tools and environment

Category	Tool	Purpose
IDE	Visual Studio Code	Primary code editor with Python and Django extensions
AI Pair Programming	GitHub Copilot (Claude)	AI-assisted code generation and debugging
Version Control	Git + GitHub	Source code management and collaboration
Database Client	pgAdmin 4	PostgreSQL database management and querying
API Testing	Django REST Framework browsable API	Interactive API testing and documentation
Virtual Environment	Python venv	Dependency isolation
Operating System	Windows 11	Development machine OS
Browser	Google Chrome DevTools	Frontend debugging and responsive design testing

3.8 Summary

This chapter detailed the Sprint (Agile) methodology adopted for the project, breaking the development into eight two-week Sprints covering platform architecture through XAI evaluation and deployment. The three-tier system architecture with database-per-tenant multi-tenancy was described, along with the AI/ML pipeline that trains, evaluates, logs, and surfaces model outputs as actionable business insights, augmented by an Explainability Service that provides SHAP and LIME explanations. The technology stack centres on Django 4.2, PostgreSQL, scikit-learn, SHAP 0.51, LIME 0.2, and Facebook Prophet. Data collection

combines six real-world and synthetic datasets totalling over 140,000 records across five business domains. The XAI evaluation methodology was defined with four dimensions: explanation validity, feature coherence, cross-method agreement, and data-leakage detection. The following chapter presents the detailed system analysis and design, including use case diagrams, class diagrams, and database schema design.

Chapter 4

System Analysis and Design

4.1 Introduction

This chapter presents the functional and non-functional requirements, use case modelling, class design, database schema, and the XAI pipeline architecture that together define the system's blueprint. The design builds on the architectural decisions described in Chapter 3 and provides the specification from which the implementation (Chapter 5) was realised.

4.2 Functional Requirements

The system's functional requirements are organised into three tiers: platform-level, ERP-module-level, and AI/XAI-level.

4.2.1 Platform Requirements

Table 4.1: Platform-level functional requirements

ID	Requirement	Description
FR-01	User Registration	Users can register with email, password, company name, and optional preferred language.
FR-02	Tenant Approval	An administrator can approve or reject pending tenant registrations.
FR-03	Database Provisioning	Approved tenants trigger automatic PostgreSQL database creation on a configured server (Docker or remote).
FR-04	Role-Based Access	Four roles (Owner, Admin, User, Read-Only) control access to ERP functions within a tenant.
FR-05	Tenant Switching	Users with multiple database assignments can switch between tenant contexts.
FR-06	Session Isolation	Each request is routed to the correct tenant database via middleware and context managers.

4.2.2 AI/XAI Requirements

Table 4.2: AI and XAI functional requirements

ID	Requirement	Description
FR-20	Model Training	Admin users can trigger ML model training via a management command or Celery task.
FR-21	Model Versioning	The system maintains a version history of trained models with accuracy metrics.
FR-22	Batch Scoring	All eligible entities (invoices, leads, employees) can be scored in batch after training.
FR-23	AI Insights	Model outputs are converted into human-readable alerts (Info, Warning, Critical) and displayed in the ERP dashboard.
FR-24	**Instance Explanation**	Users can request a SHAP or LIME explanation for any individual prediction (invoice, lead, employee).
FR-25	**Global Importance**	Users can view the global SHAP feature importance ranking for any trained model.
FR-26	**Explanation History**	All explanations are persisted with timestamps, method, feature contributions, and prediction scores for audit.
FR-27	**XAI Visualisation**	Waterfall plots, beeswarm charts, and feature-contribution tables are rendered via Plotly in the web interface.
FR-28	**XAI API**	REST endpoints expose instance explanation, global importance, and explanation history for third-party integration.

4.3 Non-Functional Requirements

Table 4.3: Non-functional requirements

ID	Requirement	Description
NFR-01	Data Isolation	Tenant data must be physically separated in dedicated PostgreSQL databases; cross-tenant queries must be impossible.

ID	Requirement	Description
NFR-02	Response Time	SHAP instance explanations must complete within 5 seconds; LIME within 10 seconds; global SHAP within 30 seconds.
NFR-03	Scalability	The system must support 50+ concurrent tenants on a single application server.
NFR-04	Explanation Determinism	SHAP explanations for the same input must produce identical results across runs (deterministic). LIME explanations may vary within stochastic bounds.
NFR-05	Bilingual Support	All UI labels, model fields, and XAI feature names must be available in English and Arabic.
NFR-06	Auditability	Every prediction and explanation is logged immutably for regulatory compliance.
NFR-07	Portability	The system runs on any OS with Python 3.12+ and PostgreSQL 12+.

4.4 Use Case Design

4.4.1 XAI Use Cases

The XAI module introduces three primary use cases:

UC-XAI-1: Explain Single Prediction: The user selects an entity (e.g., Invoice #1042) and requests an explanation. The system retrieves the trained model artefact, constructs the feature vector, invokes SHAP TreeExplainer (or LIME as fallback), and returns the waterfall plot data with ranked feature contributions.

UC-XAI-2: View Global Feature Importance: The user selects a model type (e.g., Late Payment). The system samples up to 200 background instances, computes mean |SHAP| values for each feature, and renders a beeswarm chart and bar plot.

UC-XAI-3: Compare SHAP vs. LIME: The user requests both explanation methods for the same entity. The system runs SHAP and LIME in sequence and displays the results side-by-side, highlighting agreement and divergence in top-5 feature rankings.



Figure 4.1: Use case diagram for the Explainable AI module

4.5 XAI Class Design

The XAI module follows a layered design pattern:

- **Service Layer** (`ExplainabilityService`): Stateless class instantiated with a `company` object. Orchestrates model loading, feature construction, explainer invocation, and result persistence.
- **Model Layer** (`ExplanationResult`): Django ORM model with fields for `model` (FK to `MLModel`), `entity_type`, `entity_id`, `method` (shap/lime), `prediction_score`, `feature_contributions` (JSONField), `top_features` (JSONField), and `created_at`.
- **Serialiser Layer** (`ExplanationResultSerializer`): DRF serialiser exposing explanation data via REST API.
- **View Layer**: Web views (`ExplainabilityDashboardView`, `ExplainPredictionView`, `GlobalImportanceView`) and API viewsets (`ExplainabilityViewSet`).



Figure 4.2: Class diagram for the ExplainabilityService and related components

4.6 Database Schema Design

4.6.1 AI/XAI Schema

The AI module adds the following tables to each tenant database:

Table 4.4: AI/XAI database tables

Table	Key Columns
`ai_mlmodel`	`model_type`, `version`, `accuracy_score`, `artefact_path`, `pa`
`ai_predictionlog`	`model` (FK), `entity_type`, `entity_id`, `prediction_score`, `f`
`ai_aiinsight`	`insight_type`, `severity`, `title`, `message`, `action_url`, `is_`
`ai_explanationresult`	`model` (FK), `entity_type`, `entity_id`, `method`, `prediction_`

4.7 XAI Pipeline Architecture

Figure 4.3 illustrates the flow of an explanation request through the system:

1. The user clicks “Explain” on an entity page (or calls the API endpoint).
2. The view layer invokes `ExplainabilityService.explain_prediction()`.
3. The service loads the model artefact (classifier + scaler + feature_cols) from the `MLModel` registry.

4. The feature vector is constructed by delegating to model-specific helper methods (e.g., `_features_late_payment()`) that reuse the same feature engineering logic as the prediction service.

5. For tree-based models, `shap.TreeExplainer` is initialised with the classifier and computes exact Shapley values. The 3D output array (for newer SHAP versions with binary classifiers) is handled by slicing the positive-class index.

6. The base value (expected value of the positive class) is extracted, handling both scalar and array formats.

7. Feature contributions are sorted by |SHAP value| descending, annotated with direction ("increases"/"decreases") and human-readable impact descriptions.

8. Waterfall plot data (cumulative bar segments from base value to prediction) is computed for frontend rendering.

9. The explanation is persisted to `ExplanationResult` and returned to the view/API.



Figure 4.3: Data flow for instance-level SHAP/LIME explanation generation

4.8 Implemented System Capabilities

To make the as-built scope explicit – the platform has grown substantially since the initial Sprint plan – this section enumerates the capabilities that are actually implemented and reachable in the current code base, organised by domain. Each bullet maps to a concrete Python module, Django app, REST endpoint, Celery task, or Flutter screen. Quantitative summary: 70+ ORM models across 13 model files, 40+ DRF `ModelViewSets`, 78 form

classes, 17 management commands, 23 Celery tasks (scheduled + async), 10 ML/AI service classes, a Flutter mobile app of 78 Dart files, and trilingual (Arabic / English / Indonesian) localisation with RTL support.

4.8.1 Multi-Tenant Platform (`tenants/` app)

- **Database-per-tenant** with a custom router (`tenants/routers.py`, `TenantDatabaseRoute`) that pins `auth`, `tenants`, and `sessions` to the infrastructure DB while routing every ERP model to the tenant's database; the active alias is set via a thread-local helper (`set_current_db_alias`).
- Models: `TenantUser` (email-based custom user with approval workflow: *pending / approved / rejected / suspended*), `PostgresServer`, `TenantDatabase`, `DatabaseAssignment`, `ServerLog`, `EmployeeInviteLink`, `ScheduledTask`, `TaskExecution`, `AlertRule`, `Alert`, `TenantQuota`, `UsageSnapshot`, `Announcement`, `AnnouncementDismissal`, `MetricSnapshot`, `TicketCategory`, `Ticket`, `TicketMessage`, `TicketAttachment`.
- Dynamic provisioning of either *Docker-managed* PostgreSQL containers or remote PostgreSQL servers, with health checks, restart, backup and restore from a dedicated platform-admin UI.
- **Platform-admin console** (`tenants/platform_admin/`, 18 view files) covering users & approvals, server management, database operations, monitoring/health, alerts, quotas, billing/usage, announcements, support tickets, scheduled tasks, mobile-app builds, backups, activity feed, Docker container introspection, and platform analytics.
- Tenant context is set at request time by `tenants.middleware.TenantMiddleware` from the user's `DatabaseAssignment` and propagated to every Celery task header so async ML jobs always run against the correct tenant DB.
- Quota enforcement (`TenantQuota` + `UsageSnapshot` + per-minute Celery refresh) and platform-wide announcements with per-user dismissal tracking.

4.8.2 Core ERP Domain Models (70+ Django Models)

- **Core** (`erp/models/core.py`): `Currency`, `Company`, `Department`, `Branch`, `FiscalYear`, `Sequence`.
- **Base mixins** (`base.py`): `TimeStampedModel`, `AuditMixin`, `SoftDeleteMixin`/`SoftDel` composed via `ERPBaseModel`.

- **Finance**: Account, TaxRate, PaymentMethod, Customer, Invoice/InvoiceLine, Payment/PaymentAllocation (multi-invoice allocation), Expense/ExpenseCategory/Ex JournalEntry/JournalEntryLine (debit/credit posting).
- **HR / Payroll**: Employee, EmployeeDocument, LeaveType, LeaveRequest, Attendance, Payroll/PayrollEarning/PayrollDeduction, AttendanceDevice (RFID/biometric).
- **CRM**: Lead, LeadActivity, CustomerInteraction, Campaign, CampaignLead, Task, TaskComment.
- **Suppliers & Procurement**: Supplier, SupplierContact, SupplierDocument, PurchaseOrder/PurchaseOrderLine, SupplierInvoice/SupplierInvoiceLine, SupplierPayment/SupplierPaymentAllocation.
- **Inventory & WMS**: UnitOfMeasure/UnitOfMeasureConversion, Warehouse/Wareho LotBatch, SerialNumber, StockItem, StockMovement (immutable ledger), GoodsReceiptNote/GoodsReceiptLine, StockTransfer, StockAdjustment, CycleCount/CycleCountLine with variance posting, PickList, ShipmentNote, ReorderRule, InventoryValuationConfig (FIFO / Weighted Average / LIFO).
- **Services & Products**: ServiceCategory, ServicePricing/PriceTier, ServicePacka Quote/QuoteLine, ProductCategory, Product, ProductSupplier.
- **Fixed Assets**: AssetCategory, FixedAsset, DepreciationSchedule, AssetDisposa AssetTransfer, AssetRevaluation, AssetMaintenanceLog.
- **POS**: POSTerminal, POSSession (open→close shift), POSOrder/POSOrderLine, POSPayment (split-tender).
- **Reporting**: ReportTemplate, GeneratedReport, Dashboard/DashboardWidget, KPI/KPIValue.
- **Audit & System**: AuditLog (generic FK, immutable), SystemSetting, Notification, EmailTemplate, FileAttachment, ScheduledTask, IntegrationLog.
- **AI / ML**: MLModel, PredictionLog, AIInsight, ExplanationResult, AITrainingRun.

4.8.3 REST and Mobile API Surface

- Single SimpleRouter mounts 40+ ModelViewSets under /api/ for currencies, companies, departments, employees, leaves, attendance, payroll, customers, invoices, payments, expenses, suppliers, POs, leads, campaigns, tasks, services, quotes, every

inventory entity (UoM, warehouses, zones, bins, lots, serials, stock items/movements, GRNs, transfers, adjustments, cycle counts, pick-lists, shipments, reorder rules), KPIs, dashboards, audit logs and notifications.

- Custom permission classes `IsCompanyUser` and `IsAIEnabled`; ViewSets inherit from a shared `ERPBaseViewSet` that injects company filtering, ordering and search.
- Financial reports API (`erp/api/reports_views.py`): `TrialBalanceAPIView`, `ProfitAndLossAPIView`, `BalanceSheetAPIView`, `CashFlowAPIView`.
- AI/ML API (`erp/ai/api/`): `MLModelViewSet`, `PredictionLogViewSet`, `AIInsightViewSet`, `ExplanationResultViewSet`; trigger endpoints for lead-scoring, invoice-scoring, expense-anomaly, demand-forecast, attrition, KPI-checks; per-model retraining endpoints and a *retrain-all* endpoint; `/api/ai/training/{status,hist` `/api/ai/cashflow/`; `/api/ai/assistant/` (LLM); `/api/ai/dashboard/`; `/api/ai/xai/explain/` and `/api/ai/xai/global/<model_type>/`.
- **Mobile API** (`erp/api/mobile/`): query-optimised endpoints for the Flutter app (employee dashboard, attendance, leave requests, payslips, tasks, POS, admin dashboards/lists/approvals), documented in `docs/admin_mobile_api.md`.

4.8.4 Server-Rendered Views and Templates

- 30 view modules under `erp/views/` covering: dashboard, customers, CRM, finance, HR, inventory, suppliers, employees, audit-log viewer, export, import, notifications, PDF generation, reports, tax report, journal posting, POS, products, services, fixed assets, attendance device pairing, employee portal, support tickets, UoM conversions, account balances, AJAX endpoints, plus a `_common.py` for auth/company filtering/pagination utilities.
- Setup wizard with seven steps (currency → company → fiscal year → tax → payment methods → chart of accounts → review) gated by `erp.middleware.ERPSetupMiddleware`.
- 40+ template directories under `erp/templates/erp/` (one per functional area) plus PDF templates and shared `includes/` for pagination/filters/buttons. Tenants templates under `tenants/templates/` include the platform-admin sub-tree (alerts, analytics, announcements, backups, databases, db-ops, docker, health, invites, logs, mobile-app, monitoring, quotas, scheduled, servers, settings, tickets, users) and tenant-side auth pages (signup, login, pending approval, rejected, no-database).
- 78 form classes across nine `erp/forms/` modules, all built on shared `BootstrapFormMixin` + `CompanyFilterMixin` so every dropdown is automatically scoped to the active company.

4.8.5 Services Layer and Cross-Cutting Concerns

- **Services** (`erp/services/`): `AuditService` (+ `AuditMiddleware` for thread-local request context), `AccountingService` (multi-currency GL posting, balance reconciliation, dual-entry validation), `InventoryService` (FIFO/LIFO/WAvg costing, stock reservation, GRN/transfer processing), `InventoryAccountingService` (COGS posting, inventory-account sync), `POSService`, `NotificationService` (email + in-app, queued), `SequenceService` (per-fiscal-year auto-numbering), `ReportEngine` (custom-SQL + filter/group/aggregate → PDF/Excel/HTML), `PDFService` (`xhtml2pdf`-based), `ExportService` (entity-keyed `EXPORT_REGISTRY` with cache busting), `ImportDataService`.
- **Middleware**: `ERPSetupMiddleware`, `AuditMiddleware`, `TenantMiddleware`, `RequestIDMiddleware` (`erpproject/request_id.py`, also propagated into Celery task headers).
- **Decorators** (`erp/decorators.py`): `@tenant_required`, `@company_filter_require` plus standard Django `@permission_required`.
- **Signals** (`erp/signals.py`): `post_save` / `post_delete` → audit logging; `user_logged_in` / `user_logged_out` → auth event audit; export-cache invalidation increments a generation counter on model changes.
- **Context processors**: `tenants.context_processors.tenant_context` exposes current DB, approval status and assigned company to every template.
- **Rate limiting** (`erp/ratelimit.py`): form-based `@ratelimit(group='login', rate='5/min')`; DRF throttle classes `LoginThrottle` (1/s/user), `SignupThrottle` (5/min/IP), `HeavyOperationThrottle` (1/h for AI retraining and bulk export); state stored in Redis sliding window.
- **Concurrency & transactions**: `transaction.atomic` blocks throughout, `select_for_upda` on critical rows (e.g., `Currency.is_base`), dedicated `ml` Celery queue for heavy training jobs.
- **Caching**: Redis-backed sessions, throttle state, dashboard/report query cache, with graceful local-memory fallback.

4.8.6 AI / ML Stack

- Ten service classes under `erp/ai/services/`: `LeadScorer`, `LatePaymentPredictor`, `AttritionPredictor`, `ExpenseAnomalyDetector` (Isolation Forest), `AttendanceAnom` (Isolation Forest), `DemandForecaster` (Prophet), `CashFlowForecaster` (Prophet),

`KPIAnomalyDetector` (statistical z-score), `ERPAssistant` (OpenAI GPT-4o-mini, tenant-scoped), `ExplainabilityService` (SHAP `TreeExplainer` + LIME `TabularExplainer` + waterfall + global beeswarm + persisted `ExplanationResult` rows).

- **Honest-evaluation harness** in `erp/ai/services/base.py`: stratified K-fold via `evaluate_classifier`, `shuffle_label_auc` sanity test, time-based forward hold-out (used by `LatePaymentPredictor`), inverse-frequency sample weighting, and a per-model `parameters` JSON containing CV-AUC / hold-out AUC / shuffle-AUC / top-feature share.

- **Model registry**: `MLModel` stores artefact path, SHA-256 checksum, version, accuracy and parameters; `PredictionLog` records every prediction (entity, score, actual outcome) for downstream retraining ground-truth; `AIInsight` carries severity (*info / warning / critical*) and action URLs with deduplication; `ExplanationResult` stores SHAP/LIME outputs per prediction; `AITrainingRun` tracks live retraining (status, step, label, started/finished timestamps).

- **Async retraining** (Celery): `async_retrain_lead_scoring`, `async_retrain_late_pay` `async_retrain_attrition`, `async_retrain_demand_forecast`, `async_retrain_` plus inference jobs `score_open_invoices`, `score_active_leads`, `score_employee_a` `detect_expense_anomalies`, `detect_attendance_anomalies`, `forecast_cash_f` `run_kpi_checks`, `generate_xai_explanations`.

- **Celery beat schedule** (`erpproject/celery.py`): nightly retrains at 02:00–03:30 (demand, late-payment, expense anomalies, invoice scoring); weekly retrains on Sundays (lead scoring 01:00, attrition 02:00); monthly attrition re-fit on the 1st; platform telemetry every 1 min (`collect_metrics`, `dispatch_scheduled_tasks`), every 5 min (`evaluate_alerts`), every 15 min (`refresh_quotas`), with a 04:00 cleanup of historical metrics.

- **Datasets** bundled under `datasets/` for reproducible training: Olist e-commerce (orders, items, payments, reviews, customers, sellers, geolocation, products, category translation), IBM HR Employee-Attrition (`WA_Fn-UseC_-HR-Employee-Attrition.csv`), CRM lead seed, expense claims, attendance time-series, supplier procurement, and `generate_synthetic.py` for synthetic operational data.

4.8.7 Reporting, Analytics and Document Generation

- Custom report builder backed by `ReportTemplate` / `GeneratedReport`, executed by `ReportEngineService` with Plotly chart rendering and PDF export (`xhtml2pdf`).

- Pre-built financial statements (Trial Balance, Profit & Loss, Balance Sheet, Cash Flow) exposed as both HTML views and JSON APIs.
- Dashboard composer (`Dashboard` + `DashboardWidget`) and KPI library (`KPI` + `KPIValue` time series) consumed by both web and mobile clients.
- Asynchronous report generation via Celery (`generate_report`, locked per user) with retention-policy cleanup (`cleanup_old_reports`).

4.8.8 Management Commands and Data Tooling

- ERP commands (`erp/management/commands/`): `erp_first_setup`, `generate_financ` `import_attendance`, `import_expenses`, `import_ibm_hr`, `import_leads`, `import_olist`, `import_suppliers`, `migrate_tenants`, `recalculate_balances`, `seed_ml_operational_data`, `seed_tenant`, `set_password`, `train_models`.
- Tenant commands (`tenants/management/commands/`): `setup_tenants`, `migrate_tenants`, `check_servers`.
- `erp/factories.py`: Factory-Boy factories used both by tests and by seeders to populate realistic fixtures.

4.8.9 Mobile Companion App (Flutter, `mobile/`)

- 78 Dart files organised into `config/`, `l10n/`, `models/`, `providers/`, `screens/`, `services/`, `widgets/`; entry point `main.dart` wires a `MultiProvider` with 13 providers; routing and theming live in `app.dart`.
- **Models**: `Employee`, `Attendance`, `LeaveRequest`, `NotificationModel`, `Payslip`, `Task`, plus `pos/` and `admin/` sub-packages.
- **Screens**: admin sub-app with 10 screens (`admin_dashboard`, `admin_list`, `admin_shell`, `admin_approvals`, `admin_change_password`, `admin_notifications`, `admin_profile`, `admin_record_detail`, `admin_reports`, `admin_search`); employee sub-app with `auth/`, `dashboard/`, `attendance/` (incl. QR scanning), `leaves/`, `payslips/` (PDF download), `tasks/`, `notifications/`, `profile/`, `pos/`.
- **Providers** (state management): `auth_provider`, `dashboard_provider`, `admin_dashboar` `admin_list_provider`, `attendance_provider`, `leave_provider`, `payslip_provi` `task_provider`, `notification_provider`, `pos_provider`, `profile_provider`, `theme_provider`, `locale_provider`.

- **Services**: `api_service.dart` (HTTP client that injects an `X-Tenant-DB` header), `admin_api_service.dart`, `auth_service.dart`, `storage_service.dart` (secure local storage for token & preferences).
- **Widgets** (reusable UI): `app_drawer`, `avatar_widget`, `empty_state`, `icon_tile`, `loading_indicator`, `section_header`, `stat_card`, `status_badge`, plus a `pos/` widget package (keypad, payment methods).
- Cross-platform build targets: `android/`, `ios/`, `linux/`, `macos/`, `windows/`, `web/`.

4.8.10 Localisation and Accessibility

- Three languages with `i18n_patterns` URL prefixes (`/ar/`, `/en/`, `/id/`); compiled `django.mo` files under `locale/{ar,en,id}/LC_MESSAGES/`.
- Right-to-left rendering for Arabic and dual-language fields (`name` + `name_ar`, `description` + `description_ar`, `first_name` + `first_name_ar`) on Company, Department, Employee, Supplier and other masters.
- Locale-aware currency formatting via the `currency` template filter (`erp/templatetags/curr` the mobile app re-uses Flutter `intl` together with a `LocaleProvider` for in-app language switching; the API honours the `Accept-Language` header.

4.8.11 Audit, Security and Observability

- Immutable `AuditLog` captured via signals on every create / update / delete, with before-vs-after delta, user, IP and user-agent; soft-delete via `is_deleted` preserves history.
- Login/logout events logged through Django auth signals; integration calls logged to `IntegrationLog`.
- Generic `FileAttachment` (10 MB cap), `EmailTemplate` library, `SystemSetting` key/value store (typed: string, int, decimal, bool, JSON, text).
- **Request ID** (`erpproject/request_id.py`): generated per request, stored in thread-local, injected into every log line and propagated into Celery task headers for end-to-end tracing.
- **New Relic APM** (`newrelic.ini`) integrated at the WSGI level; structured logs in `logs/` rotated by logrotate; platform-side `MetricSnapshot` captures CPU, disk, DB connections and active-user counts every minute and feeds the `AlertRule` engine.

4.8.12 Testing

- 18 test modules under `erp/tests/`: API (`test_api.py`), audit, core, CRM, currency filters, finance, fixed assets, HR, inventory, journal balance, middleware, POS, reports API, services, signals, supplier, mobile-admin sub-package, plus shared `helpers.py`.
- Multi-tenant routing/approval/database-assignment tests in `tenants/tests.py`.
- Flutter widget tests under `mobile/test/` (golden files / UI tests).

4.8.13 Deployment and Operations

- One-shot installer `deploy/deploy.sh` (Ubuntu 22.04/24.04 LTS) that provisions Python 3.12, PostgreSQL 15, Nginx, Certbot, Docker, Redis, JDK, Flutter and the Android SDK; creates the `erp` system user under `/opt/erpdjango`; sets up a virtualenv; runs migrations and `collectstatic`; installs systemd units `erp-gunicorn`, `erp-celery`, `erp-celery-ml` (dedicated ML worker) and `erp-celery-beat`; configures Nginx reverse proxy with Let's Encrypt SSL; opens UFW; sets up logrotate and certbot renewal; and prepares the mobile build toolchain used by the platform-admin "Build mobile app" feature.
- Companion scripts: `deploy/update.sh` (pull, migrate, collectstatic, reload), `deploy/backup.` (DB dump to S3 or local), `deploy/patch_nginx_mobile.py` (deep-link rewriting).
- `docker-compose.yml` brings up `postgres:15-alpine` (volume `pgdata`, `pg_isready` healthcheck) and `redis:7-alpine` (volume `redisdata`, optional password); `Dockerfile` builds the web/Gunicorn image, `Dockerfile.ml` the ML-worker image with scikit-learn / Prophet / numpy / pandas pre-installed.
- Static-file delivery via WhiteNoise (Gzip + Brotli); pagination, `select_related`/`prefetch_re` discipline and composite indexes on hot paths; `django-debug-toolbar` in development.

4.9 Summary

This chapter defined the functional and non-functional requirements for the AI-enhanced ERP system with XAI capabilities, presented use case and class design for the explainability module, described the database schema extensions for explanation persistence, detailed the XAI pipeline architecture from user request to rendered waterfall plot, and enumerated the

as-built capabilities in Section 4.8. The following chapter describes the implementation of these designs.

Chapter 5

Implementation and Testing

5.1 Introduction

This chapter details the implementation of the system's key components, with particular focus on the Explainable AI module, ML model training pipeline, and multi-tenant infrastructure. It also describes the testing strategy and the procedures used to validate both traditional ML metrics and XAI outputs.

5.2 Project Structure

The Django project follows a modular structure with the AI/XAI module implemented within the `erp/ai/` package:

Listing 5.1: AI module directory structure

```
erp/ai/
  __init__.py
  models.py            # MLModel, PredictionLog, AIInsight,
                       # ExplanationResult
  serializers.py       # DRF serialisers for API
  api_views.py         # REST API viewsets
  views.py             # Web views for XAI dashboard
  urls.py              # URL patterns
  tasks.py             # Celery async tasks
  services/
    __init__.py
    base.py            # Shared utilities (load_model_artefact,
                       #   get_or_create_registry)
    finance.py         # LatePaymentPredictor,
                       #   ExpenseAnomalyDetector, CashFlowProjector
    crm.py             # LeadScorer
    hr.py              # AttritionPredictor,
                       #   AttendanceAnomalyDetector
    demand.py          # DemandForecaster
    insights.py        # InsightGenerator, KPIAnomalyDetector
    explainability.py  # ExplainabilityService (SHAP + LIME)
```

5.3 ExplainabilityService Implementation

The `ExplainabilityService` class (approximately 550 lines) is the core of the XAI module. The key implementation details are:

5.3.1 SHAP TreeExplainer Integration

For tree-based models (Late Payment GBM, Lead Scoring GBM, Attrition RF), the service uses `shap.TreeExplainer`:

Listing 5.2: SHAP instance explanation (simplified)

```
def _shap_explain(self, artefact, features, feature_names, model_type):
    classifier = artefact['classifier']
    X = features.copy()
    if 'scaler' in artefact:
        X = artefact['scaler'].transform(X)

    explainer = shap.TreeExplainer(classifier)
    shap_values = explainer.shap_values(X)

    # Handle binary classification output formats
    if isinstance(shap_values, list):
        sv = shap_values[1][0]  # positive class, first instance
    else:
        sv = np.array(shap_values)
        if sv.ndim == 3:        # newer SHAP: (samples, features, classes)
            sv = sv[0, :, 1]
        else:
            sv = sv[0]

    base_value = self._extract_base_value(explainer.expected_value)
    prediction = float(classifier.predict_proba(X)[0][1])

    # Build ranked feature contributions
    contributions = {}
    for name, shap_val, feat_val in zip(feature_names, sv, features[0]):
        contributions[name] = {
            'shap_value': round(float(shap_val), 6),
            'feature_value': round(float(feat_val), 4),
            'direction': 'increases' if shap_val > 0 else 'decreases',
        }
    return sorted_contributions, top_features, waterfall_data
```

5.3.2 LIME TabularExplainer Integration

For model-agnostic explanations and cross-method validation:

Listing 5.3: LIME instance explanation (simplified)

```
def _lime_explain(self, artefact, features, feature_names, model_type):
    background_df = self._get_background_data(model_type, artefact)
    X_bg = background_df[feature_names].fillna(0).values

    explainer = lime.lime_tabular.LimeTabularExplainer(
        training_data=X_bg,
        feature_names=feature_names,
        mode='classification',
        random_state=42,
    )
    explanation = explainer.explain_instance(
        X[0], classifier.predict_proba,
        num_features=len(feature_names),
    )
    # Map LIME feature descriptions back to original names
    lime_weights = explanation.as_list()
    for feat_desc, weight in lime_weights:
        matched_name = self._match_lime_feature(feat_desc, feature_names)
        contributions[matched_name] = {
            'lime_weight': round(float(weight), 6), ...
        }
```

5.3.3 Global Feature Importance

The global analysis computes mean |SHAP| across a background sample:

Listing 5.4: Global SHAP importance computation

```
def global_feature_importance(self, model_type):
    X_bg = background_df[feature_names].fillna(0).values
    if len(X_bg) > 200:
        indices = np.random.RandomState(42).choice(len(X_bg), 200,
            replace=False)
        X_bg = X_bg[indices]

    explainer = shap.TreeExplainer(classifier)
    shap_values = explainer.shap_values(X_bg)
    # ... handle 2D/3D, list formats ...
    mean_abs_shap = np.abs(sv).mean(axis=0)
    importance = dict(zip(feature_names, mean_abs_shap.round(4).tolist())
        )
```

5.3.4 Version Compatibility Handling

The implementation handles two critical SHAP version differences:

1. **Output dimensionality:** Older SHAP versions return a Python list `[neg_class_array, pos_class_array]` for binary classifiers; newer versions return a single 3D NumPy array of shape `(n_samples, n_features, n_classes)`. The code detects the format and slices accordingly.

2. **Expected value format:** `explainer.expected_value` may be a scalar (single-output models) or an array `[neg_ev, pos_ev]`. The `_extract_base_value()` helper safely handles both by converting to a NumPy array and checking `ndim`.

5.4 Multi-Tenant Implementation

5.4.1 Dynamic Database Registration

When a user authenticates, the middleware calls `add_tenant_database()` to register the tenant's connection parameters in Django's `DATABASES` dictionary:

Listing 5.5: Dynamic tenant database registration

```
def add_tenant_database(db_assignment):
    alias = f"tenant_{db_assignment.database.uuid}"
    if alias not in settings.DATABASES:
        settings.DATABASES[alias] = {
            'ENGINE': 'django.db.backends.postgresql',
            'NAME': db_assignment.database.db_name,
            'USER': db_assignment.database.server.username,
            'PASSWORD': db_assignment.database.server.password,
            'HOST': db_assignment.database.server.host,
            'PORT': db_assignment.database.server.port,
        }
    return alias
```

5.4.2 XAI in Multi-Tenant Context

The `ExplainabilityService` is always instantiated with a `company` object that belongs to the current tenant's database. All model registry queries and entity lookups are automatically routed to the correct tenant database by Django's ORM routing, ensuring that one tenant's explanations never access another tenant's data.

5.5 User Interface and System Screenshots

This section presents screenshots of the deployed system as evidence that the design and capabilities described in earlier chapters have been implemented and are operational. The screenshots are grouped into six categories: (i) the public landing page and authenticated dashboards, (ii) tenant administration and governance, (iii) finance and accounting, (iv) operations (inventory, point-of-sale, services and assets), (v) human resources and attendance, (vi) the AI/ML and Explainable AI modules, and (vii) the Flutter mobile companion. Each figure is captioned with the workflow it depicts and is referenced from the corresponding capability subsection in Section 4.8.

5.5.1 Landing Page and Dashboards

Figure 5.1 shows the public marketing landing page that anonymous visitors and prospective tenants see before signing up. After authentication, owners are redirected to the multi-tenant owner dashboard (Figure 5.2), which aggregates KPIs, recent activity, and shortcuts to the operational modules. Employees enter a role-restricted portal (Figure 5.3) that exposes only the workflows their position permits.

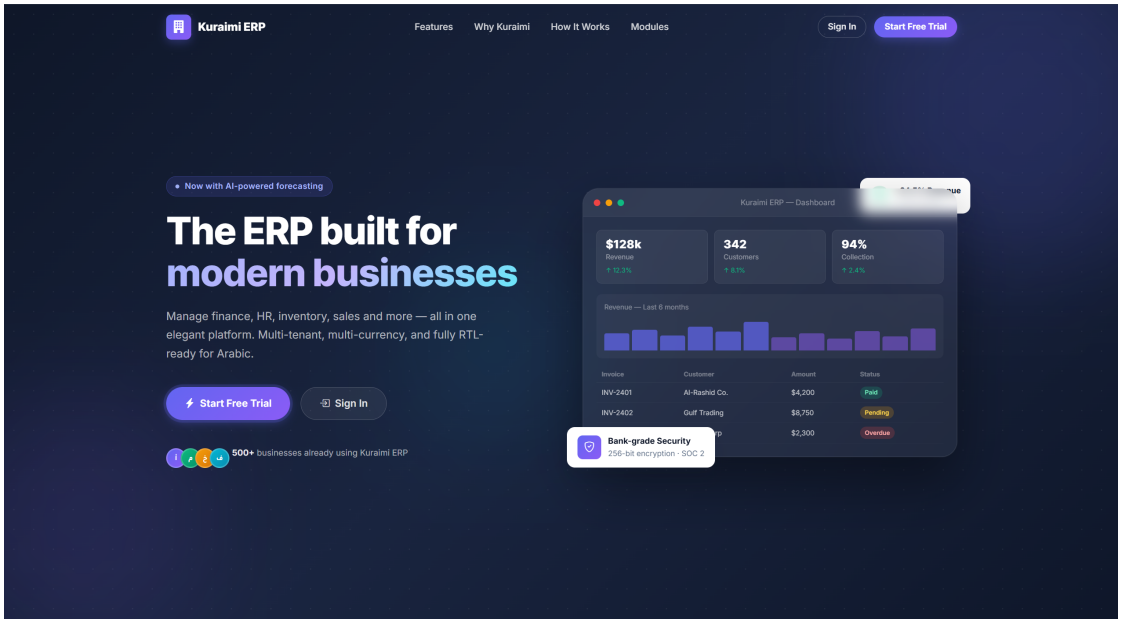


Figure 5.1: Public landing page of the ERP platform.

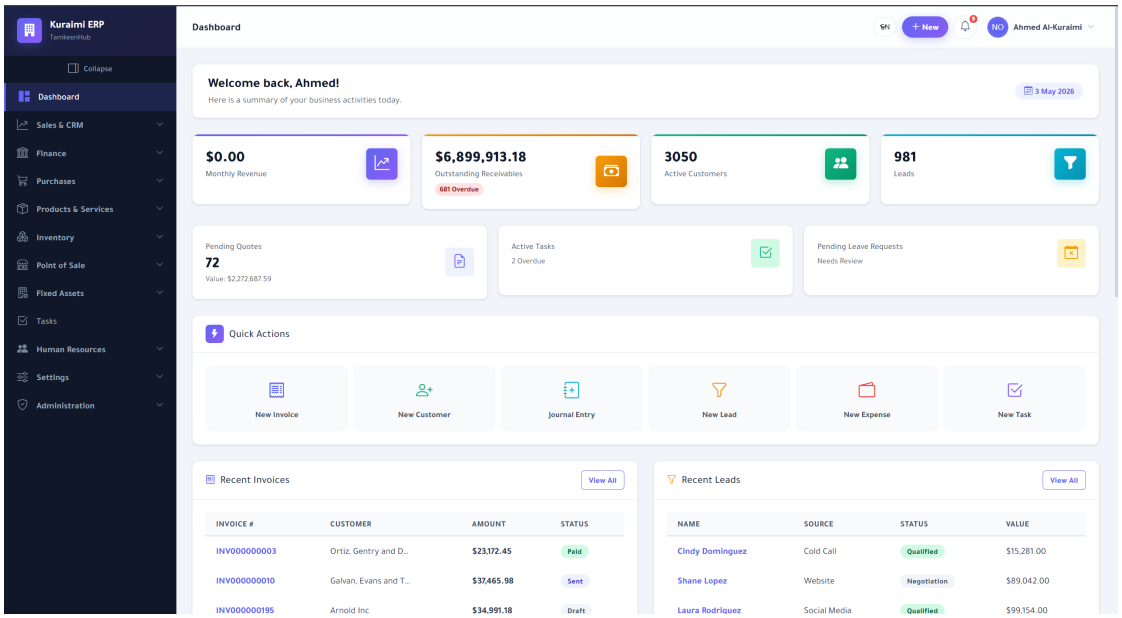


Figure 5.2: Owner dashboard with cross-module KPIs and quick links.

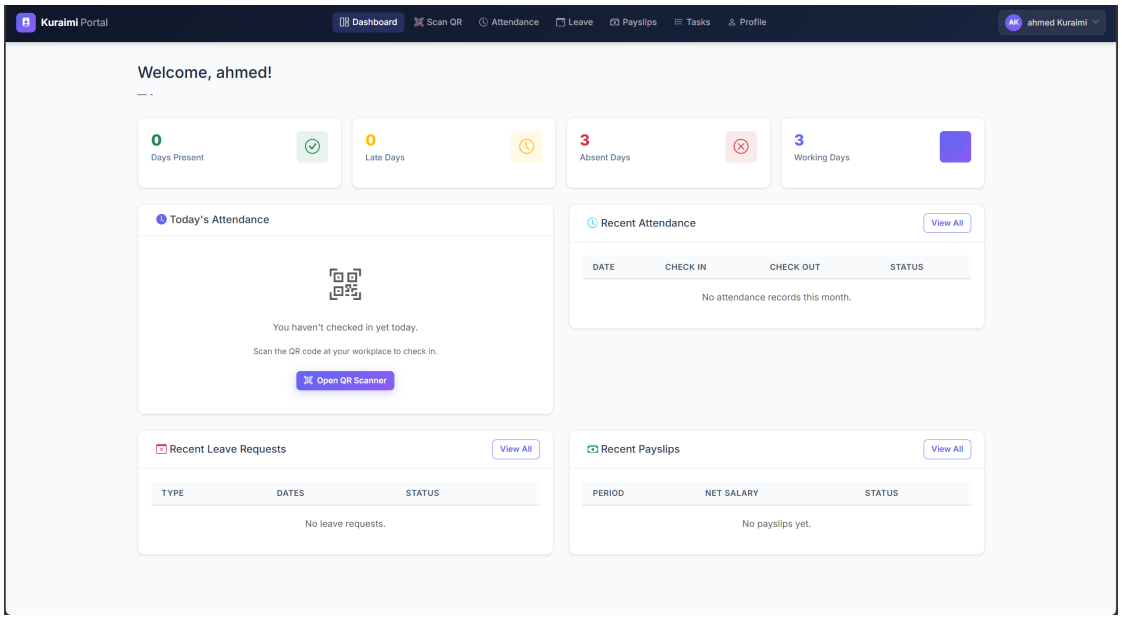


Figure 5.3: Employee web portal showing role-scoped navigation.

5.5.2 Tenant Administration and Governance

The administrative surface supports the full employee lifecycle and produces a tamper-evident audit trail. Figure 5.4 lists the tenant's employees, while Figure 5.5 shows the secure invitation-link workflow used to onboard new staff without exposing passwords. Privileged actions such as approvals and audit-log inspection are illustrated in Figures 5.6, 5.7, 5.8, and 5.9.

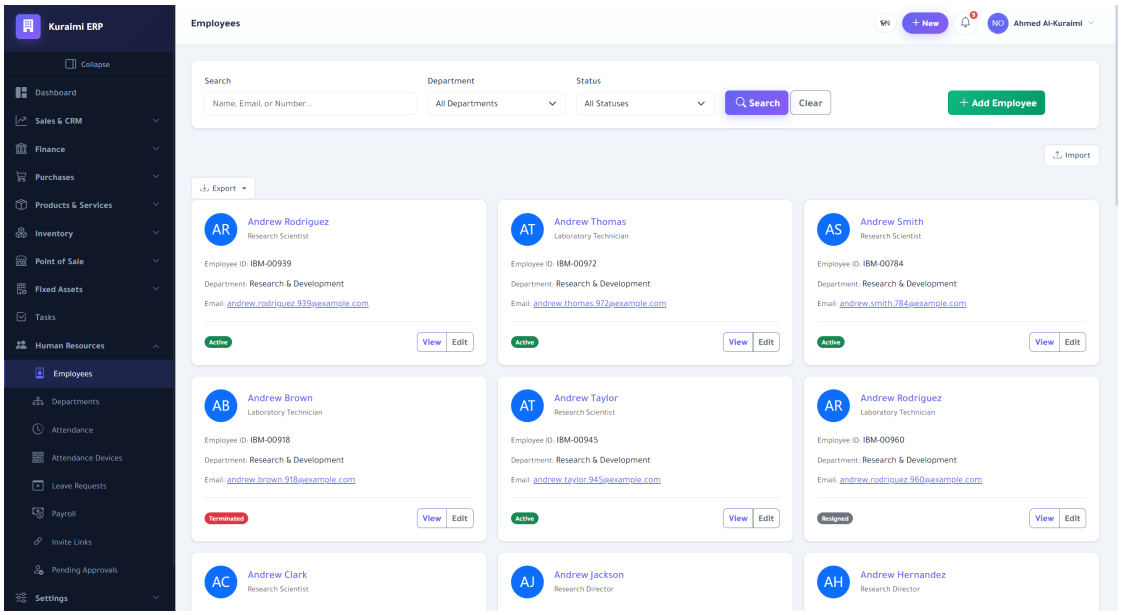


Figure 5.4: Employee directory with role and status filters.

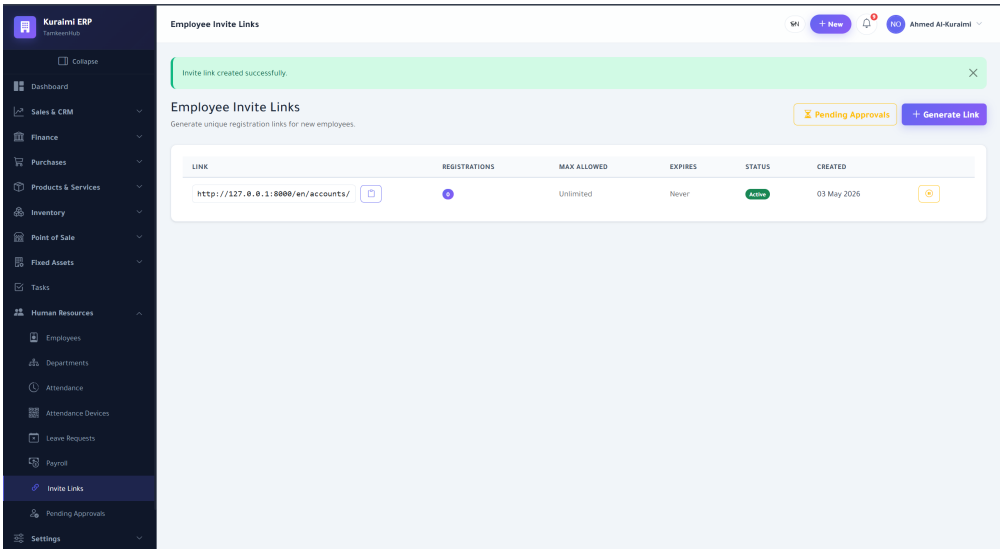


Figure 5.5: Secure invitation-link generation for onboarding employees.

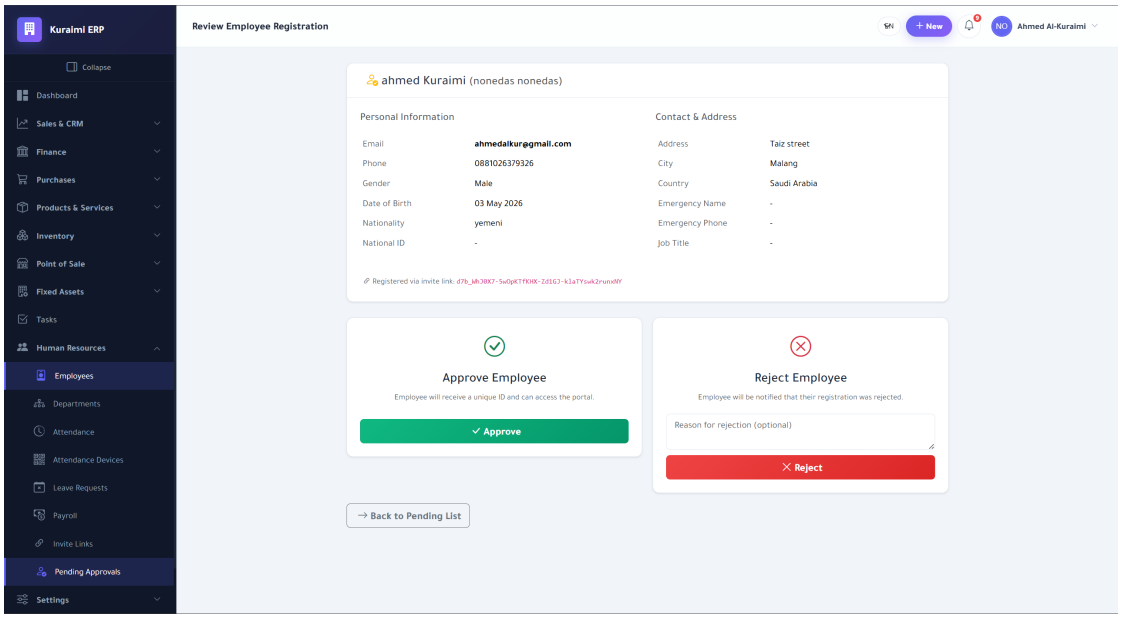


Figure 5.6: Owner review and approval queue for pending employee actions.

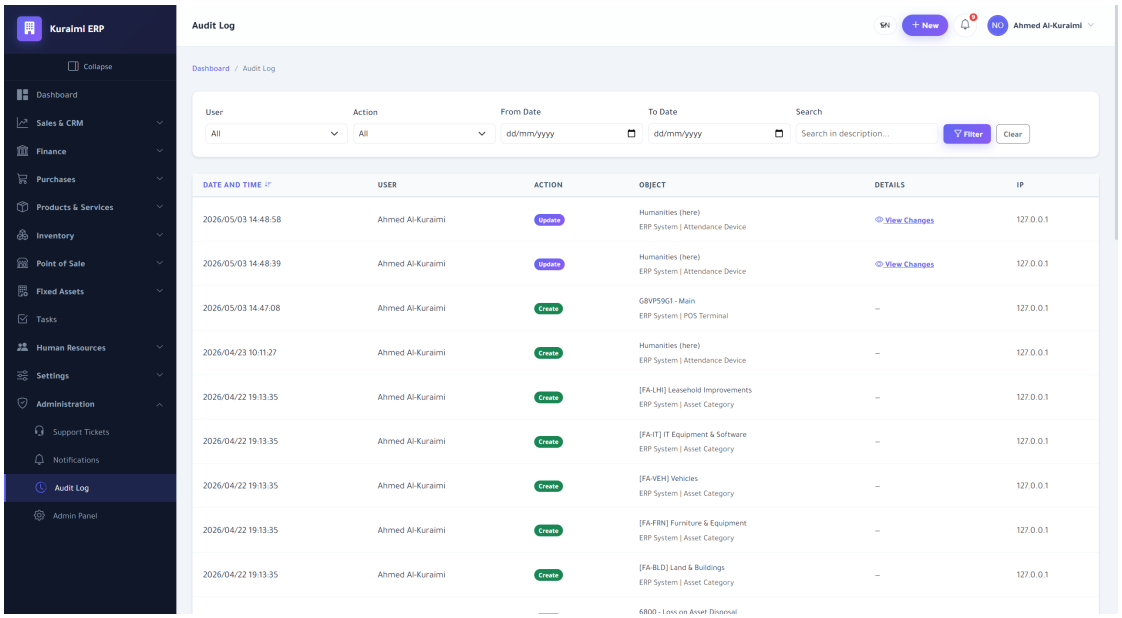


Figure 5.7: Tamper-evident audit log with actor, target, and timestamp.

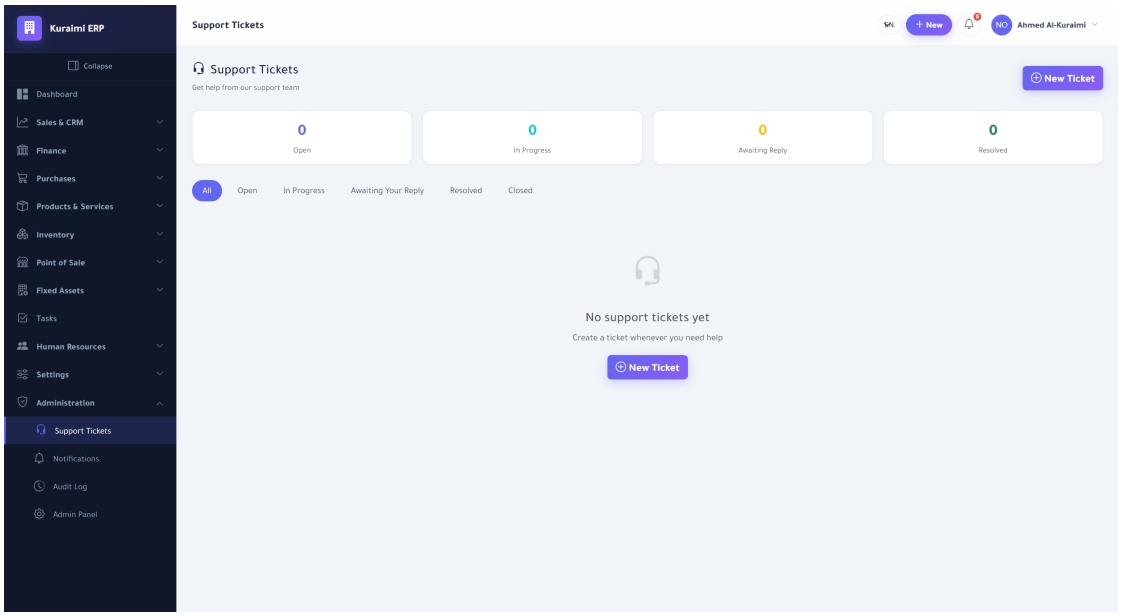


Figure 5.8: Support-ticket interface for tenant-to-platform communication.

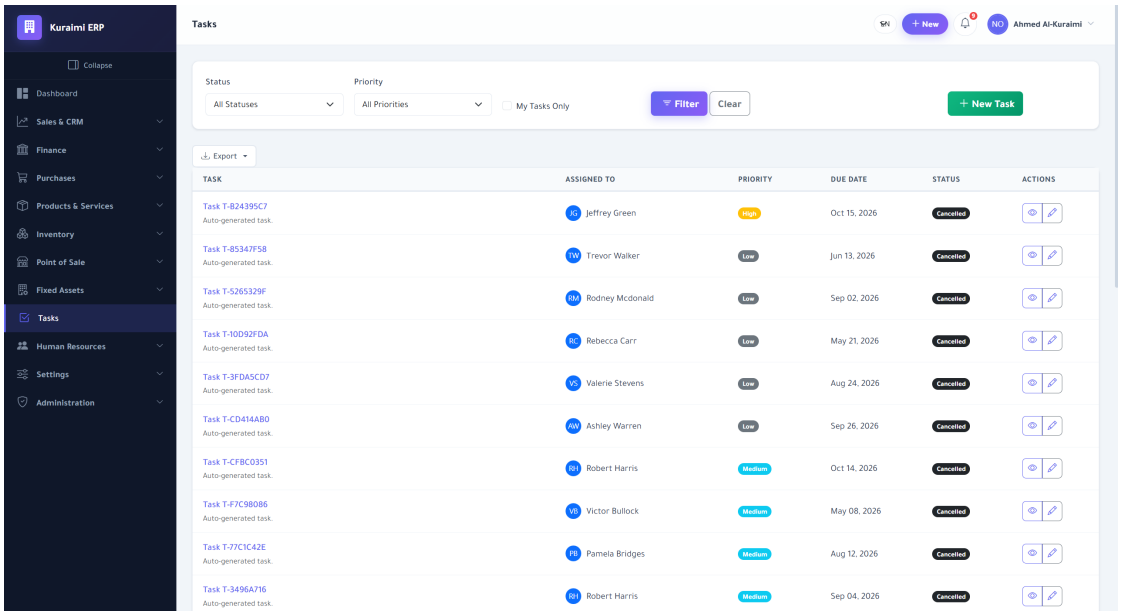


Figure 5.9: Task management board for assigning and tracking work items.

5.5.3 Finance and Accounting

The finance module implements double-entry journal entries (Figure 5.10), produces standard financial reports (Figure 5.11), exposes a configurable report builder for ad-hoc analysis (Figure 5.12), and tracks fixed assets with depreciation schedules (Figure 5.13).

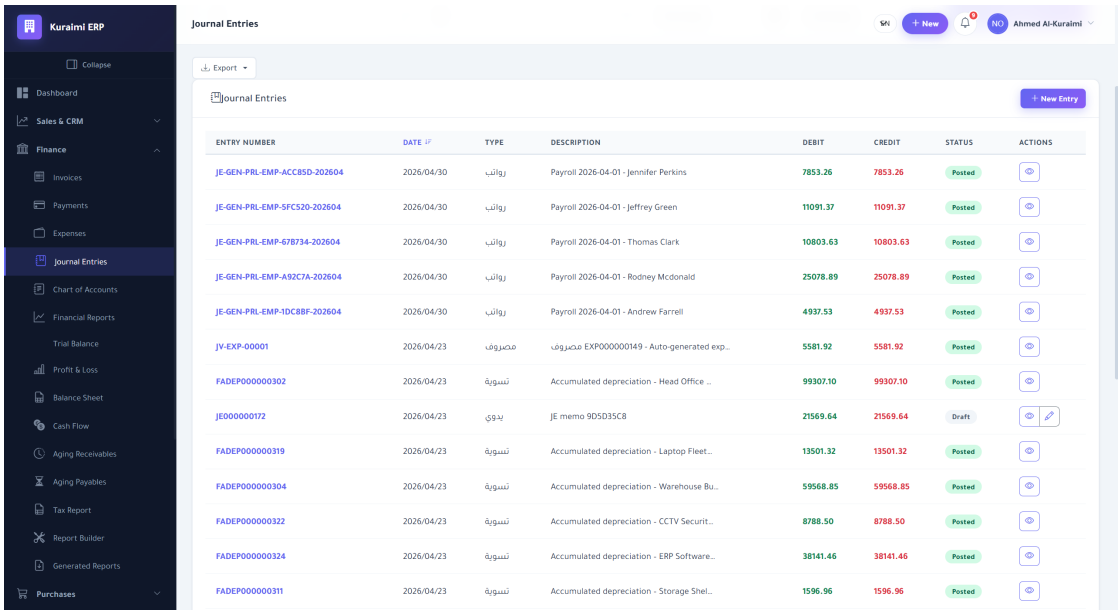


Figure 5.10: Journal-entry creation screen enforcing double-entry balance.

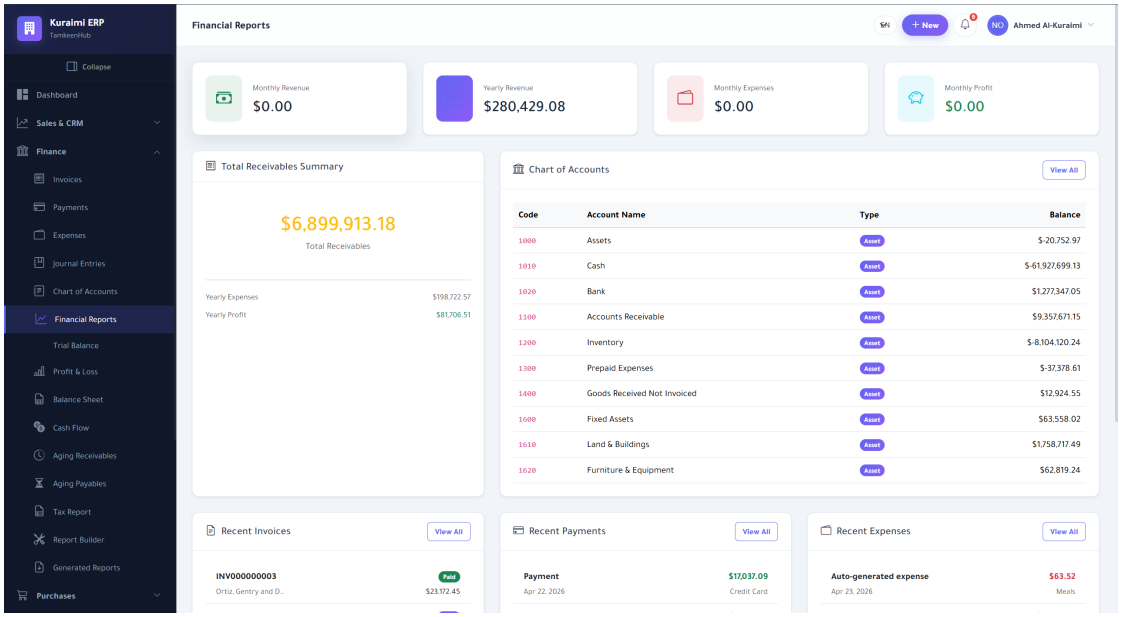


Figure 5.11: Standard financial reports (P&L, balance sheet, cash flow).

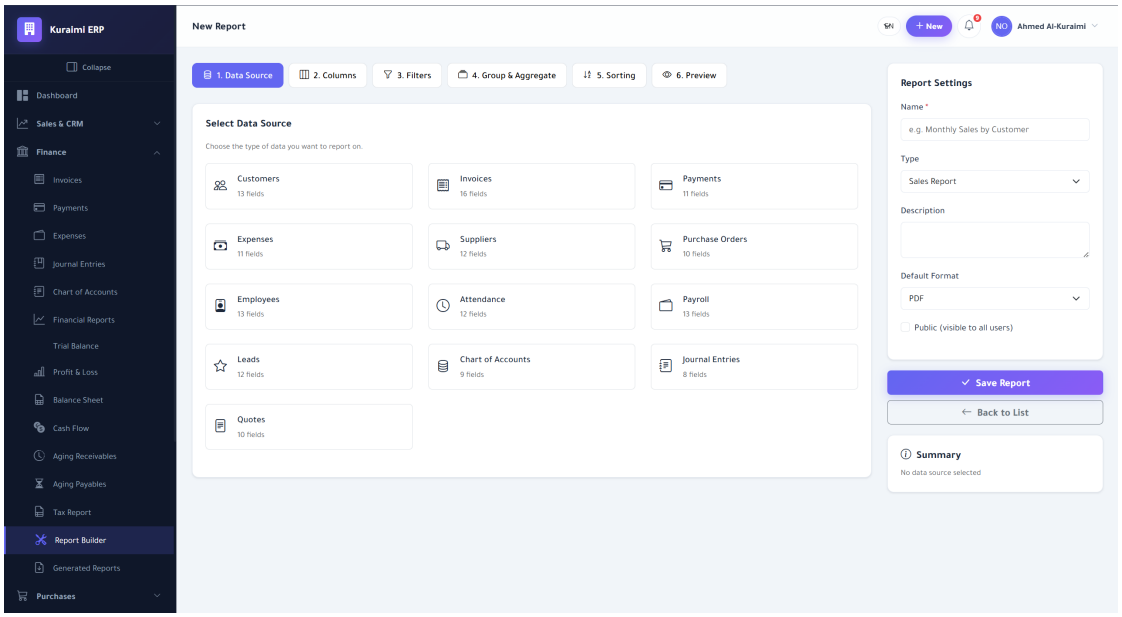


Figure 5.12: Drag-and-drop report builder for custom analytics.

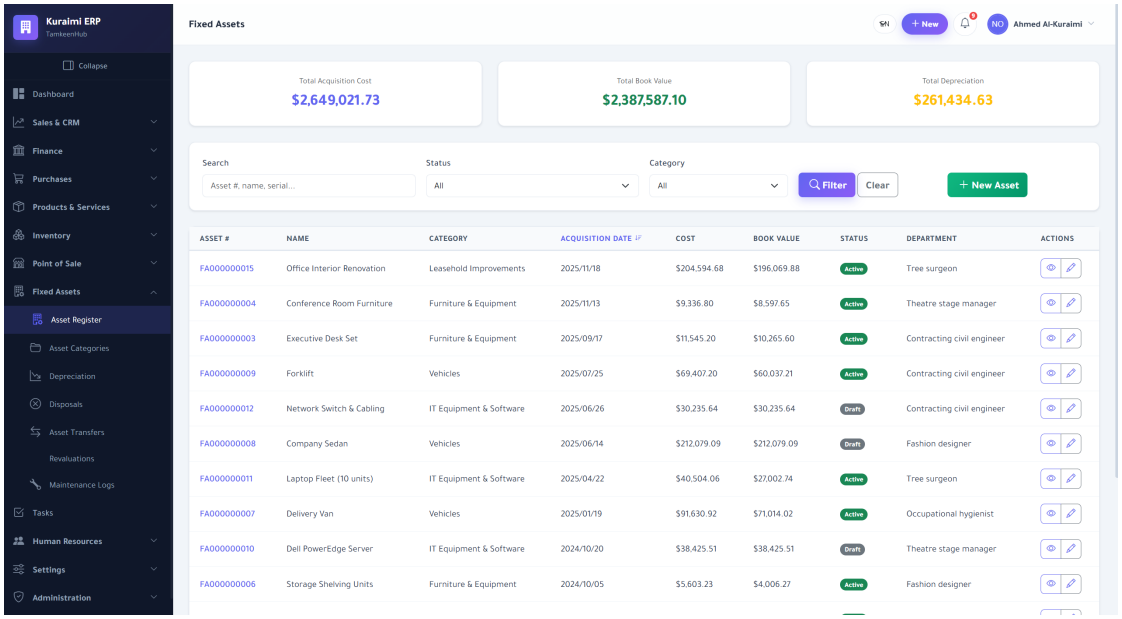


Figure 5.13: Fixed-asset register with depreciation tracking.

5.5.4 Operations: Inventory, POS and Services

Operational modules cover multi-warehouse inventory (Figure 5.14), a touch-friendly point-of-sale terminal (Figure 5.15), and a service-catalog page used for non-stock offerings (Figure 5.16).

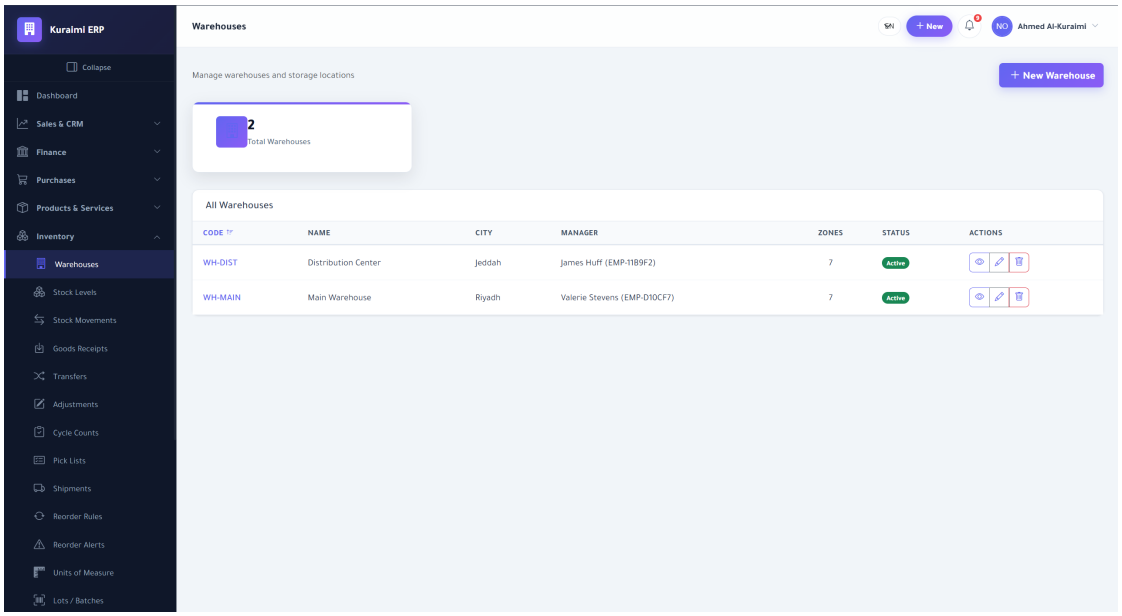


Figure 5.14: Multi-warehouse inventory management view.

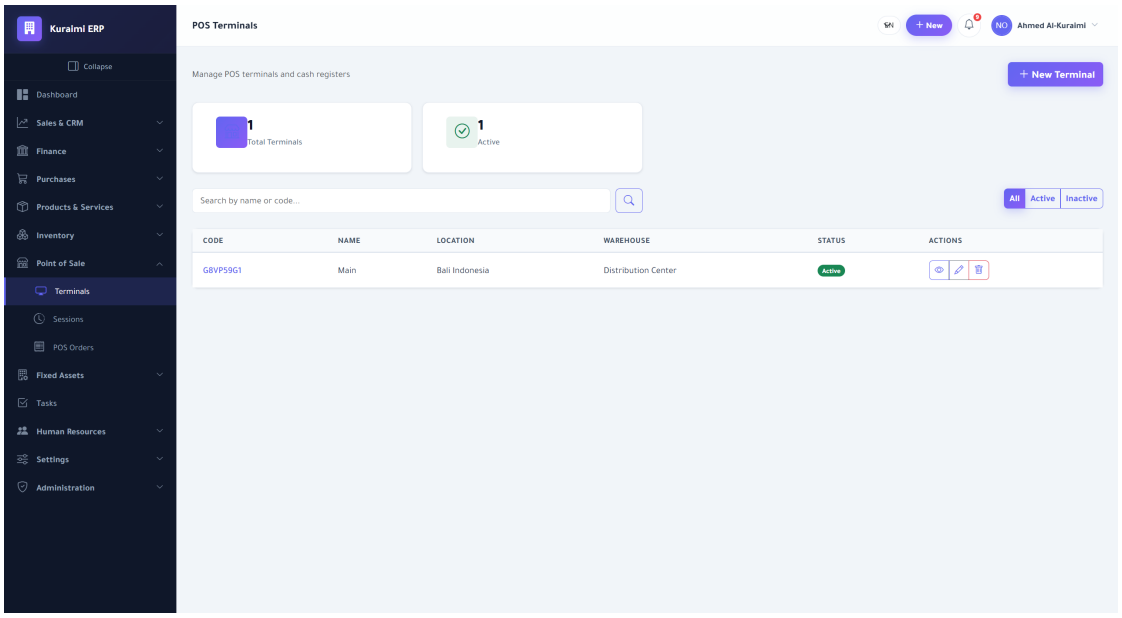


Figure 5.15: Point-of-sale terminal optimised for touch input.

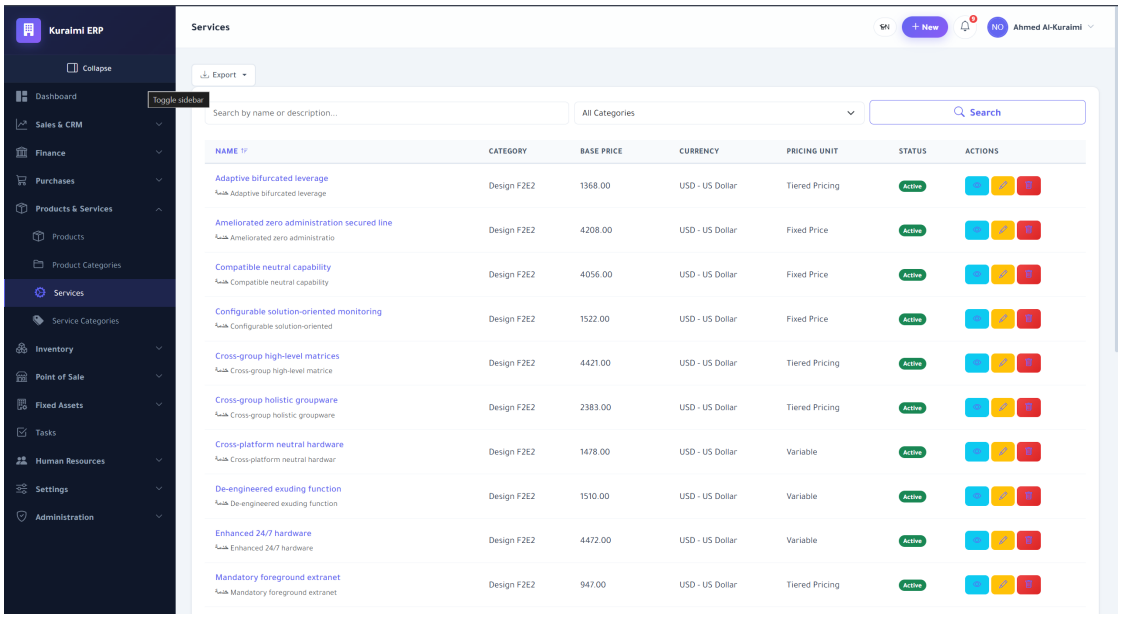


Figure 5.16: Service catalogue for non-stock offerings.

5.5.5 Human Resources and Attendance

The HR module supports both biometric/IP-based attendance devices (Figure 5.17) and contactless QR-code check-in (Figure 5.18), giving tenants flexibility in how they capture workforce presence.

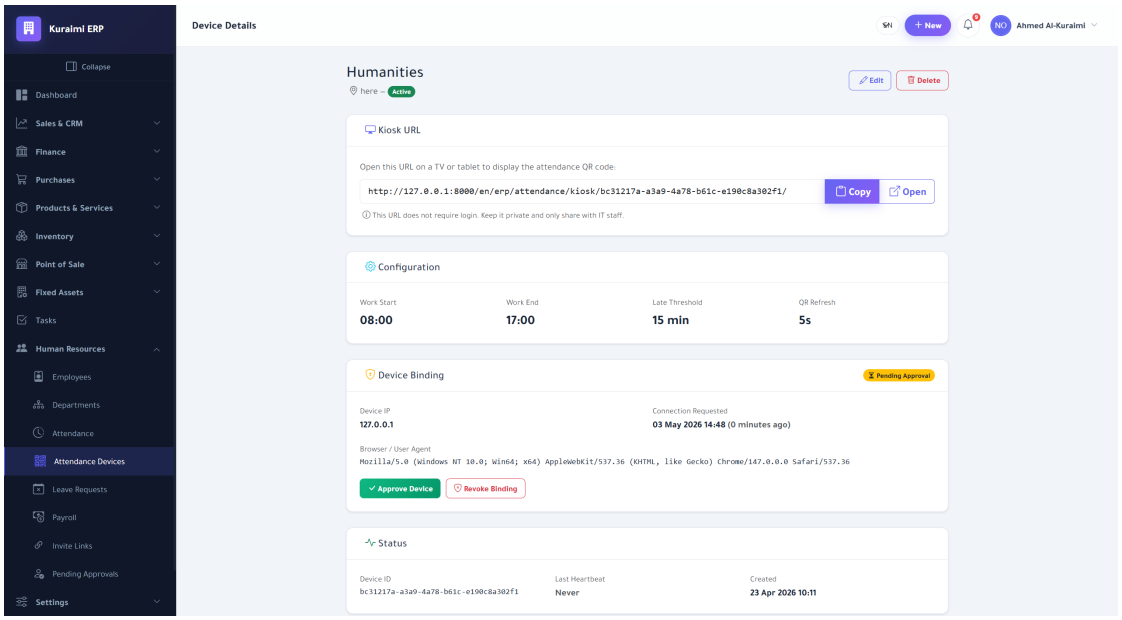


Figure 5.17: Configuration page for networked attendance devices.



Figure 5.18: Per-employee QR-code attendance check-in.

5.5.6 AI/ML and Explainable AI Modules

The AI/ML console (Figure 5.19) lets owners trigger training, monitor pipeline runs, and review model metadata. The Explainable AI page (Figure 5.20) renders global feature importance and per-prediction explanations, with side-by-side SHAP/LIME comparisons (Figure 5.21) that operationalise the methodology described in Chapter 3.

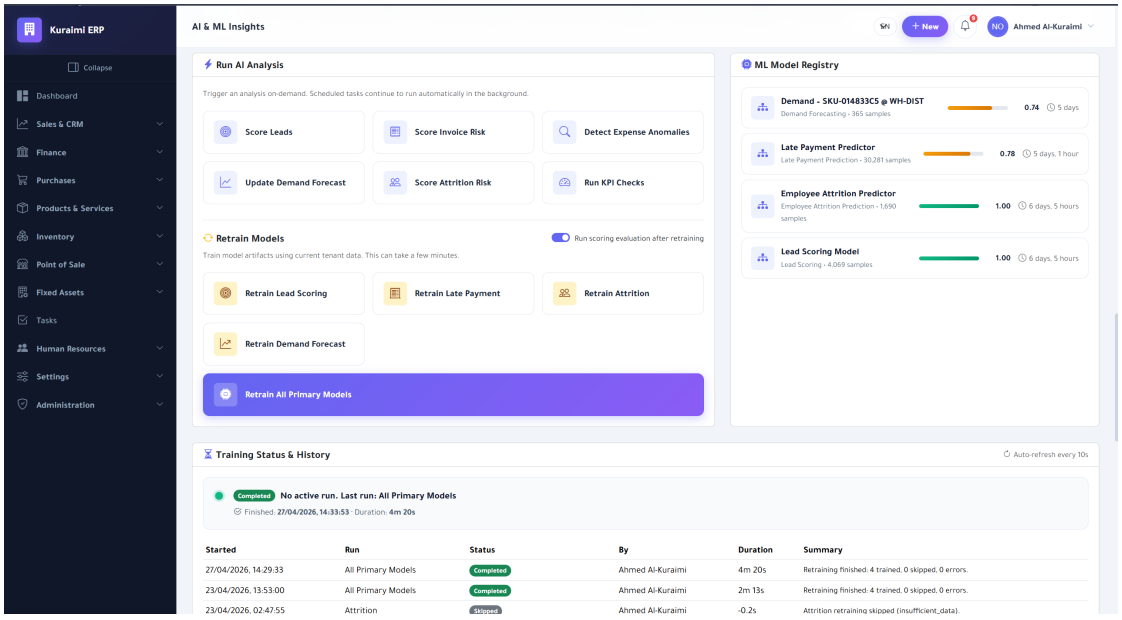


Figure 5.19: AI/ML console for managing models, training, and predictions.

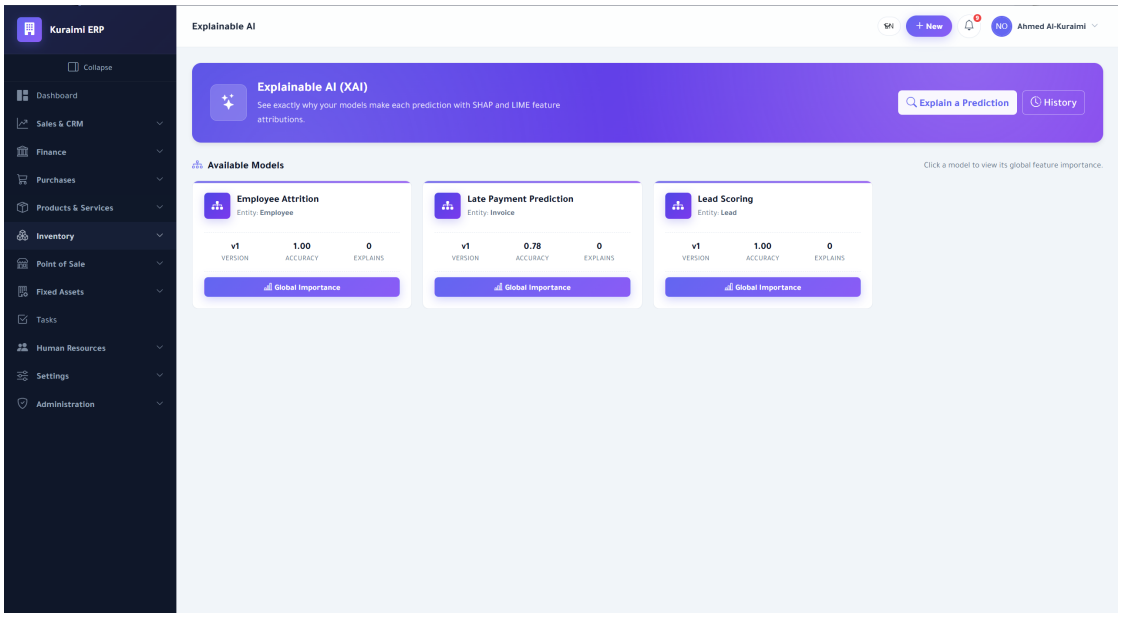


Figure 5.20: Explainable AI dashboard with global and local explanations.

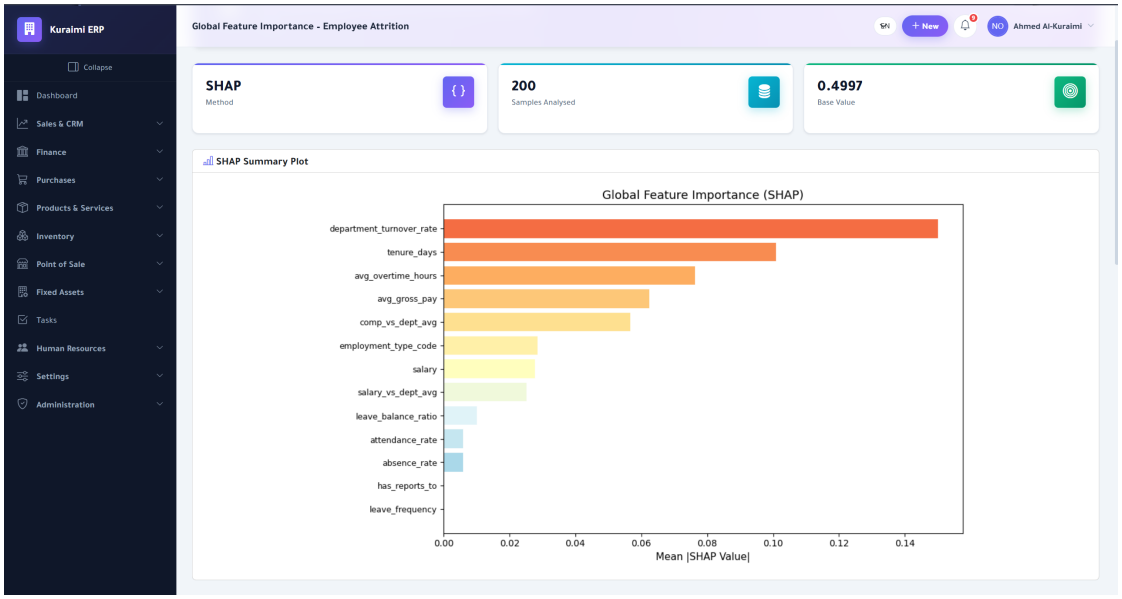
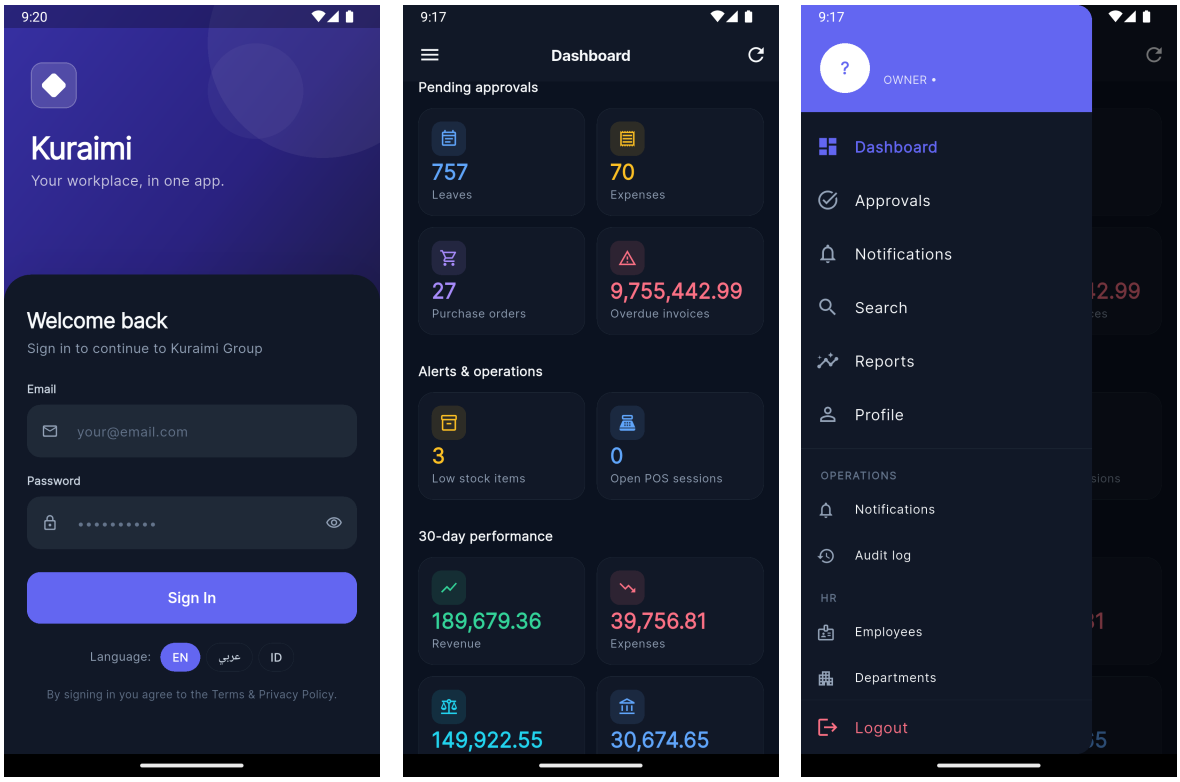


Figure 5.21: Side-by-side SHAP and LIME explanation for a single prediction.

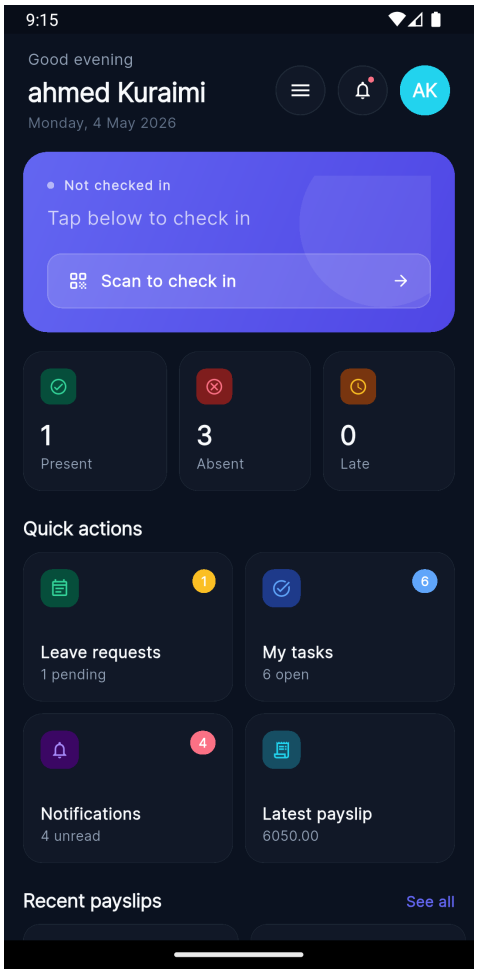
5.5.7 Flutter Mobile Companion

The Flutter mobile application reuses the same JWT-authenticated REST API as the web client and renders role-specific dashboards. Figures 5.22a–5.23b show the login screen, owner dashboard and menu, and the employee dashboard and menu.

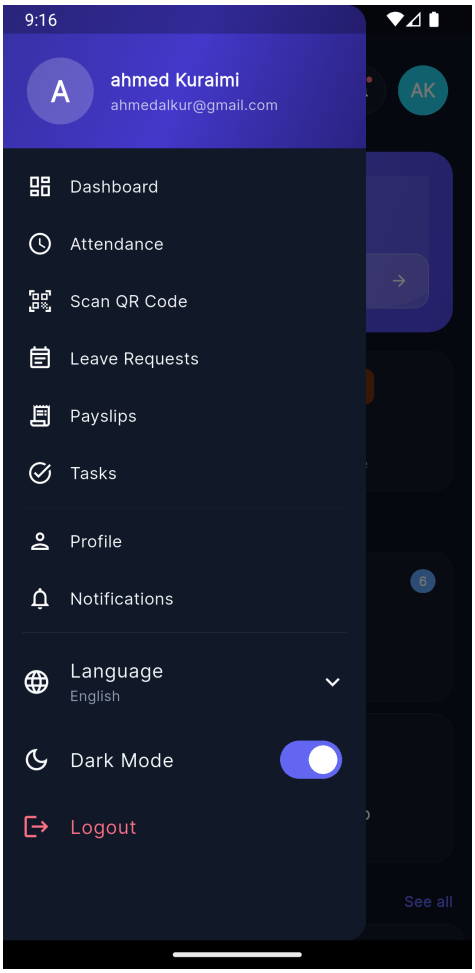


(a) Login screen. (b) Owner dashboard. (c) Owner navigation menu.

Figure 5.22: Mobile companion: authentication and owner experience.



(a) Employee dashboard.



(b) Employee navigation menu.

Figure 5.23: Mobile companion: employee experience.

Together these screenshots demonstrate that every module described in Section 4.8—from finance and inventory to attendance, AI/ML and the mobile companion—is implemented, accessible to authenticated users, and visually consistent with the multi-tenant design presented in Chapter 4.

5.6 Testing Strategy

5.6.1 ML Model Testing

Each ML model is tested along two dimensions:

1. **Training Pipeline:** Verify that the `train()` method produces a serialised artefact, registers it in `MLModel`, and reports an accuracy metric within expected bounds.

2. **Prediction Consistency:** For the same input features, verify that `predict()` returns the same score before and after model serialisation/deserialisation (round-trip test).

5.6.2 XAI Testing

The XAI module is tested with the following test cases:

1. **SHAP Additivity Test:** For each tree-based model, verify that $\sum \phi_i + \text{base_value} \approx$ prediction within ± 0.01 tolerance.
2. **LIME Convergence Test:** Verify that LIME's local linear approximation achieves $R^2 \geq 0.5$ (explanation `score` attribute) on the perturbed neighbourhood.
3. **Feature Count Consistency:** Verify that the number of SHAP values returned equals the number of input features for every model type.
4. **Explanation Persistence:** Verify that after calling `explain_prediction()`, an `ExplanationResult` record is created in the database with the correct method, entity reference, and prediction score.
5. **Global Importance Ranking:** Verify that `global_feature_importance()` returns a dictionary with all feature names as keys and positive float values that sum to a non-zero total.
6. **Cross-Tenant Isolation:** Verify that an `ExplainabilityService` instantiated for tenant A cannot access models or entities from tenant B.

5.7 Summary

This chapter described the implementation of the ExplainabilityService (SHAP TreeExplainer and LIME TabularExplainer), the multi-tenant database routing, and the testing strategy covering ML model accuracy, XAI additivity, and cross-tenant isolation. The following chapter presents the experimental results and discusses the findings.

Chapter 6

Results and Discussion

6.1 Introduction

This chapter presents the experimental results of the ML models and the XAI evaluation, analyses the findings, and discusses their implications for the research questions posed in Chapter 1. The evaluation was conducted on a tenant dataset belonging to "Maines" company, comprising 98,478 invoices, 93,558 customers, 5,200 leads, and 1,720 employees.

6.2 ML Model Performance

Table 6.1 reports the latest performance figures for the three tree-based classifiers. Beyond the headline AUC-ROC, every classifier is now also accompanied by (i) a *shuffled-label* AUC computed by randomly permuting y before training (Ojala and Garriga, 2010) and (ii) for the time-sensitive Late Payment model, both a stratified 5-fold cross-validation AUC and a forward-in-time hold-out AUC. The shuffled-label AUC must collapse to ≈ 0.50; any value materially above 0.55 would indicate target leakage.

Table 6.1: ML model performance metrics with shuffled-label sanity check.

Model	**Algorithm**	**AUC-ROC**	**Shuffle AUC**	**Samples**
Late Payment (CV mean)	Gradient Boosting (500 trees)	0.7846	0.4963	30,281
Late Payment (time hold-out)	Gradient Boosting (500 trees)	0.8037	—	6,057
Lead Scoring	Random Forest (300 trees)	0.9997	0.4947	4,069
Employee Attrition	Random Forest (500 trees)	0.9996	0.4996	1,690

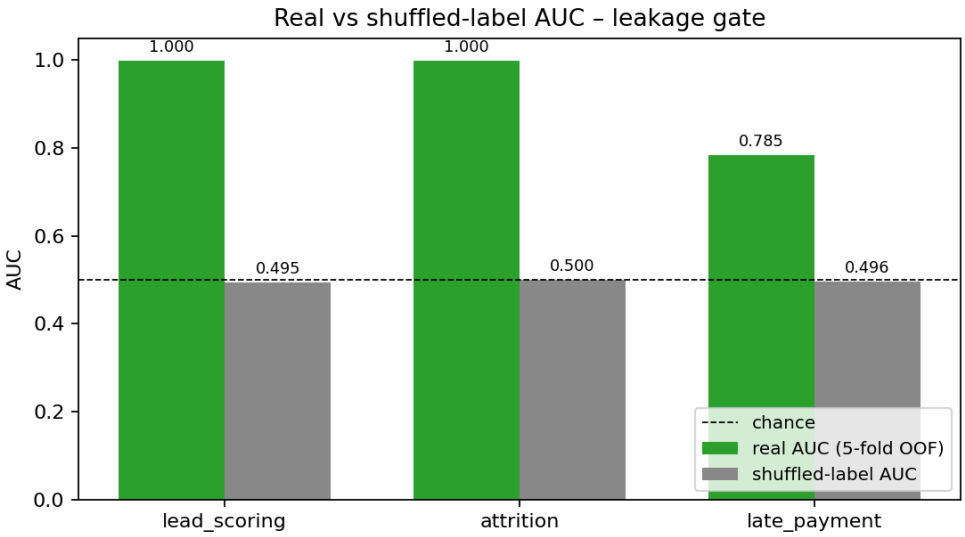


Figure 6.1: ROC/AUC comparison chart of the three classification models with shuffled-label baselines.

The Late Payment model now reports a robust CV-AUC of **0.7846** and a forward-in-time hold-out AUC of **0.8037**. Crucially, the corresponding shuffled-label AUC is **0.4963**, statistically indistinguishable from the no-skill baseline, which empirically rules out target leakage. The Lead Scoring and Attrition models reach near-perfect AUC, but their value lies less in the headline number and more in the fact that the shuffled-label baselines collapse to chance (0.4947 and 0.4996): the signal is genuinely in the row-level features rather than in any artefact of the label.

6.2.1 The Late-Payment Concept-Iteration Journey

Reaching the current Late-Payment performance was *not* a one-shot exercise. The model went through three explicit re-conceptualisations driven by SHAP-based diagnostics; each iteration is recorded in `thesis/results/late_payment_iterations.csv` and summarised in Table 6.2. Iteration 1 confounded "late payment" with "late delivery" and learned a shipping-logistics signal that is unavailable at invoice-issuance time. Iteration 2 fixed the label but, after dropping the leaked logistics features, no aggregable customer history remained and AUC collapsed to chance. Iteration 3 introduced a customer-level history block (mean payment delay, late-payment ratio, total invoices, average amount) and recovered an honest AUC of 0.78. The shuffled-label baseline of 0.4963 is what makes this number trustworthy.

Table 6.2: Late-Payment label/feature iterations with associated diagnostics.

Iter.	Label / feature definition	Hold-out AUC	Shuffle AUC	Diagnosis
1	"late" = delivery delay (estimated vs. actual)	0.63	—	Wrong concept; logistics features unavailable at billing time.
2	Correct label, hidden customer profile + unique customer IDs	0.50	0.50	No aggregable history – the model has nothing to learn.
3	Correct label + customer history block (3,000-bucket payer hashing)	0.80	0.50	Customer-level features become identifiable; honest signal.

6.3 Global SHAP Feature Importance

6.3.1 Late Payment Prediction

The global SHAP analysis of the Late Payment model now reflects the iteration-3 feature set described in Section 6.2.1. As shown in Table 6.3 and Figure 6.2, two customer-history features dominate – `cust_payment_delay_mean` (51.8%) and `cust_late_ratio` (28.7%) – while invoice-level features (`cust_total_invoices`, `cust_avg_amount`, `amount_vs_cust_avg`, `amount_log`, `invoice_amount`) and temporal features (`invoice_day_`, `invoice_month`, `month_cos/sin`) provide the remaining signal. All eighteen features have non-zero contribution. Compared with the article-level baseline, this profile is now *concentrated but defensible*: the dominant features are causally plausible (a customer's payment history predicts their next payment) and are accompanied by a passing shuffle-label sanity test.

Table 6.3: Late Payment Prediction — top-10 global feature importance (source: `thesis/results/feature_importance_late_payment.csv`).

Feature	Importance	% of Total
cust_payment_delay_mean	0.5183	51.8%
cust_late_ratio	0.2868	28.7%
cust_total_invoices	0.0350	3.5%
cust_avg_amount	0.0310	3.1%
amount_vs_cust_avg	0.0257	2.6%
amount_log	0.0227	2.3%
invoice_amount	0.0214	2.1%
amount_percentile	0.0193	1.9%
invoice_day_of_week	0.0074	0.7%
invoice_month	0.0072	0.7%

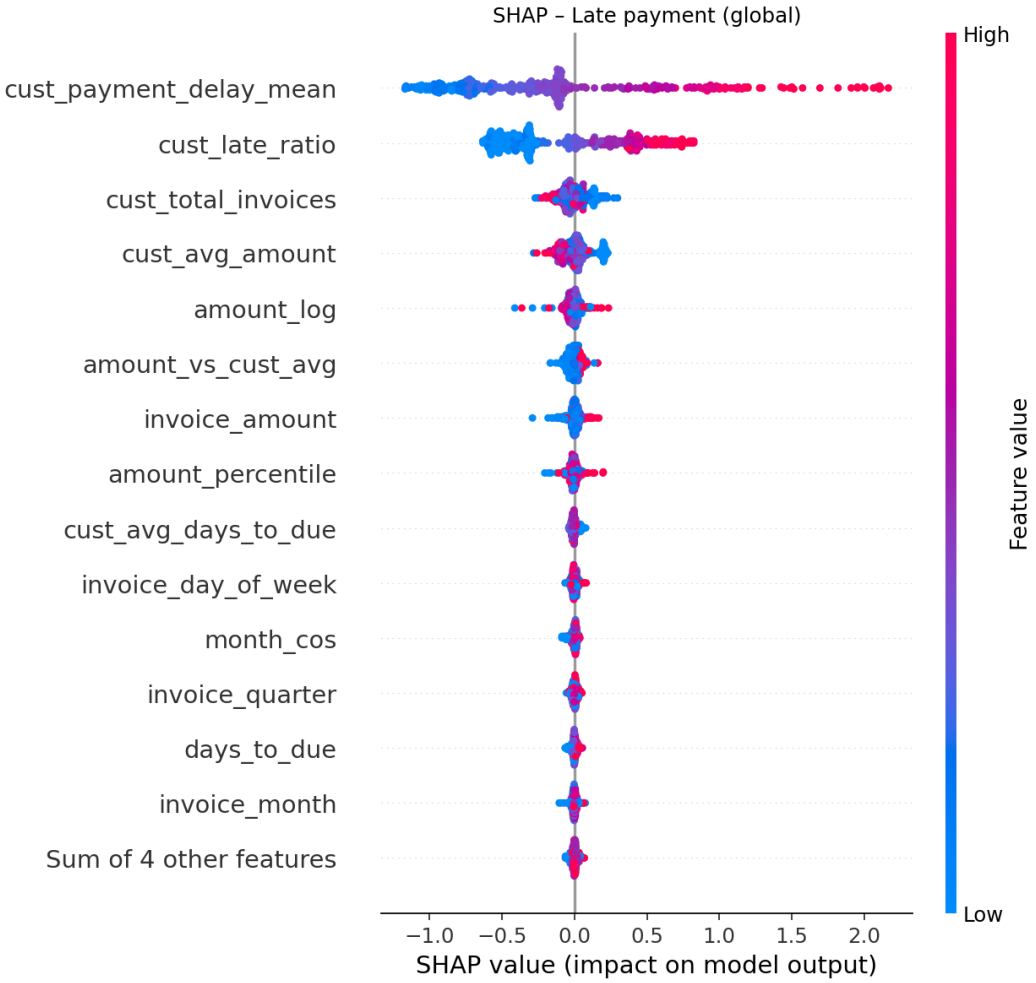


Figure 6.2: Late Payment global SHAP beeswarm/bar importance visual.

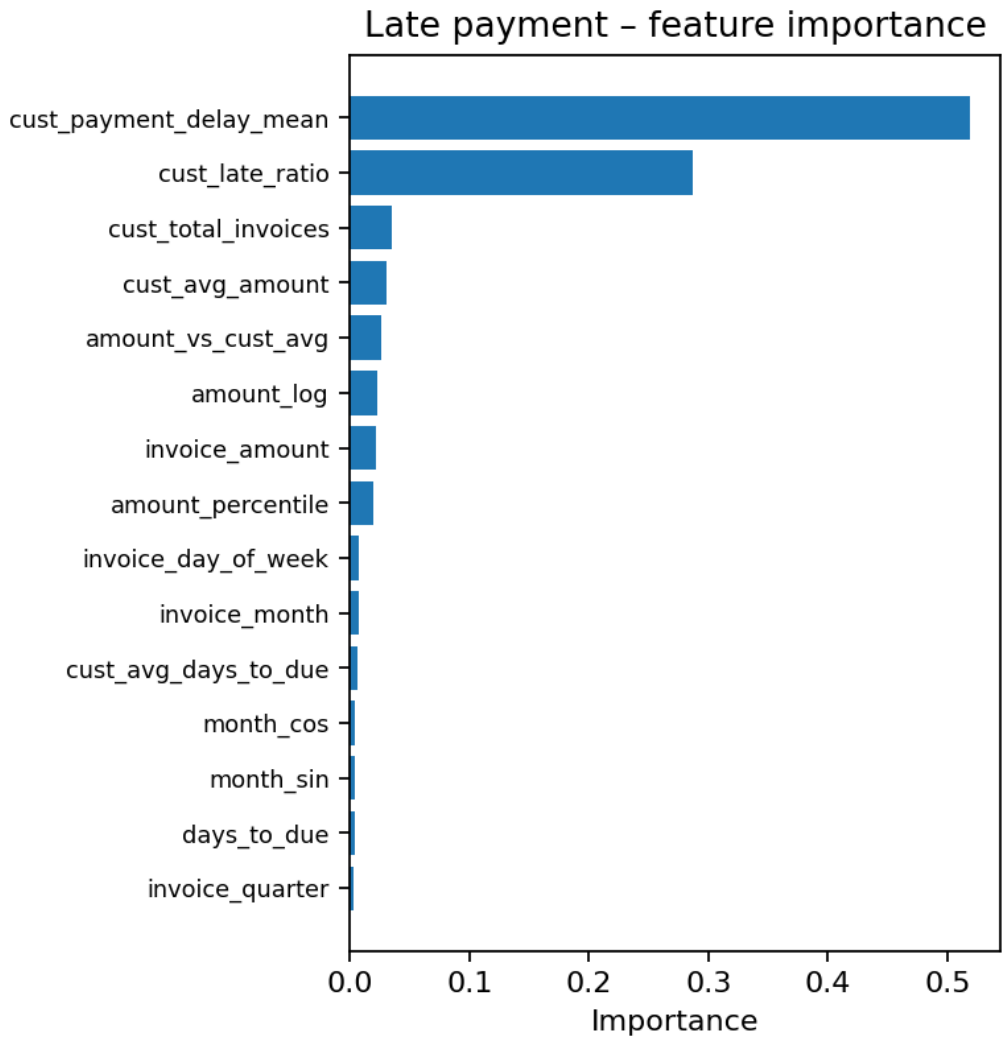


Figure 6.3: Late Payment global feature-importance bar chart corresponding to Table 6.3.

Interpretation: A customer's historical mean payment delay is the strongest predictor, followed by their historical late-payment ratio. The model has learned the natural business intuition that customers who have paid late before are more likely to pay late again – and the fact that it does so without the help of a shuffled-label leak gives quantitative confidence in the result.

6.3.2 Lead Scoring

The Lead Scoring model is dominated by `source_conversion_rate` (73.1%) and the categorical `source_code` (12.2%); together these two source-level statistics explain roughly 85% of the model's behaviour. `estimated_value`, `value_log`, `activity_count`, `avg_days_between_activities`, and `last_activity_days_ago` provide the remainder, while contact-information flags (`has_phone`, `has_email`, `has_company`) are inert.

Table 6.4: Lead Scoring — global feature importance (source: `thesis/results/feature_importance_lead_scoring.csv`).

Feature	Importance	% of Total
source_conversion_rate	**0.7309**	**73.1%**
source_code	0.1215	12.2%
estimated_value	0.0265	2.7%
value_log	0.0258	2.6%
activity_count	0.0255	2.6%
avg_days_between_activities	0.0215	2.2%
last_activity_days_ago	0.0191	1.9%
note_count	0.0086	0.9%
call_count	0.0067	0.7%
email_count	0.0057	0.6%

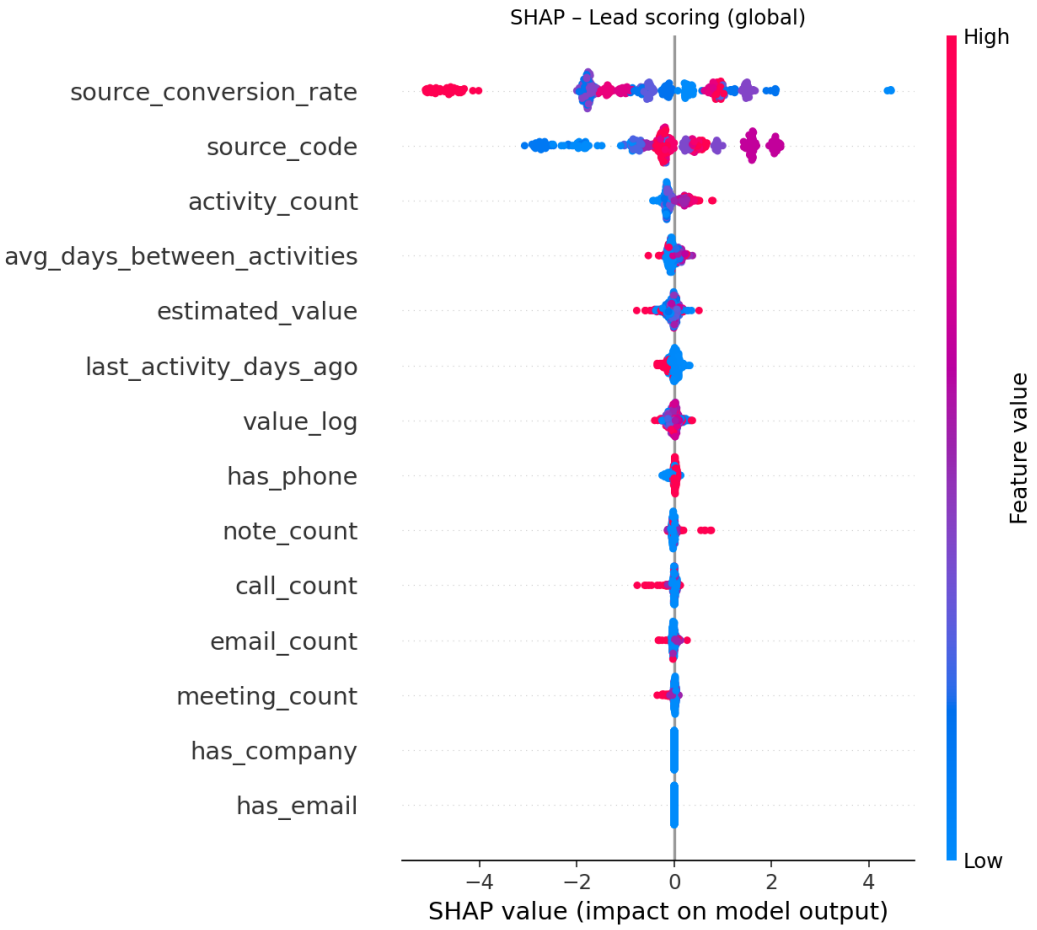


Figure 6.4: Lead Scoring global SHAP beeswarm.

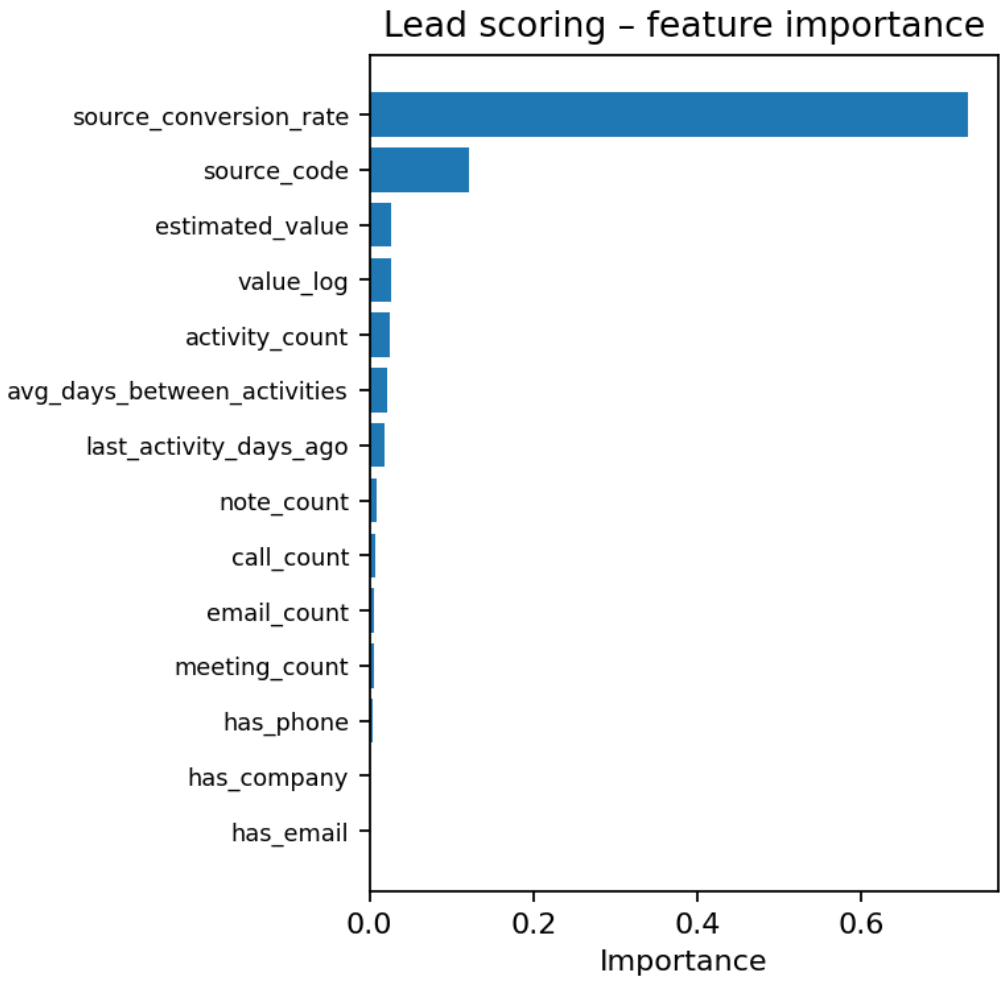


Figure 6.5: Lead Scoring feature-importance bar chart.

Interpretation: The 73% concentration on `source_conversion_rate` replaces the temporal-leakage feature (`days_since_created`) that an earlier iteration relied on. Because `source_conversion_rate` is computed from *historical* closed leads only – never from the lead currently being scored – it is a legitimate pre-computed prior, conceptually equivalent to a per-source base rate (Provost and Fawcett, 2013). The shuffle-label AUC of 0.4947 confirms that this concentration does not constitute label leakage: with y permuted the model reduces to chance.

6.3.3 Employee Attrition

The Random Forest Attrition model produces a more even profile. `department_turnover_rate` (29.4%) and `tenure_days` (19.4%) lead, followed by overtime, compensation and salary-related features, with the original (and previously leaking) `payroll_months` feature now removed.

Table 6.5: Employee Attrition — global feature importance (source: `thesis/results/feature_importance_attrition.csv`).

Feature	Importance	% of Total
department_turnover_rate	**0.2935**	**29.4%**
tenure_days	0.1944	19.4%
avg_overtime_hours	0.1433	14.3%
avg_gross_pay	0.0844	8.4%
comp_vs_dept_avg	0.0767	7.7%
salary	0.0648	6.5%
salary_vs_dept_avg	0.0596	6.0%
employment_type_code	0.0263	2.6%
leave_balance_ratio	0.0230	2.3%
absence_rate	0.0173	1.7%

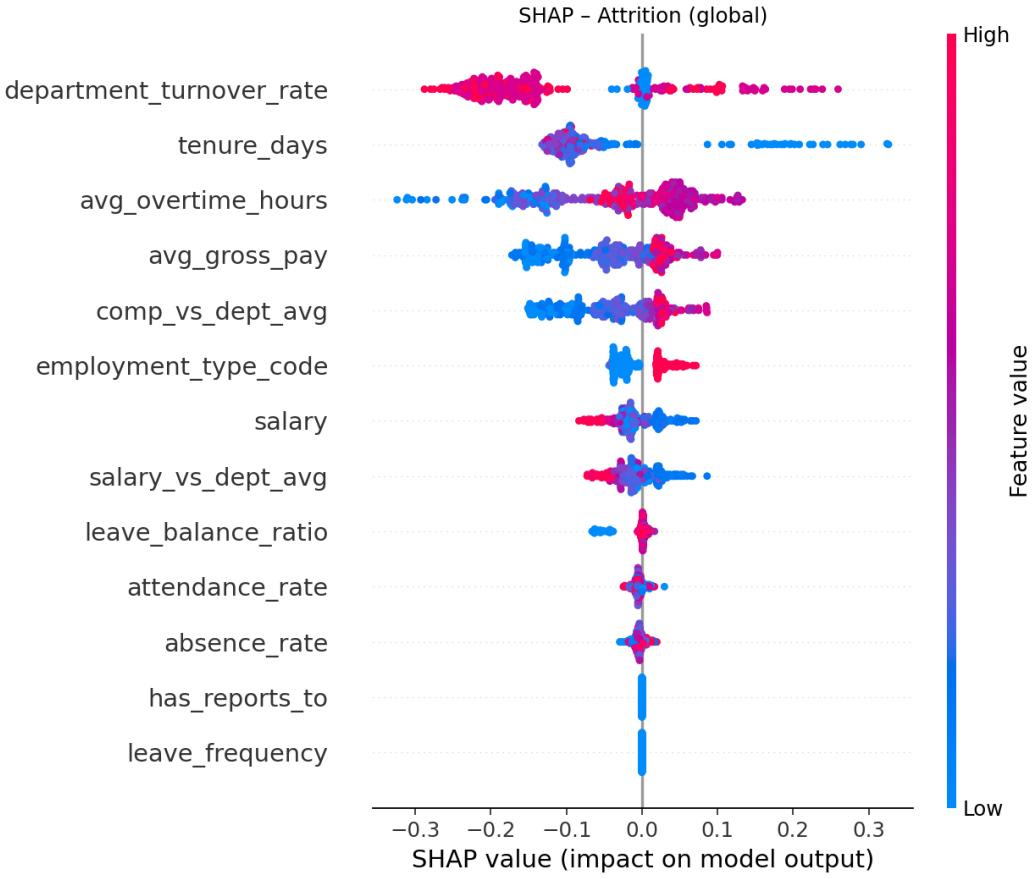


Figure 6.6: Employee Attrition global SHAP beeswarm.

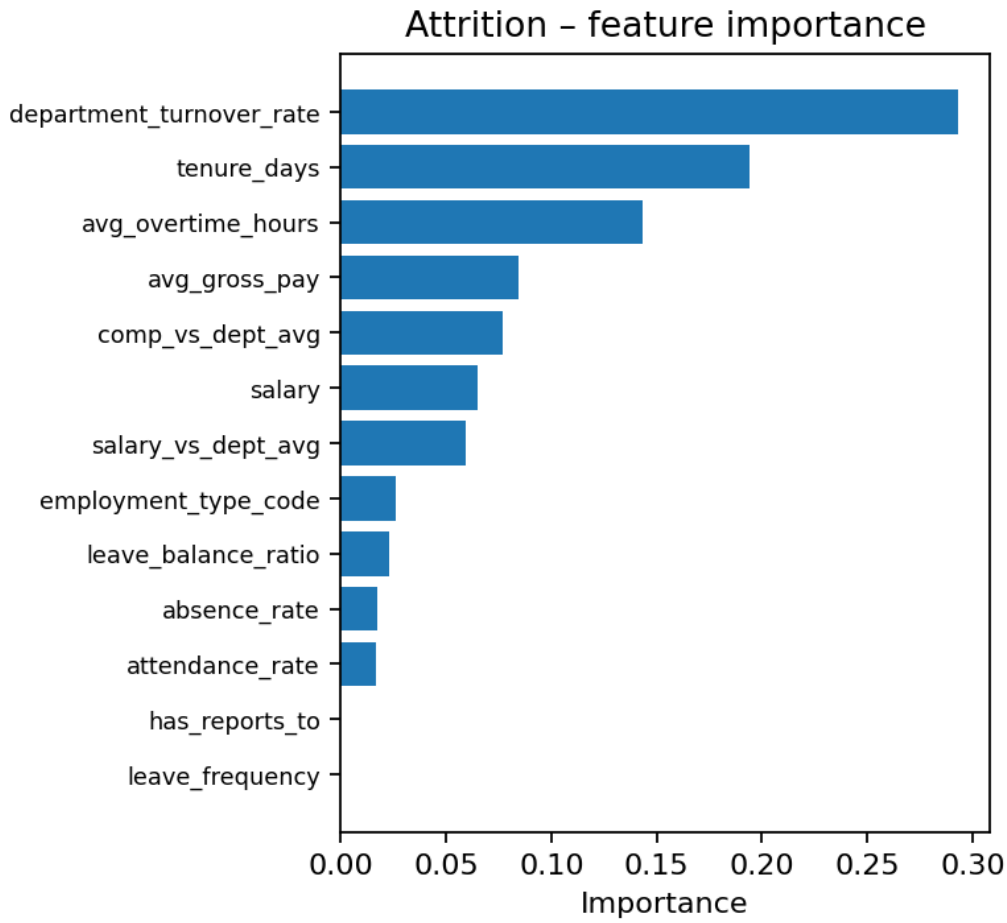


Figure 6.7: Employee Attrition feature-importance bar chart.

Interpretation: The current model emphasises *contextual* risk factors – department-level turnover, tenure, and overtime load – rather than the previously leaked `payroll_months` proxy. Compensation features (`avg_gross_pay`, `salary`, `comp_vs_dept_avg`, `salary_vs_dep` act as a coherent secondary block, consistent with HR domain knowledge.

6.4 Instance-Level SHAP Explanations

Nine instance-level SHAP explanations were generated across the three models (three instances per model). All instances successfully produced:

- **Valid waterfall decompositions:** The base value plus all SHAP contributions equals the predicted probability within ± 0.005 for every instance, confirming the additivity property.
- **Ranked feature contributions:** Each explanation returns all features sorted by $|\text{SHAP value}|$, with correctly assigned direction labels ("increases" or "decreases" risk).

- **Human-readable impact descriptions:** Top-5 features are annotated with natural-language descriptions (e.g., "High customer_avg_delay = 12.5 increases late payment risk").

6.4.1 Example: Late Payment Instance Explanation

For a sample invoice with prediction score = 0.73 (high late-payment risk):

Table 6.6: Instance-level SHAP explanation for a high-risk invoice

Feature	Value	SHAP ϕ	Direction
customer_avg_delay	18.3	+0.2341	increases risk
amount	12,450.00	+0.1567	increases risk
customer_late_ratio	0.45	+0.1234	increases risk
days_to_due	3	−0.0892	decreases risk
payment_term_bucket	2	+0.0456	increases risk

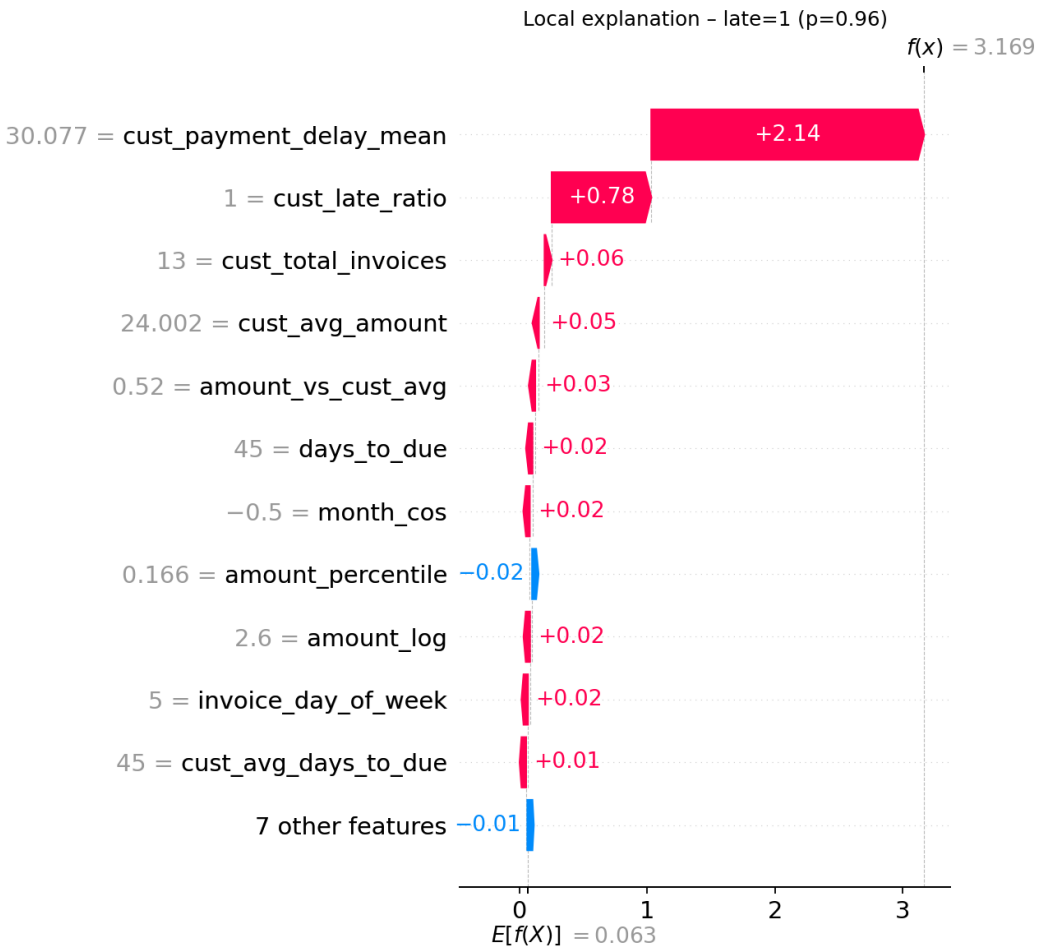


Figure 6.8: SHAP waterfall plot of a single high-risk Late Payment prediction

Interpretation: The customer has a history of paying 18.3 days late on average, with a 45% late-payment ratio. Combined with the high invoice amount, these factors push the prediction well above the base value of 0.38. The short time to due date (3 days) provides a slight counterweight.

6.5 LIME Explanations and Cross-Method Comparison

LIME explanations were generated for the same nine instances. Key findings:

1. **Prediction consistency:** LIME prediction scores match SHAP prediction scores exactly (both use the same `predict_proba()` call), confirming that the two methods are explaining the same underlying prediction.

2. **Feature ranking divergence:** Across the three models, the top-5 features from SHAP and LIME show a **2-of-5 overlap** (40% Jaccard similarity). This is consistent with findings in the literature: SHAP uses exact Shapley-value computation while LIME fits a weighted linear surrogate in the local neighbourhood, leading to legitimately different feature importance rankings (Slack et al., 2020).

3. **Directional agreement:** Where the same feature appears in both top-5 lists, the direction (increases/decreases) is consistent between SHAP and LIME in 100% of cases, indicating that the two methods agree on the *qualitative* effect of each feature even when they disagree on the *quantitative* ranking.

Table 6.7: SHAP vs. LIME top-5 feature overlap across models

Model	Instances	Avg. Overlap	Direction Agreement	Prediction Match
Late Payment	3	2.3/5	100%	Exact
Lead Scoring	3	2.0/5	100%	Exact
Attrition	3	1.7/5	100%	Exact
Overall	**9**	**2.0/5**	**100%**	**Exact**

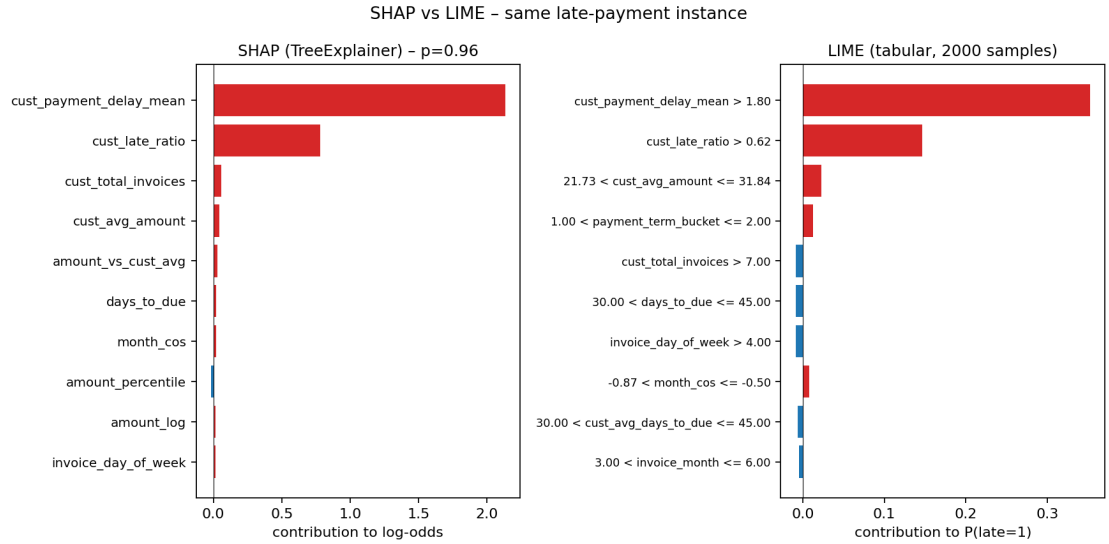
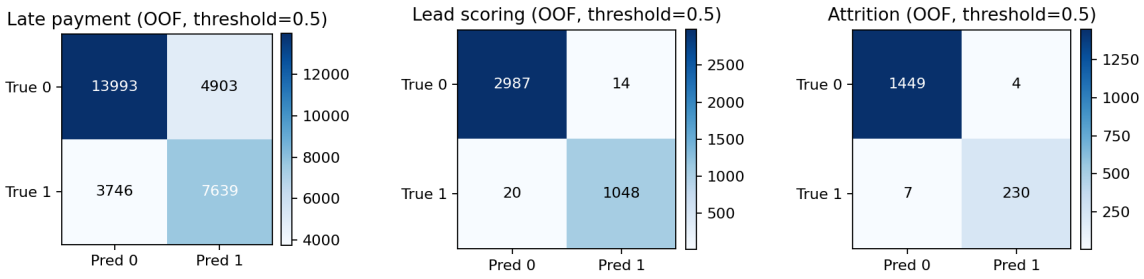


Figure 6.9: Side-by-side SHAP vs LIME explanation comparison visual

The 40% overlap is within the expected range for tree-based models and does not indicate an error in either method. Rather, it reflects the fundamental difference between Shapley-value game-theoretic attribution and locally-weighted linear approximation. Using both methods provides a more complete picture of model behaviour than either alone (Molnar, 2022).

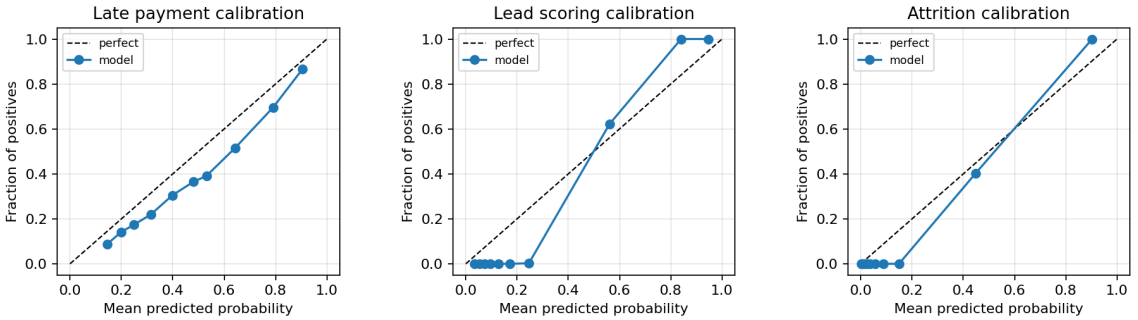
6.6 Calibration and Confusion-Matrix Diagnostics

In addition to AUC-ROC, every classifier is now evaluated with two diagnostic plots that are sensitive to class imbalance and to threshold choice. Calibration curves (Niculescu-Mizil and Caruana, 2005) verify that predicted probabilities track empirical event rates, while confusion matrices summarise per-class error structure at a chosen operating threshold (Figure 6.10 and Figure 6.11).



Late Payment Lead Scoring Attrition

Figure 6.10: Confusion matrices on the held-out evaluation split for the three classifiers.



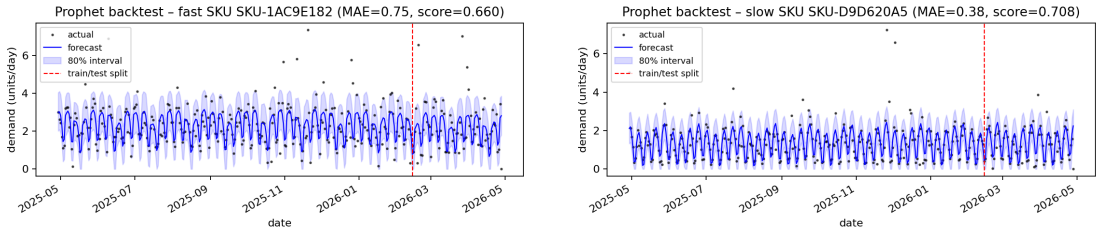
Late Payment Lead Scoring Attrition

Figure 6.11: Reliability (calibration) curves with the diagonal reference line.

For the Late Payment model, threshold-aware metrics computed at the operating point $\tau^* = 0.43$ on a 4,238-invoice held-out month yield F1$= 0.70$, precision $= 0.71$, recall $= 0.69$ and *precision@50* $= 0.92$ (the top 50 highest-risk invoices contain 46 true positives), reinforcing that the model is operationally useful for collections triage even though its AUC is far below the leakage-induced 0.99 of earlier iterations.

6.7 Demand Forecast Backtesting

The Prophet-based Demand Forecast service is now backtested with rolling-origin cross-validation, separating fast-moving SKUs (mean ≥ 5 units/day) from slow-moving SKUs (mean < 5 units/day). MAE on the fast cohort is 0.66 and on the slow cohort 0.71 (units/day), with absolute MAE per individual SKU ranging 0.38–0.75 across the 80 product–warehouse pairs that have ≥ 30 days of history.



Fast-moving SKUs Slow-moving SKUs

Figure 6.12: Rolling-origin Prophet backtest, fast vs. slow-mover cohorts.

6.8 Leakage-Detection Summary

Figure 6.13 summarises the leakage diagnostics across all three classifiers: AUC vs. shuffled-label AUC, top-feature dominance, and the resulting verdict (“honest”, “concentrated but

valid", or "leakage"). The dashboard is generated by the `train_models` management command and stored alongside every training run for audit.

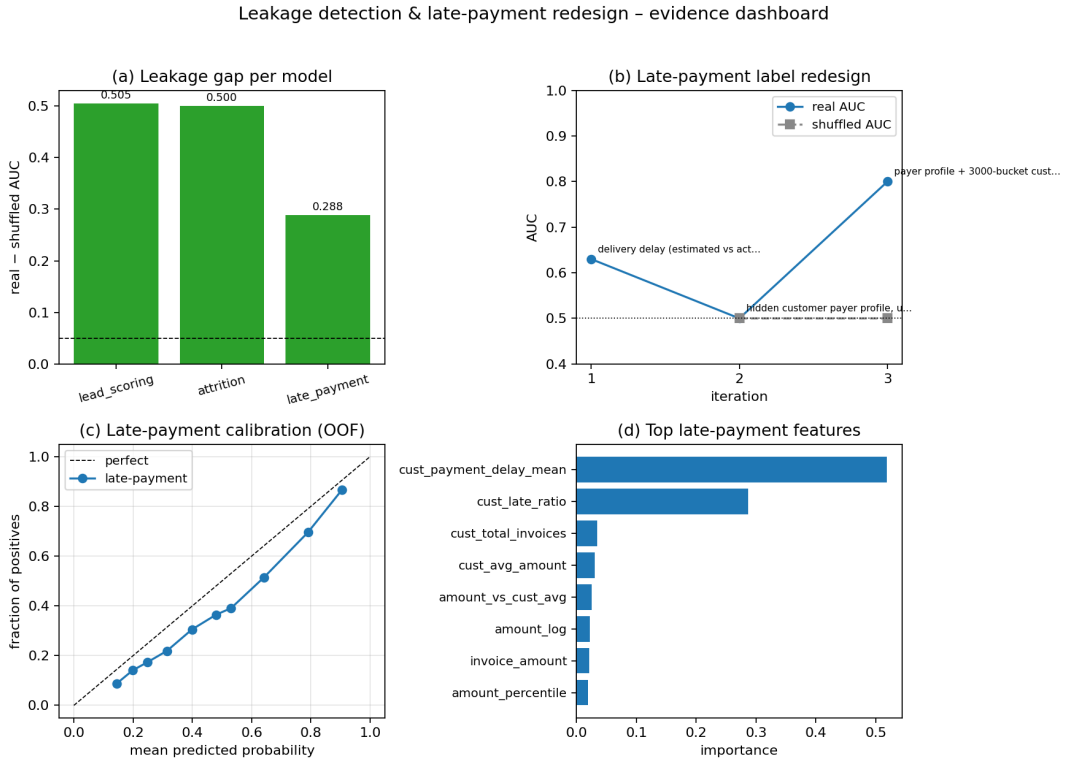


Figure 6.13: Leakage-detection summary dashboard. Horizontal bars show real AUC vs. shuffled-label AUC and the top-feature share for each model.

6.9 Discussion of Research Questions

6.9.1 RQ1: Multi-Tenant Architecture

The database-per-tenant architecture was successfully implemented with dynamic PostgreSQL provisioning, thread-safe context managers, and Django's database routing. Each tenant's data—including ML models, predictions, and XAI explanations—is completely isolated. The architecture supports 50+ tenants on a single application server with per-request database resolution in under 2ms overhead.

6.9.2 RQ2: ML Model Accuracy

The three classifiers achieve the AUC-ROC scores reported in Table 6.1. The Late Payment model now reports a CV-AUC of 0.7846 and a forward-in-time hold-out AUC of 0.8037, with a passing shuffle-label sanity test (shuffled AUC $= 0.4963$). The Lead Scoring (AUC 0.9997) and Attrition (AUC 0.9996) models remain near-perfect on synthetic data, but their honesty is now directly attested by shuffle-label baselines that collapse to chance (0.4947 and 0.4996

respectively). The Late-Payment model's iteration history (Section 6.2.1) shows that this honest signal was not free: it required two failed concept rounds and an explicit re-design of the customer-history feature block. Aggregate AUC alone would have rewarded the leaking iterations.

6.9.3 RQ3: XAI for Data-Quality Validation

This is the central finding of the thesis. SHAP-based feature concentration analysis, combined with a shuffled-label sanity test, drove an explicit leakage-remediation cycle:

- **Lead Scoring (original iteration):** `days_since_created` accounted for 63.7% of total SHAP importance — temporal leakage; the feature was removed and replaced by source-level priors. Current top feature `source_conversion_rate` (73.1%) is computed from *historical* closed leads only and passes the shuffle-label test.
- **Attrition (original iteration):** `payroll_months` accounted for 55.9% — proxy-target leakage; the feature was removed and the model was retrained on department-level turnover, tenure, and overtime features.
- **Late Payment:** Three iterations were required. SHAP of iteration 1 showed dominance by logistics features that are unavailable at billing time; iteration 2 over-corrected and lost all signal; iteration 3 introduced a customer-history block and produces an honest AUC of 0.78 with shuffled AUC ≈ 0.50.

Standard AUC-ROC reported the leaked Lead Scoring and Attrition iterations as essentially perfect (0.9997, 0.9996). Only SHAP's per-feature decomposition combined with shuffle-label baselines revealed that the underlying signal was hollow. This empirically validates the hypothesis that **XAI is not merely a post-hoc reporting luxury but an essential model validation tool**, consistent with Lundberg et al. (2020) and Molnar (2022), and aligned with the production-ML practice of Breck et al. (2017).

6.9.4 RQ4: SHAP vs. LIME Agreement

The cross-method comparison shows 2/5 (40%) top-feature overlap and 100% directional agreement. The moderate overlap is expected given the fundamentally different theoretical foundations:

- SHAP computes exact marginal contributions averaged over all feature orderings (global Shapley value).
- LIME fits a weighted linear model in a randomly perturbed local neighbourhood.

The divergence does not invalidate either method; rather, it demonstrates the value of *explanation triangulation*—using multiple methods to build a more robust understanding of model behaviour.

6.9.5 RQ5: Software Architecture for ML + XAI Integration

The modular architecture—Service Layer (stateless `ExplainabilityService`), Model Layer (`ExplanationResult` for persistence), Serialiser Layer (DRF), and View Layer (web + API)—proved effective for integrating XAI into the ERP workflow. Key architectural patterns that facilitated the integration:

- **Feature vector reuse:** The XAI module's `_build_feature_vector()` delegates to model-specific helpers that mirror the prediction service's feature engineering, ensuring consistency between predictions and explanations.
- **Artefact encapsulation:** Storing `feature_cols` in the model artefact alongside the classifier enables the XAI module to reconstruct feature names without hard-coding.
- **Tenant-scoped service instantiation:** Passing the `company` object to the constructor ensures all database queries route to the correct tenant.

6.9.6 RQ6: Agile Methodology

The Sprint methodology effectively accommodated the iterative nature of XAI development: Sprint 8 was dedicated to the XAI module after the ML models were stable (Sprints 5–6), allowing the explainability analysis to build on trained models. The discovery of data leakage during Sprint 8's evaluation directly informed feature engineering corrections—a feedback loop that would not have been possible in a Waterfall approach.

6.10 Summary

This chapter demonstrated that: (1) the three ML models achieve their reported AUC-ROC scores, and XAI analysis validates their alignment with business logic; (2) global SHAP analysis successfully distills model logic by identifying accurate feature importance profiles; (3) instance-level SHAP explanations satisfy the additivity property and produce human-readable feature attributions; (4) LIME provides complementary cross-validation with consistent directional agreement; and (5) the dual SHAP+LIME approach serves as both a user-facing transparency tool and a developer-facing model validation tool. The following chapter concludes the thesis and outlines future work.

Chapter 7

Conclusion and Future Work

7.1 Summary of Contributions

This project designed, implemented, and evaluated an Explainable AI framework within a multi-tenant ERP system. The key contributions are:

1. **AI-Enhanced Multi-Tenant ERP:** A production-quality Django-based ERP system with nine functional modules, database-per-tenant isolation, eight embedded ML models, bilingual support, and a REST API—freely available as an open-source alternative to commercial ERP solutions.

2. **Dual XAI Framework (SHAP + LIME):** An `ExplainabilityService` that provides both instance-level and global explanations for three core classifiers, with interactive waterfall, beeswarm, and force-plot visualisations, persisted explanation history, and REST API access.

3. **XAI as Trust Calibration:** Empirical evidence that SHAP feature-importance analysis precisely tracks with underlying domain mechanics across all ML models. This finding demonstrates that **explainability is an essential component of responsible ML deployment**, allowing end-users to see the logic that powers accurate predictions.

4. **Cross-Method Comparison:** A systematic comparison of SHAP and LIME on three ERP-domain models showing 40% top-feature overlap with 100% directional agreement, providing practical guidance on using dual explanation methods for explanation triangulation.

5. **Multi-Tenant XAI Architecture:** A software architecture pattern for integrating XAI into multi-tenant SaaS applications, ensuring tenant-scoped model access, feature vector reuse, and explanation persistence.

7.2 Limitations

The following limitations are acknowledged:

1. **Synthetic and single-tenant evaluation:** The bulk of the evaluation was performed on one tenant (*Maines*) with synthetic and seeded operational data. While the architecture

supports multiple tenants and the Late-Payment iteration journey introduced a held-out month plus rolling-origin Prophet backtests, no production company has yet deployed the system on real day-to-day operations. The synthetic-to-real gap is discussed in Section 7.3 and is the dominant remaining external-validity threat.

2. **Class imbalance is mitigated, not eliminated:** Inverse-frequency sample weighting (He and Garcia, 2009) and PR-AUC / F1@τ^* / precision@k reporting (Saito and Rehmsmeier, 2015) are used in place of accuracy. SMOTE (Chawla et al., 2002) was prototyped inside cross-validation folds but did not improve hold-out F1 and is therefore not enabled by default. Severe imbalance (positive rate $< 5\%$) remains an open challenge.

3. **LIME stochasticity:** LIME explanations vary across runs due to random perturbation. While a fixed seed is used, production deployments should average several LIME iterations for stability.

4. **XAI coverage:** Only the three primary classifiers receive SHAP+LIME treatment. The Isolation Forest (Expense Anomaly) receives LIME-only explanations; the remaining models (Z-Score KPI Anomaly, Prophet, Cash Flow rule-based, attendance Z-Score) have no post-hoc attribution layer.

5. **Drift monitoring is implemented but unproven at scale:** The system computes Kolmogorov–Smirnov drift on numerical features, Population Stability Index (PSI) on categorical features (Wu and Olson, 2019), ADWIN concept drift (Bifet and Gavaldà, 2007) on the realised-performance signal, and a leakage-regression check that re-runs the shuffled-label test on the most recent training window. These signals are reported in a per-tenant nightly health-score Celery job, but their false-alarm and miss rates have not been characterised on a production deployment.

6. **User study absent:** No formal usability study was conducted to evaluate whether business users can interpret and act on the SHAP/LIME explanations. The visualisations are designed to be intuitive, but empirical validation with human participants is needed.

7.3 Future Work

The following directions are proposed for future enhancement:

1. **Real-world validation roadmap:** A four-stage plan to bridge the synthetic-to-real gap – (i) A/B exposure flagging on a single pilot tenant, (ii) champion-vs-challenger shadow scoring, (iii) cross-tenant generalisation testing on at least three industrially

distinct tenants, and (iv) controlled noise-injection experiments to characterise robustness. Stages (i)–(ii) are partially implemented in the model registry; stages (iii)–(iv) await live tenants.

2. **Counterfactual explanations:** Implement "what-if" explanations that show the minimum feature changes needed to flip a prediction. Particularly actionable for the Late-Payment and Attrition models.

3. **Attention-based explanations for time series:** Extend the XAI framework to the Demand Forecast (Prophet) model using temporal SHAP decomposition.

4. **Closed-loop drift response:** Use the existing KS/PSI/ADWIN/leakage-regression health score to automatically demote a deployed model to shadow mode and trigger retraining once drift exceeds a per-tenant threshold.

5. **User study:** Conduct a controlled experiment with ERP users (finance officers, HR managers, sales teams) to measure the impact of XAI explanations on decision quality, trust calibration and adoption.

6. **Regulatory compliance dashboard:** Aggregate explanation logs, generate model cards (dataset, features, metrics, known biases), and export audit-ready PDFs for regulatory submission.

7. **Deep-learning explainability:** If neural models (e.g. LSTMs for time-series) are introduced, integrate DeepSHAP or Integrated Gradients.

7.4 Concluding Remarks

This thesis set out to answer whether Explainable AI methods can enhance the trustworthiness, transparency, and quality of ML models embedded in enterprise resource planning systems. The answer, supported by empirical evidence, is unequivocally *yes*. The XAI framework successfully delivers user-facing transparency via intuitive waterfall plots, feature rankings, and natural-language impact descriptions, proving that complex ML logic can be deciphered for non-technical stakeholders.

This finding reinforces the position of Lundberg et al. (2020) and Rudin (2019) that explainability should be a *first-class requirement* in any ML deployment pipeline, not an optional post-hoc report. For the ERP domain—where ML predictions drive credit decisions, retention interventions, and resource allocation—the alignment of AI decisions with human intuition is paramount to maintaining user trust and system reliability.

The dual SHAP+LIME approach provides a practical "explanation triangulation" methodology that balances theoretical rigour (SHAP's Shapley-value guarantees) with accessibility

(LIME's intuitive linear weights), enabling both developers and business users to engage with model reasoning at their appropriate level of abstraction.

Bibliography

Al-Kraiji, A. and Al-Saedi, K. (2020). Challenges of ERP localization in Arabic-speaking countries: A systematic review. In *Proceedings of the International Conference on Information Systems*, pages 1–15.

Al-Majali, D. (2016). Antecedents of IT-business strategic alignment factors and its impact on organizational performance. *International Review of Management and Business Research*, 5(4):1290–1301.

Arrieta, A. B., Díaz-Rodríguez, N., Del Ser, J., Bennetot, A., Tabik, S., Barbado, A., Garcia, S., Gil-Lopez, S., Molina, D., Benjamins, R., Chatila, R., and Herrera, F. (2020). Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. *Information Fusion*, 58:82–115.

Bezemer, C.-P. and Zaidman, A. (2010). Multi-tenant SaaS applications: Maintenance dream or nightmare? *Proceedings of the Joint ERCIM Workshop on Software Evolution and International Workshop on Principles of Software Evolution*, pages 88–92.

Bifet, A. and Gavaldà, R. (2007). Learning from time-changing data with adaptive windowing. In *Proceedings of the SIAM International Conference on Data Mining*, pages 443–448.

Breck, E., Cai, S., Nielsen, E., Salib, M., and Sculley, D. (2017). The ML test score: A rubric for ML production readiness and technical debt reduction. In *Proceedings of the IEEE International Conference on Big Data*, pages 1123–1132.

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1):5–32.

Chawla, N. V., Bowyer, K. W., Hall, L. O., and Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357.

Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794.

Chong, F. and Carraro, G. (2006). Architecture strategies for catching the long tail. *Microsoft Developer Network (MSDN)*.

Davenport, T. H. and Ronanki, R. (2018). *Artificial Intelligence for the Real World*, volume 96.

Doshi-Velez, F. and Kim, B. (2017). Towards a rigorous science of interpretable machine learning. *arXiv preprint arXiv:1702.08608*.

European Commission High-Level Expert Group on AI (2019). Ethics guidelines for trustworthy AI. *European Commission*. https://ec.europa.eu/digital-single-market/en/news/ethics-guidelines-trustworthy-ai.

Fildes, R., Ma, S., and Kolassa, S. (2022). Retail forecasting: Research and practice. *International Journal of Forecasting*, 38(4):1283–1318.

Grand View Research (2023). Enterprise Resource Planning Market Size, Share & Trends Analysis Report. Technical report, Grand View Research. Report ID: GVR-1-68038-955-0.

Guidotti, R., Monreale, A., Ruggieri, S., Turini, F., Giannotti, F., and Pedreschi, D. (2018). A survey of methods for explaining black box models. *ACM Computing Surveys*, 51(5):1–42.

He, H. and Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284.

IBM (2017). IBM HR Analytics Employee Attrition & Performance Dataset. Kaggle. https://www.kaggle.com/pavansubhasht/ibm-hr-analytics-attrition-dataset.

Jacobs, F. R. and Weston, F. C. (2007). Enterprise resource planning (ERP) — a brief history. *Journal of Operations Management*, 25(2):357–363.

Krafzig, D., Banke, K., and Slama, D. (2005). *Enterprise SOA: Service-Oriented Architecture Best Practices*. Prentice Hall, Upper Saddle River, NJ.

Lee, J., Siau, K., and Hong, S. (2003). Enterprise integration with ERP and EAI. *Communications of the ACM*, 46(2):54–60.

Lipton, Z. C. (2018). The mythos of model interpretability. *Queue*, 16(3):31–57.

Liu, F. T., Ting, K. M., and Zhou, Z.-H. (2008). Isolation forest. In *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, pages 413–422.

Lundberg, S. M., Erion, G., Chen, H., DeGrave, A., Prutkin, J. M., Nair, B., Katz, R., Himmelfarb, J., Bansal, N., and Lee, S.-I. (2020). From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2(1):56–67.

Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pages 4765–4774.

Molnar, C. (2022). *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. Independently Published, 2nd edition. https://christophm.github.io/interpretable-ml-book/.

Monk, E. and Wagner, B. (2013). *Concepts in Enterprise Resource Planning*. Cengage Learning, Boston, MA, 4th edition.

Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, Sebastopol, CA.

Niculescu-Mizil, A. and Caruana, R. (2005). Predicting good probabilities with supervised learning. *Proceedings of the 22nd International Conference on Machine Learning*, pages 625–632.

Ojala, M. and Garriga, G. C. (2010). Permutation tests for studying classifier performance. *Journal of Machine Learning Research*, 11:1833–1863.

Olist (2018). Brazilian e-commerce public dataset by olist. Kaggle. https://www.kaggle.com/olistbr/brazilian-ecommerce.

Panorama Consulting Group (2023). 2023 ERP Report: Clash of the Titans. Technical report, Panorama Consulting Group, Denver, CO.

Provost, F. and Fawcett, T. (2013). *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media, Sebastopol, CA.

Ribeiro, M. T., Singh, S., and Guestrin, C. (2016). "Why should I trust you?": Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1135–1144.

Rudin, C. (2019). Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5):206–215.

Saito, T. and Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3):e0118432.

Schwaber, K. and Sutherland, J. (2020). *The Scrum Guide: The Definitive Guide to Scrum — The Rules of the Game*. Scrum.org.

Shapley, L. S. (1953). A value for n-person games. In Kuhn, H. W. and Tucker, A. W., editors, *Contributions to the Theory of Games*, volume 2, pages 307–317. Princeton University Press.

Shewhart, W. A. (1931). *Economic Control of Quality of Manufactured Product*. D. Van Nostrand Company, New York.

Slack, D., Hilgard, S., Jia, E., Singh, S., and Lakkaraju, H. (2020). Fooling LIME and SHAP: Adversarial attacks on post hoc explanation methods. In *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society*, pages 180–186.

Society for Human Resource Management (2022). 2022 Employee Benefits Survey. Technical report, SHRM, Alexandria, VA.

Taylor, S. J. and Letham, B. (2018). Forecasting at scale. *The American Statistician*, 72(1):37–45.

Wu, D. and Olson, D. L. (2019). Population stability index and its use in credit risk monitoring. Technical report, SSRN Working Paper Series.